STOPWATCH · WATCH · TIMER

Basys-3 FPGA 3-in-1 디지털 시계 — 설계 · 시뮬레이션 검증

STOPWATCH WATCH TIMER

2026. 10. 19

팀원 정안성 · 정보윤 · 이종혁

Contents

Basys-3 FPGA 위에 세 가지 시간 기능을 하나의 Top 으로 통합,
Top 테스트벤치로 버튼 → FND 전체 경로를 검증

01 프로젝트 개요
팀 구성 · 기능별 목표 · 입출력 사양

02 시스템 구조
Top 블록도 · 코어 모듈 · 기능별 흐름

03 기능별 설명 · 시뮬레이션 검증
기능 설명 → 시뮬레이션 (TB 24 개 전부) : 03-1 시계 · 03-2 스톱워치 · 03-3 타이머 · 03-4 공통 · 03-5 tick · 타이밍 · bug 주입

04 트러블 슈팅
현상 → 원인 → 해결 → 적용 결과

05 결론 · 고찰
구현 결과 · 결론 · 고찰 · 개선 방향

06 시연 영상
보드 동작

WATCH

12:00 시작 · SW2 자리 단위 설정 · 12/24 h

STOPWATCH

시작 · 정지 · 초기화 · 업/다운 + LAP

TIMER

+10 초 · +1 분 · +10 분 · 0 도달 알람

01 프로젝트 개요

팀 구성 · 역할 분담

결론 **기능 단위로 나눠 각자 설계 · 자기 TB 작성 → Top TB 13 개로 통합 검증**

정

정안성

FND · Top 검증

1. FND 컨트롤러 설정 자리 점멸 기능 추가
2. Top 모듈 전체 TB 시나리오 구성
3. 시연 시나리오 구성

FND · TOP TB

정

정보윤

Timer · Watch 설계

1. Timer · Watch 모듈 설계
2. 자원 최적화 — tick 구조 4 종 비교 (LUT · FF)
3. 타이머 검증 · 트러블 슈팅 1

TIMER · WATCH · 자원

이

이종혁

Stopwatch · LAP

1. Stopwatch 기능 동작 확인 (R · L · U)
2. D 버튼 LAP time 추가
3. LAP 저장 · 해제 파형 검증

STOPWATCH · LAP

기능별 목표 · 핵심 결과

결론 **목표 기능 전부 구현 — Top TB 13 개 · 검사 447 개 PASS · 보드 시연 확인 · 칩 자원 1.8 %**

기능	입력	목표 동작	검증 (Top TB)
시계	SW2 설정 · L/R 커서 · U/D ±	12:00 시작 · 자리 단위 설정 · 설정 자리 0.5 s 점멸 · 12/24 h	TB 4 개 · 검사 264 PASS
스톱워치	R 시작/정지 · L 초기화 · U 업/다운 · D LAP	1/100 초 측정 · 0 ↔ 23:59:59.99 · LAP 중에도 내부 측정 계속	TB 3 개 · 검사 53 PASS
타이머 (새 기능)	U +10 초 · R +1 분 · D +10 분 · L 시작/취소	상한 59:59 · 0 도달 → LED 16 개 0.5 s 점멸 알람	TB 3 개 · 검사 73 PASS
공통	버튼 4 개 공유 · SW4 > SW0 화면 선택	디바운스 2 ms · FND 스캔 2 ms · 버튼 동시 입력 무시	TB 3 개 · 검사 57 PASS

447 / 447

Top TB 13 개 · 검사 PASS

1.8 %

LUT 384 / 20,800 · FF 228 / 41,600 (0.5 %)

+3.440 ns

WNS @ 100 MHz · 래치 0

10 s + 20 ns

10 초 타이머 실측 (2 클럭 고정 지연)

Note. 수치 출처 — Vivado 2022.2 구현 결과 (xc7a35t) · xsim Top TB [SUMMARY] 자동 판정 (부록 A)

입출력 사양 — 클럭 · 비트 폭 · 입출력을 왜 이렇게 정했나

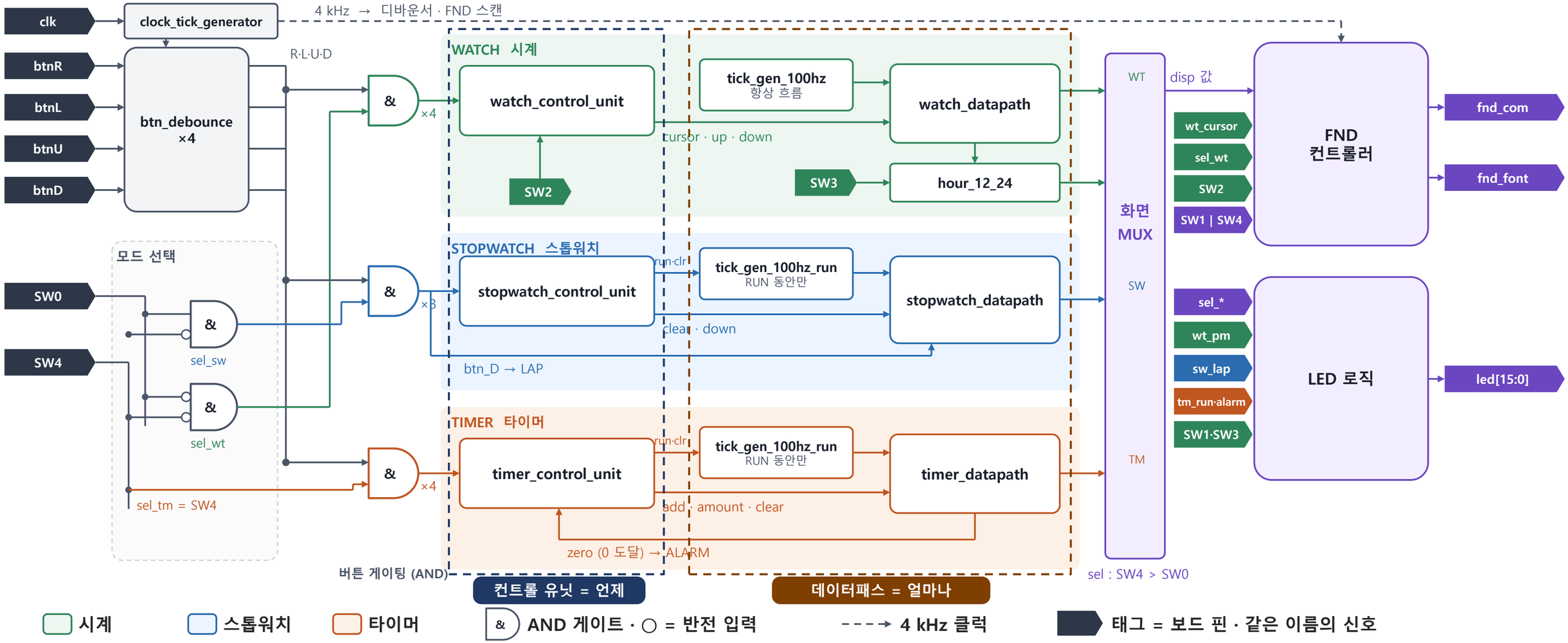
결론 **주파수는 표시 단위 · 버튼 튐 · FND 깜빡임에서 정함 — 100 Hz = 1/100 초 표시, 4 kHz = 2 ms 디바운스 · 2 ms 스캔**

항목	값	이유
시스템 클럭	100 MHz (10 ns)	Basys-3 보드 발진기 (W5 핀) — 모든 레지스터를 이 클럭 하나로 동작
표시 tick	100 Hz (10 ms)	화면 최소 단위가 **1/100 초** → 10 ms 마다 1 증가 (100 MHz 를 1,000,000 클럭 세어 펄스 1 개)
공통 tick	4 kHz (250 µs)	디바운서 8 샘플 × 250 µs = **2 ms** 연속 눌림만 인정 · FND 8 칸 × 250 µs = **2 ms** 한 바퀴 (Basys-3 권장 1 ~ 16 ms)
시간 비트 폭	1/100 초 7 · 초 6 · 분 6 · 시 5	0 ~ 99 · 0 ~ 59 · 0 ~ 23 을 담는 최소 비트 ($2^7 = 128 \geq 100$ · $2^6 = 64 \geq 60$ · $2^5 = 32 \geq 24$)
버튼	BTN R · L · U · D	4 개를 세 기능이 공유 → 지금 화면 기능에만 전달 (게이팅), 동시에 2 개 이상이면 무시
스위치	SW0 · SW4 화면 / SW1 표시 / SW2 설정 / SW3 12·24	화면 3 개를 스위치 2 개로: **SW4 = 1 이면 SW0 무시** → 남는 조합 없이 화면 하나로 정해짐
출력	FND 4 자리 · LED 16 개	FND 4 자리가 세그먼트 선을 공유 → 한 자리씩 스캔 / LED = 화면 · 모드 표시 + 알람 점멸

● 02 시스템 구조

Top 블록 다이어그램 — 버튼에서 FND · LED 까지

결론 **버튼 4 개 → 게이팅으로 한 기능에만 → 기능마다 컨트롤 유닛(언제) + 데이터패스(얼마나) → 화면 MUX 로 FND 하나 공유**



02 시스템 구조

코어 모듈 — 무엇을 하고 왜 필요한가

결론 **기능마다 컨트롤 유닛(언제) + 데이터패스(얼마나) 로 나눠 세 기능이 같은 구조 — 분주기 · FND 는 하나만 두고 공유**

모듈	하는 일	왜 필요
btn_debounce × 4	4 kHz 로 8 번 연속 1 → 1 클럭 펄스	버튼 접점이 튀어 한 번 눌러도 여러 번 들어옴
버튼 게이팅 (Top)	화면 선택 · 동시 입력 검사 → 한 기능에만	버튼 4 개를 세 기능이 공유
clock_tick_generator	100 MHz → 4 kHz 펄스	디바운서 · FND 스캔이 같은 박자 → 분주기 1 개
tick_gen_100hz (_run)	10 ms 마다 1 클럭 tick	1/100 초 단위. _run 은 누른 순간 0 부터 (TS 1)
***_control_unit (FSM)**	상태 · 버튼 → run · clear · add 등 제어 신호	언제 무엇을 할지(버튼 규칙)를 숫자와 분리
***_datapath**	1/100 초 · 초 · 분 · 시 카운터, 더하기 · 빌림	얼마나(숫자)만 계산 → 같은 카운터 틀 재사용
화면 MUX (Top)	SW4 > SW0 순서로 표시할 값 선택	FND 하나를 세 기능이 공유
FND 컨트롤러	자리 분리 · 4 자리 스캔 · 점 · 설정 점멸	4 자리가 세그먼트 선을 공유

컨트롤 유닛 / 데이터패스로 나눈 이유

① 버튼 규칙(FSM)을 바꿔도 카운터는 그대로

② 블록마다 따로 시험 · 디버깅 — 파형에서 c_state 와 값을 나눠 봄

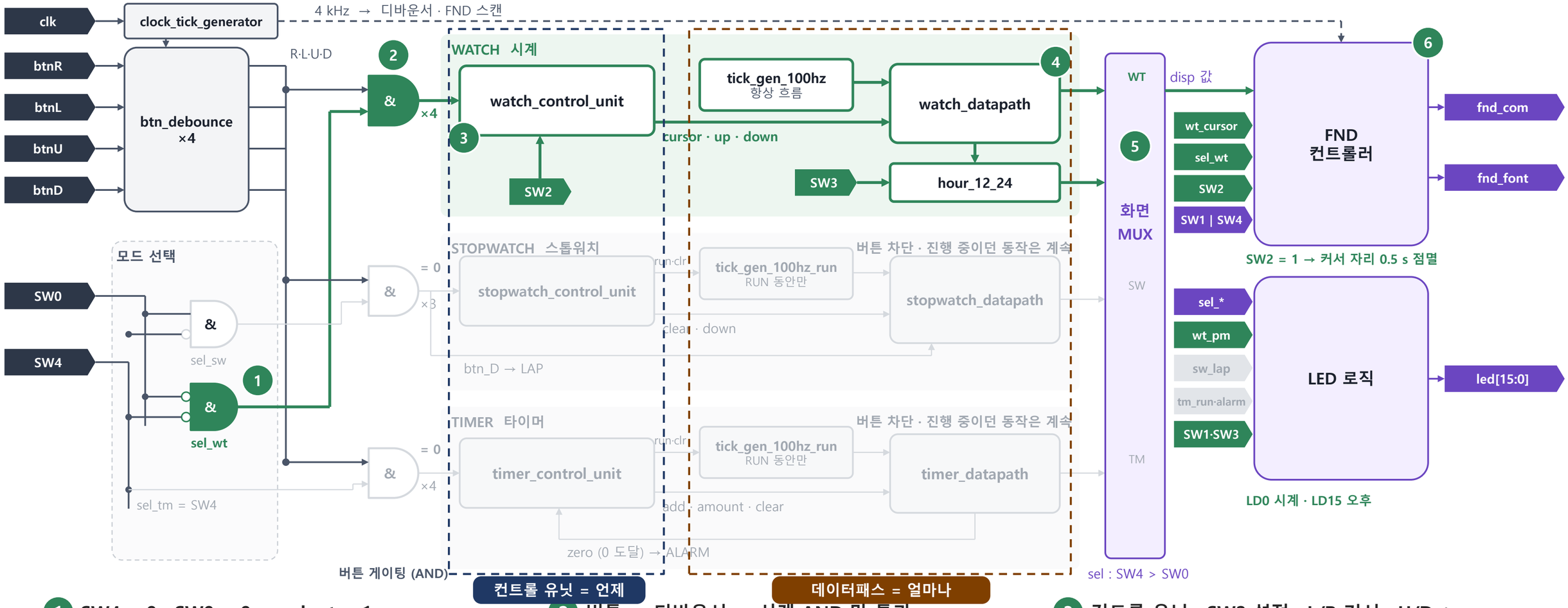
③ 세 기능이 같은 구조 → 같은 방식으로 읽고 고침

④ 타이머 추가 때 기존 카운터 틀 재사용

02 시스템 구조 · 기능별 흐름

시계 흐름 — SW4 = 0 · SW0 = 0

결론 **시계는 100 Hz 로 항상 흐르고, 버튼은 SW2 설정 중에만 커서 자리 하나를 바꿈 → 설정 중에도 시각은 멈추지 않음**

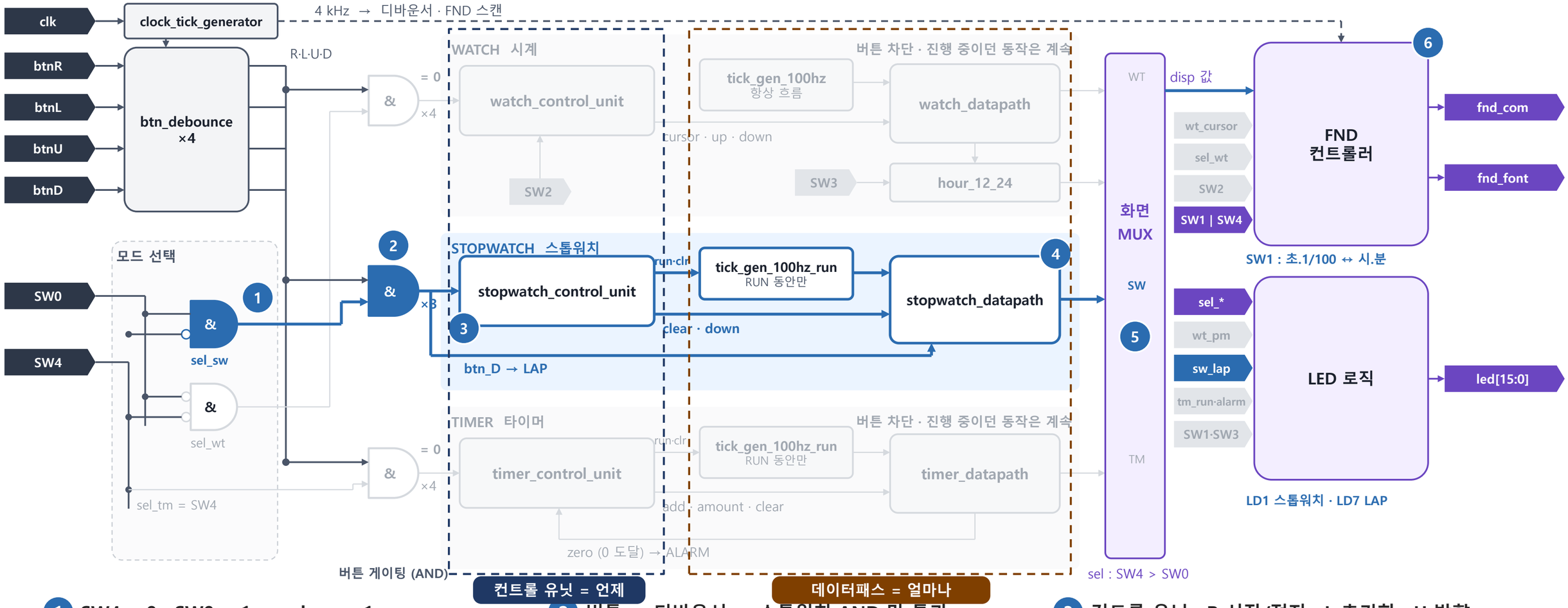


1. SW4 = 0 · SW0 = 0 → sel_wt = 1
2. 버튼 → 디바운서 → 시계 AND 만 통과
3. 컨트롤 유닛 : SW2 설정 · L/R 커서 · U/D ±
4. 데이터패스 : 100 Hz 로 항상 흐름 · 커서 자리 ±
5. 12/24 변환 (SW3) → 화면 MUX 가 시계 값 선택
6. FND 표시 · 설정 자리 점멸 · LD0 · LD15

02 시스템 구조 · 기능별 흐름

스톱워치 흐름 — SW4 = 0 · SW0 = 1

결론 **스톱워치는 RUN 동안만 자기 tick 으로 세고, D(LAP) 는 화면 값만 고정 → 측정은 멈추지 않음**

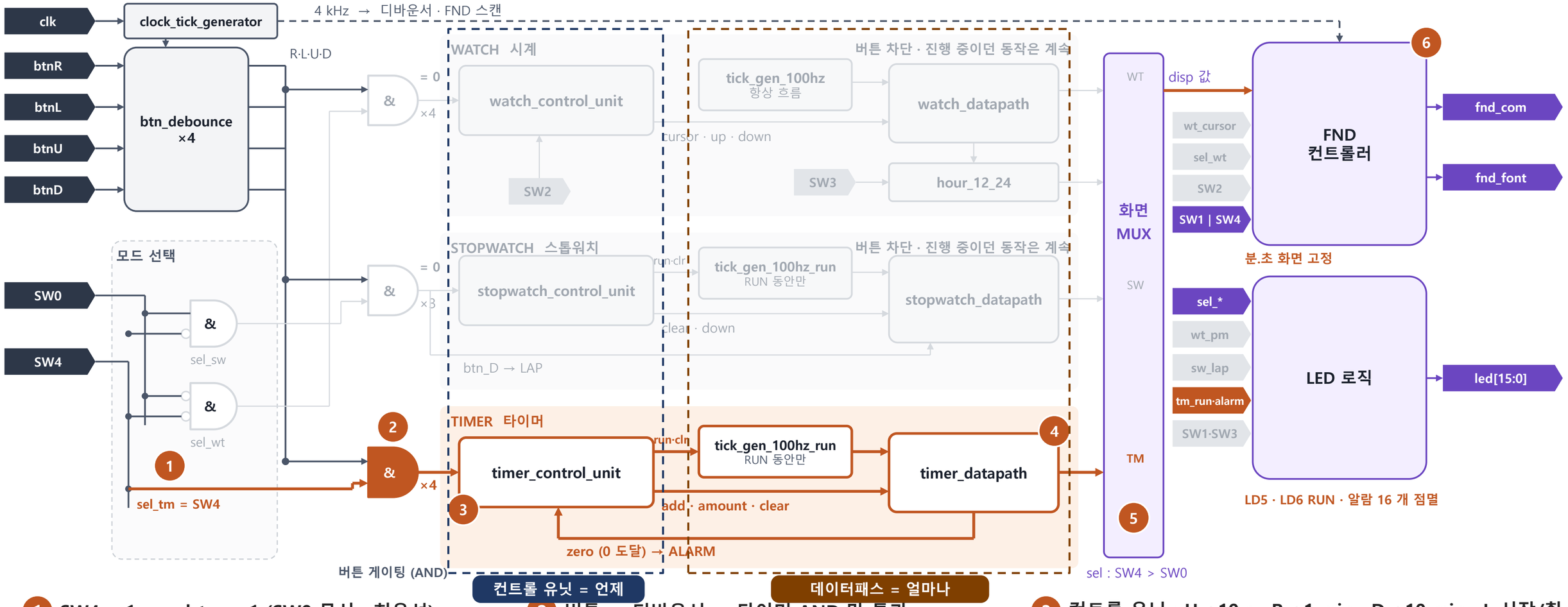


1. SW4 = 0 · SW0 = 1 → sel_sw = 1
2. 버튼 → 디바운서 → 스톱워치 AND 만 통과
3. 컨트롤 유닛 : R 시작/정지 · L 초기화 · U 방향
4. 전용 tick : RUN 동안만 셈 · D 버튼 = LAP
5. 화면 MUX 가 스톱워치 값 선택 (SW1 화면 전환)
6. FND 표시 · LD1 · LD7 LAP

02 시스템 구조 · 기능별 흐름

타이머 흐름 — SW4 = 1 (최우선)

결론 **SW4 = 1 이면 타이머가 최우선 — 다른 화면에서도 타이머는 계속 줄고 0 이면 LED 16 개로 알림**



1. SW4 = 1 → sel_tm = 1 (SW0 무시 · 최우선)
2. 버튼 → 디바운서 → 타이머 AND 만 통과
3. 컨트롤 유닛 : U +10 s · R +1 min · D +10 min · L 시작/취소
4. 전용 tick 다운카운트 → 0 도달 zero → ALARM
5. 화면 MUX 가 타이머 값 선택 (분.초 고정)
6. FND 표시 · LD5 · LD6 RUN · 알람 때 LED 16 개 점멸

03

기능별 설명 및 시뮬레이션 검증

공통 검증 방법
· Top 모듈을 넣은 TB · 100 MHz 그대로 (주파수 축소 없음)
· 시나리오 표 (입력 → 확인 신호 → 예상 결과) → 파형 · 콘솔 판정

03-1 시계

기능 설명 (조작법) → 시뮬레이션 P1 ~ P8 + 1초 · 자정 · 12/24 · 버튼 16 조합

03-2 스톱워치

기능 설명 (조작법) → 시뮬레이션 5 개 · R · L · U 기본 동작 + D 버튼 LAP

03-3 타이머

기능 설명 (조작법 · FSM · 구조)
→ 시뮬레이션 · 더하기 · 59:59 상한 · 카운트다운 · 알람

● 03-1 시계 · 기능 설명

시계 조작법 — 버튼 · 스위치별 동작

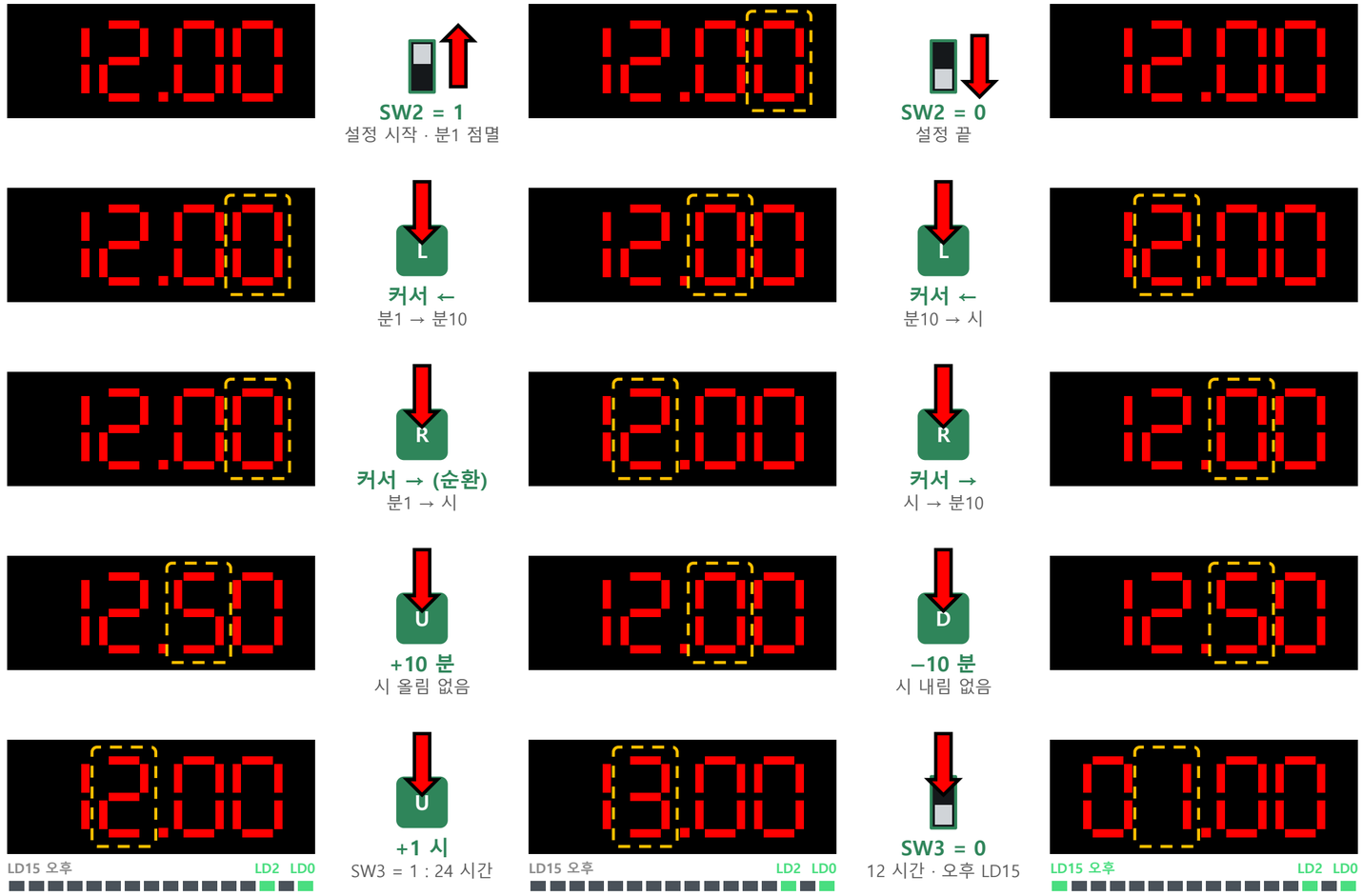
결론 **버튼 4 개 + SW2 로 시 · 분 설정 — 설정 자리만 0.5 s 점멸, 다른 자리 · 시각 흐름은 그대로**

WATCH SW4 = 0 · SW0 = 0 → LD0



#	입력	동작
	SW2	1 : 설정 모드 (분 1 의 자리부터) · 0 : 끝
①②	BTNL / R	설정 자리 이동 시 ↔ 분10 ↔ 분1
③	BTNU	그 자리 + 1 (분10 자리 : + 10 분)
④	BTND	그 자리 – 1 (분10 자리 : – 10 분)
	SW3	0 : 12 시간 (오후 LD15) / 1 : 24 시간
	SW1	1 : 시.분 화면 — 설정 자리 점멸

화면 : SW1 = 1 → 시 . 분 (HH.MM) ■ = 설정 중인 자리 (0.5 s 점멸)



03-1 시계 · 시뮬레이션

시계 검증 시나리오 — 버튼 동작 · 점멸 (P1 ~ P8)

결론 **설정 진입 · 커서 · 자리별 경계 · 12 시간 표시 · 화면 분리 · 점멸 8 항목 모두 기대값과 일치 (tb_wt_1 ~ 4 검사 264 개 PASS)**

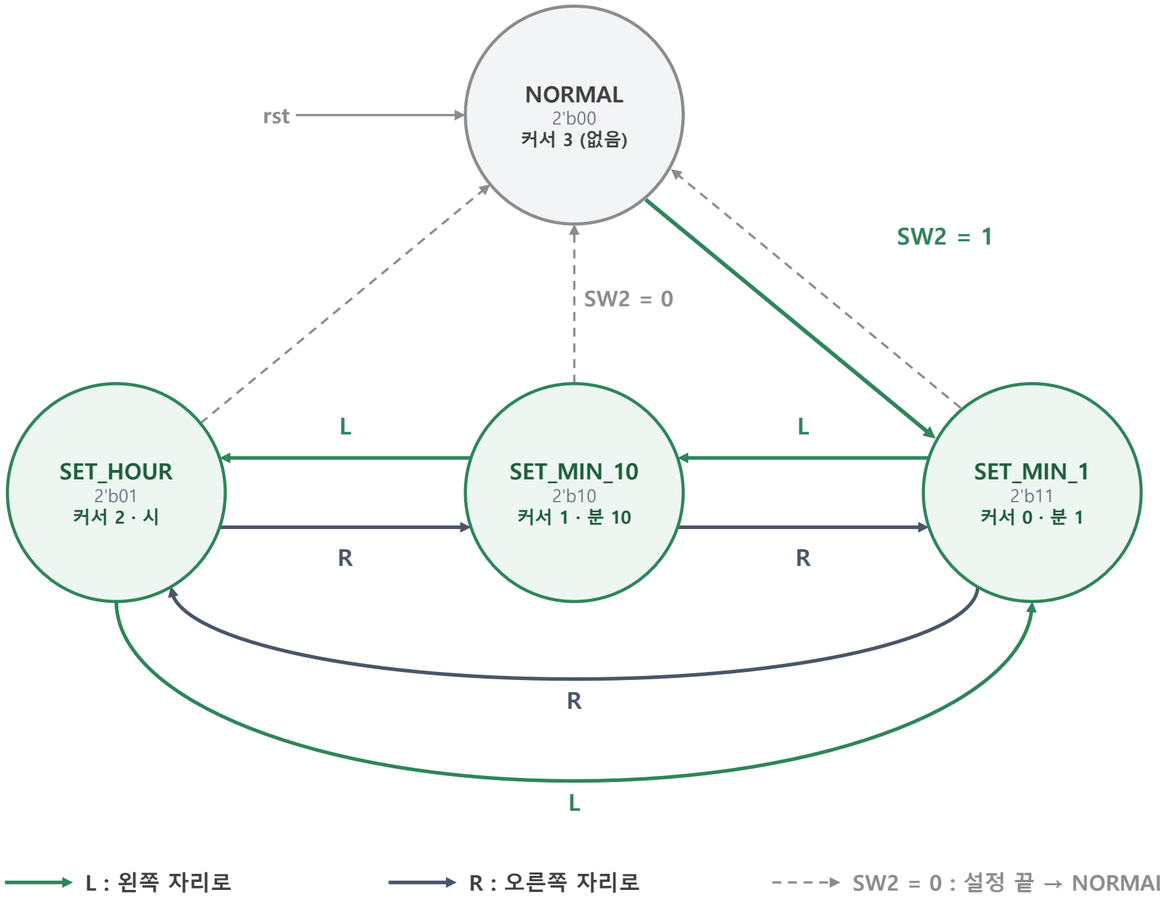
#	시나리오	입력 (Stimulus)	확인 신호	확인 결과
P1	**SW2 = 0 버튼 무시** 15 장	SW2 = 0, R · L · U · D 각 5 ms	c_state · cursor · tick	NORMAL · 커서 3 유지, tick 은 누른 뒤 1.75 ms
P2	**SW2 = 1 설정 진입** 16 장	SW2 0 → 1	n_state · c_state · cursor	다음 clk 에 SET_MIN_1, 커서 3 → 0
P3	**커서 이동 (L · R)** 17 ~ 19 장	L 3 번 → R 3 번	c_state · cursor · i_tick_10ms	L : 0 → 1 → 2 → 0 · R : 0 → 2 → 1 → 0
P4	**분 1 자리 경계** 20 장	12:59 주입 → U → D → D	o_wt_min · AN0 ~ AN3 BCD	분 59 → 00 → 59 → 58, 시 12 유지
P5	**분 10 자리 경계** 21 장	12:55 주입 → U → D → D	o_wt_min · AN0 ~ AN3 BCD	분 55 → 05 → 55 → 45, 1 자리 5 유지
P6	**시 경계 · 12 시간 표시** 22 장	SW3 = 0, 23:30 주입 → U → D	o_wt_hour · hour_disp · LED15	시 23 → 0 → 23, 표시 11 → 12 → 11, LED15 1 → 0 → 1
P7	**스톱워치 화면 버튼 분리** 23 장	SW2 = 1 · SW0 = 1, D 입력	o_sw_lap · 시계 c_state · o_wt_msec	LAP 저장, 시계 설정값 유지 · 시계는 계속 증가
P8	**커서 자리 점멸** 25 장 (원리 24 장)	12:34 · 커서 0 · 1 · 2, 1/100 초 49 → 50	o_blink_off · fnd_an · fnd_num	커서 자리 fnd_num 만 f, 나머지 자리 값 유지

03-1 시계 · 설계

시계 설정 FSM — SW2 와 커서 4 상태

결론

- SW2 = 1 → **SET_MIN_1** 진입 · SW2 = 0 → 어느 상태든 **NORMAL**
- 커서 = U · D 로 바꿀 자리 + 점멸할 자리 (0 분 1 · 1 분 10 · 2 시 · 3 없음)



버튼 입력이 없으면 지금 상태 유지 · U · D 는 상태를 바꾸지 않고 커서 자리 값만 ±

SW1 = 1 (시 . 분) 화면 노란 점선 = 0.5 s 점멸 자리



03-1 시계 · 시뮬레이션 버튼 동작

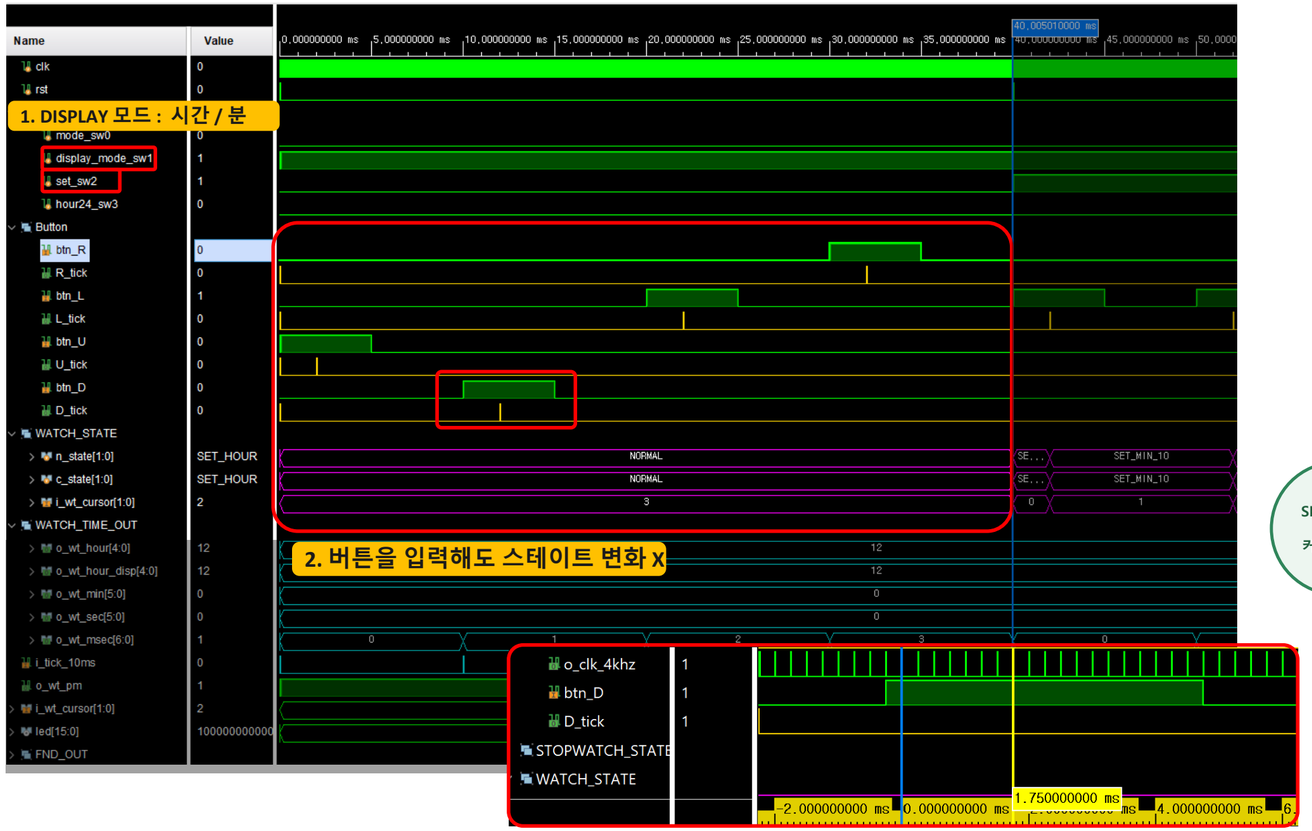
P1 SW2 = 0 — 버튼을 눌러도 상태 유지

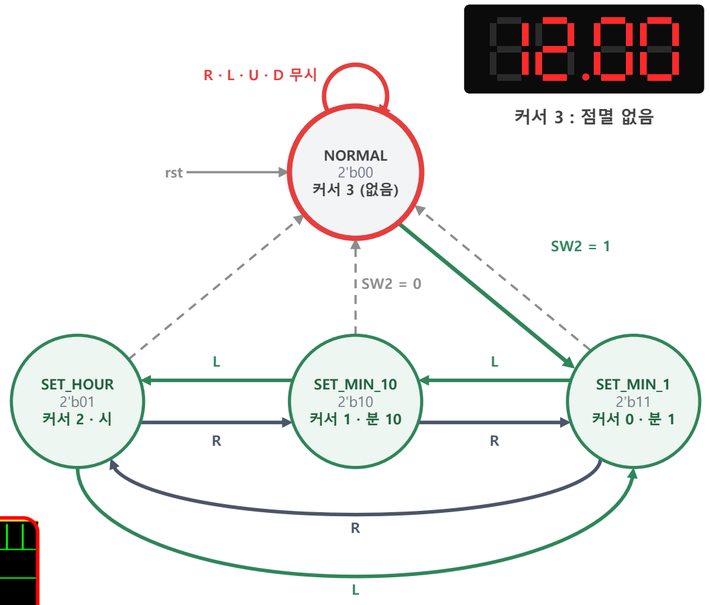
결론

- c_state **NORMAL** · 커서 **3** 유지 → SW2 = 0 이면 버튼 무시
- 4 kHz 로 8 번 연속 1 확인 → 누른 뒤 **1.75 ms** (7 주기) 에 tick 1 클럭

판별

- 설정 꺼짐(SW2 = 0) 에서 R · L · U · D 입력 → 상태 · 커서 변화 여부
- 버튼 입력 → tick 까지 디바운스 지연





커서 3 : 점멸 없음

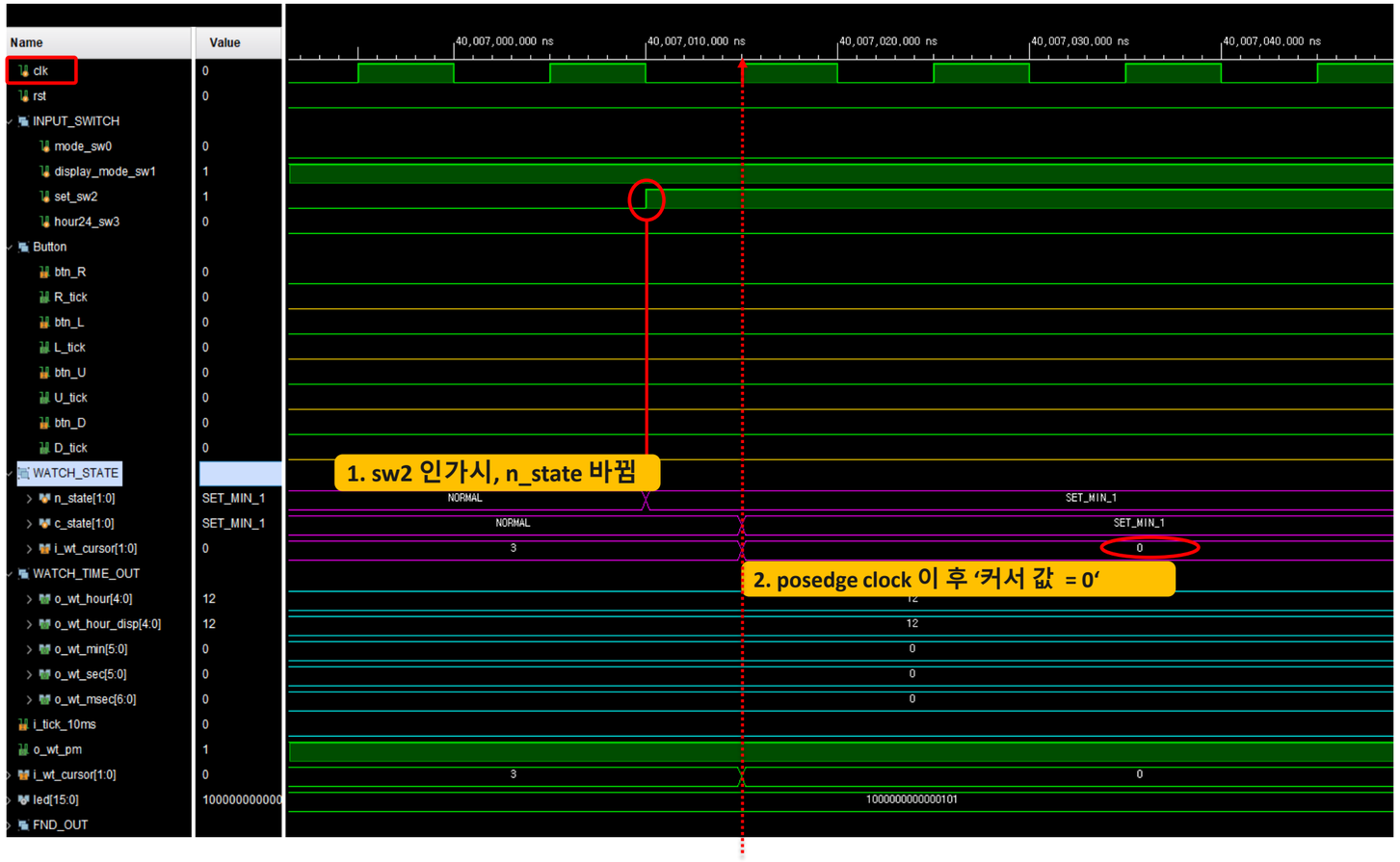
P2 SW2 = 1 — NORMAL → SET_MIN_1 진입

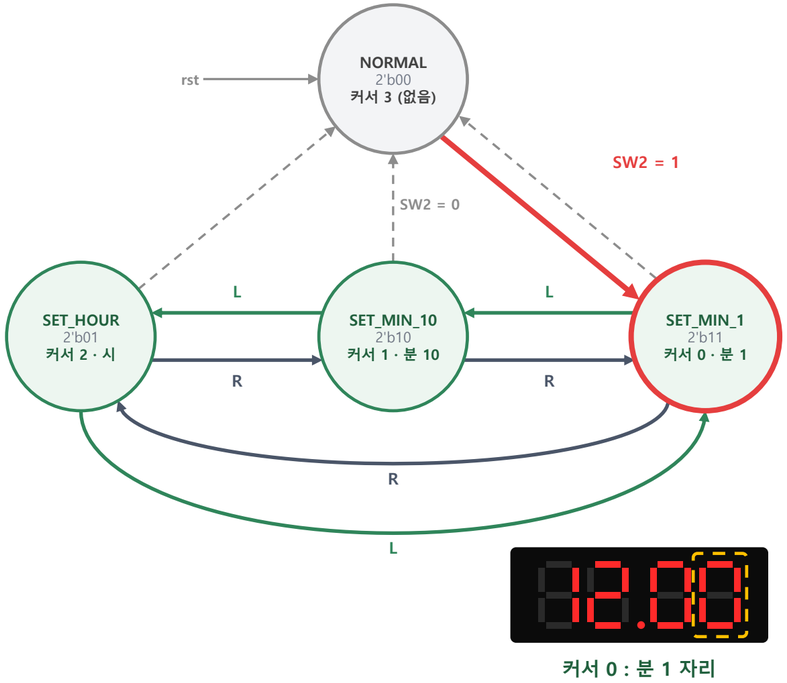
결론

- SW2 상승 즉시 n_state = **SET_MIN_1** (조합 논리)
- 다음 clk 상승에서 c_state 갱신 → 커서 **3 → 0** (분 1 자리부터 설정)

판별

- SW2 0 → 1 순간 n_state · c_state · 커서가 바뀌는 시점





커서 0 : 분 1 자리

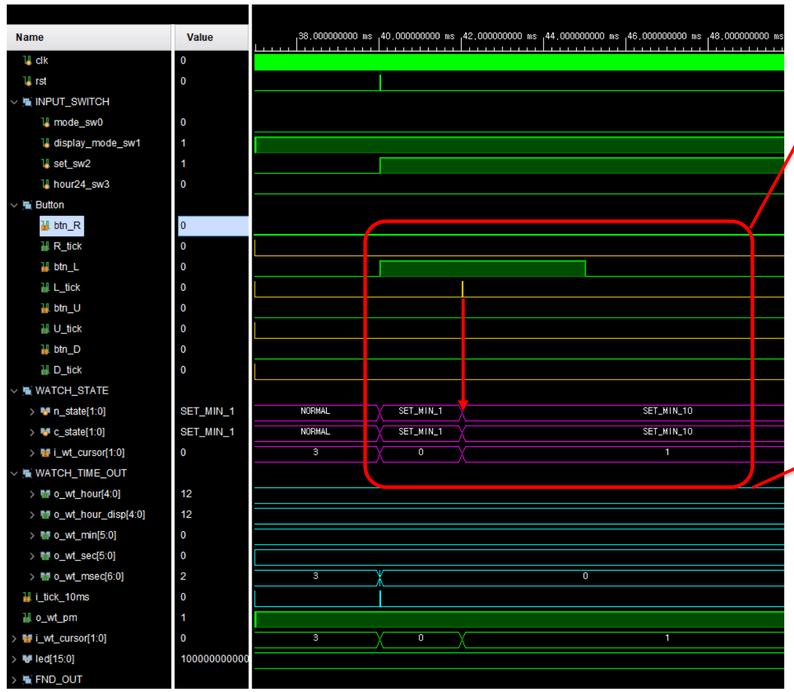
P3-1 L tick — 다음 clk 에 상태 · 커서 갱신

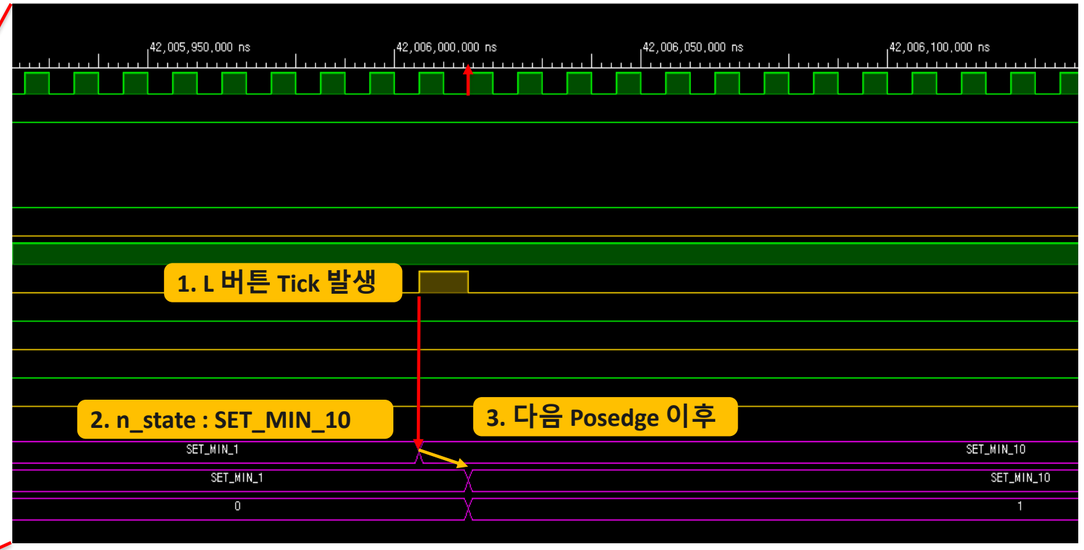
결론

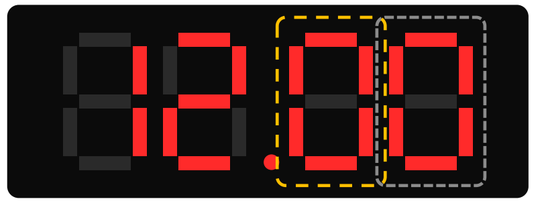
- tick 과 같은 클럭에 n_state = **SET_MIN_10**, 다음 clk 상승에서 c_state · 커서 **0 → 1**
- tick 은 1 클럭 펄스 → 버튼 1 번에 커서 1 칸 이동

판별

- L 버튼 tick 이 나온 뒤 상태 · 커서가 바뀌는 시점







노란 점선 = 지금 커서
회색 점선 = 이전 커서

L tick → 커서 0 → 1

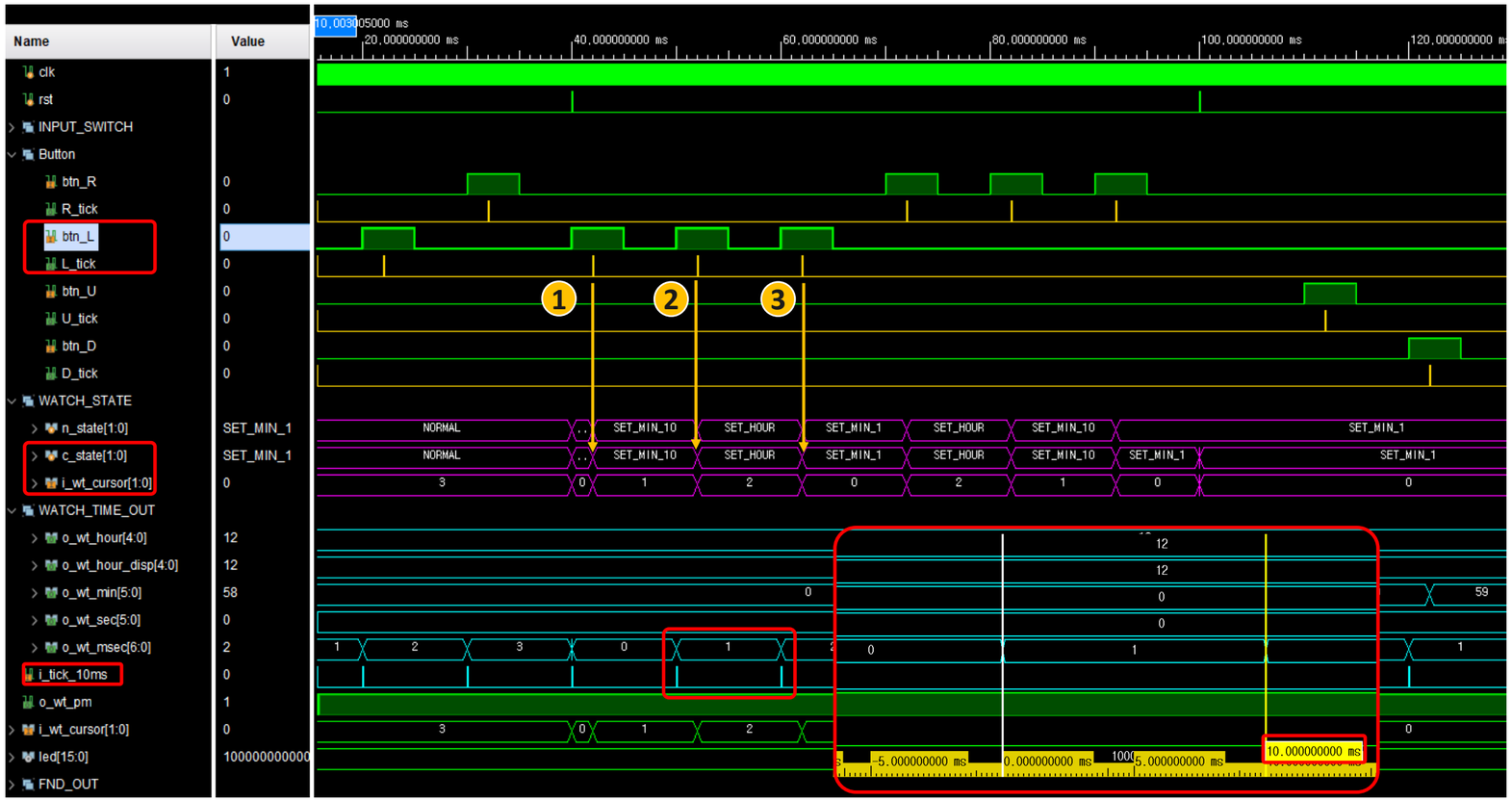
P3-2 L 3 번 — 커서 왼쪽 순환 (분 1 → 분 10 → 시)

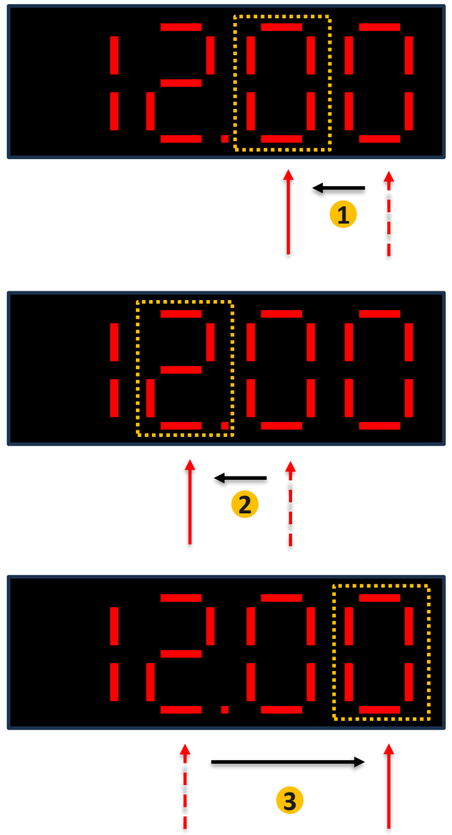
결론

- 커서 **0 → 1 → 2 → 0** : 시 다음은 분 1 자리로 돌아감
- i_tick_10ms **10 ms** 간격 → 설정 중에도 1/100 초 정상 증가

판별

- L tick 마다 상태 · 커서 이동 방향, 왼쪽 끝(시) 다음 위치
- 시계 1/100 초 박자 (i_tick_10ms) 간격





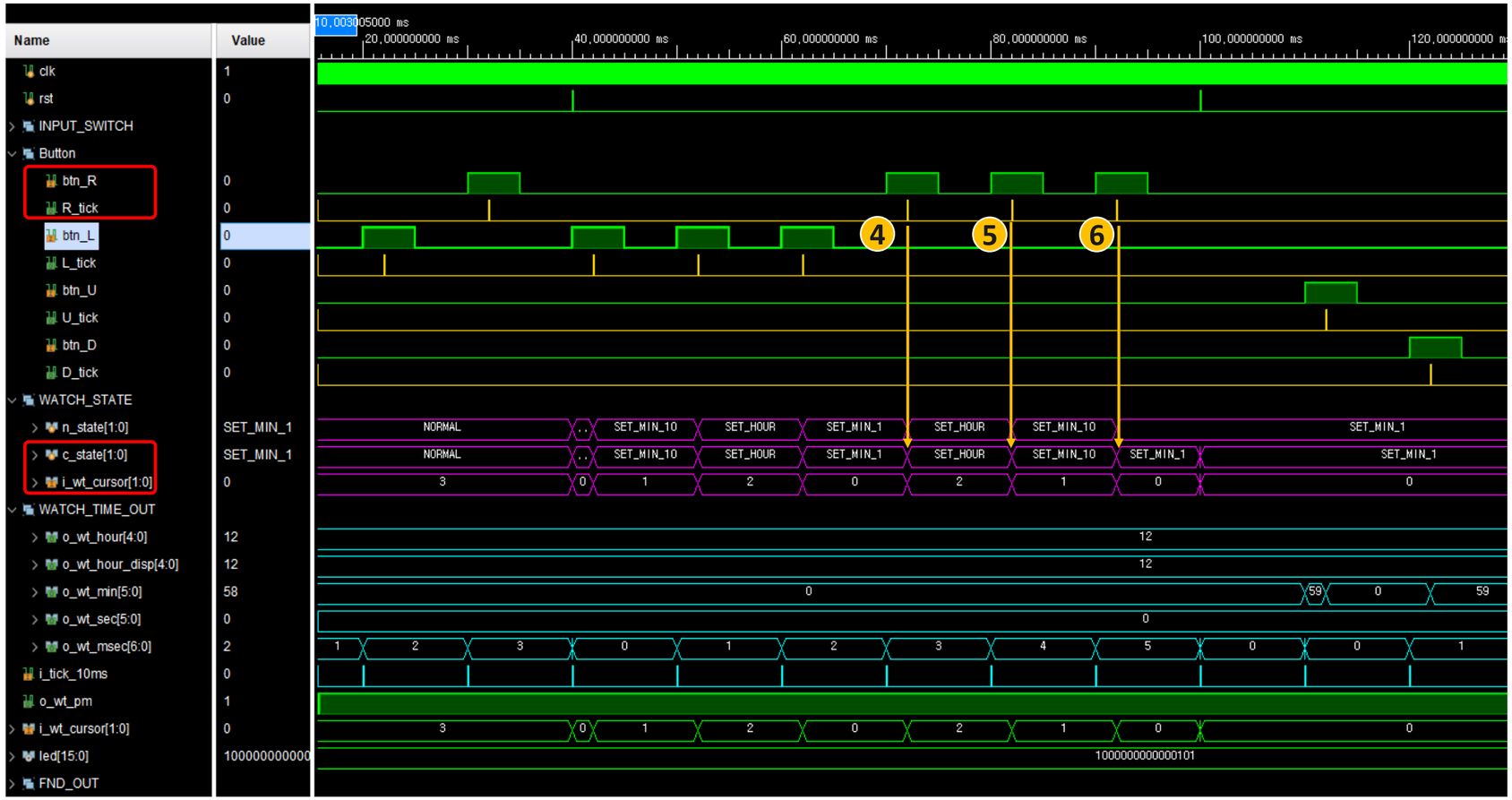
P3-3 R 3 번 — 커서 오른쪽 순환 (분 1 → 시 → 분 10)

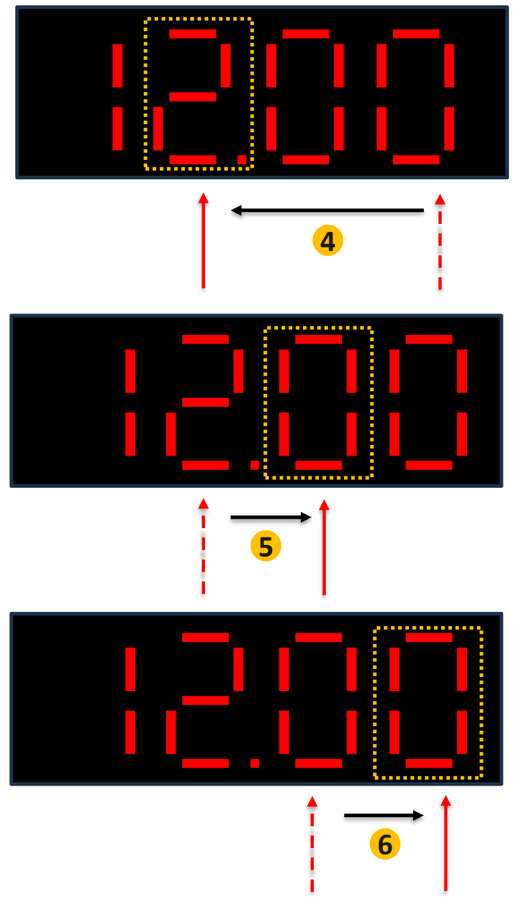
결론

- 커서 **0 → 2 → 1 → 0** : 분 1 다음은 시 자리로 돌아감 (L 과 반대 방향)
- L · R 모두 버튼 1 번에 1 칸, 3 자리 사이 순환 정상

판별

- R tick 마다 상태 · 커서 이동 방향, 오른쪽 끝(분 1) 다음 위치





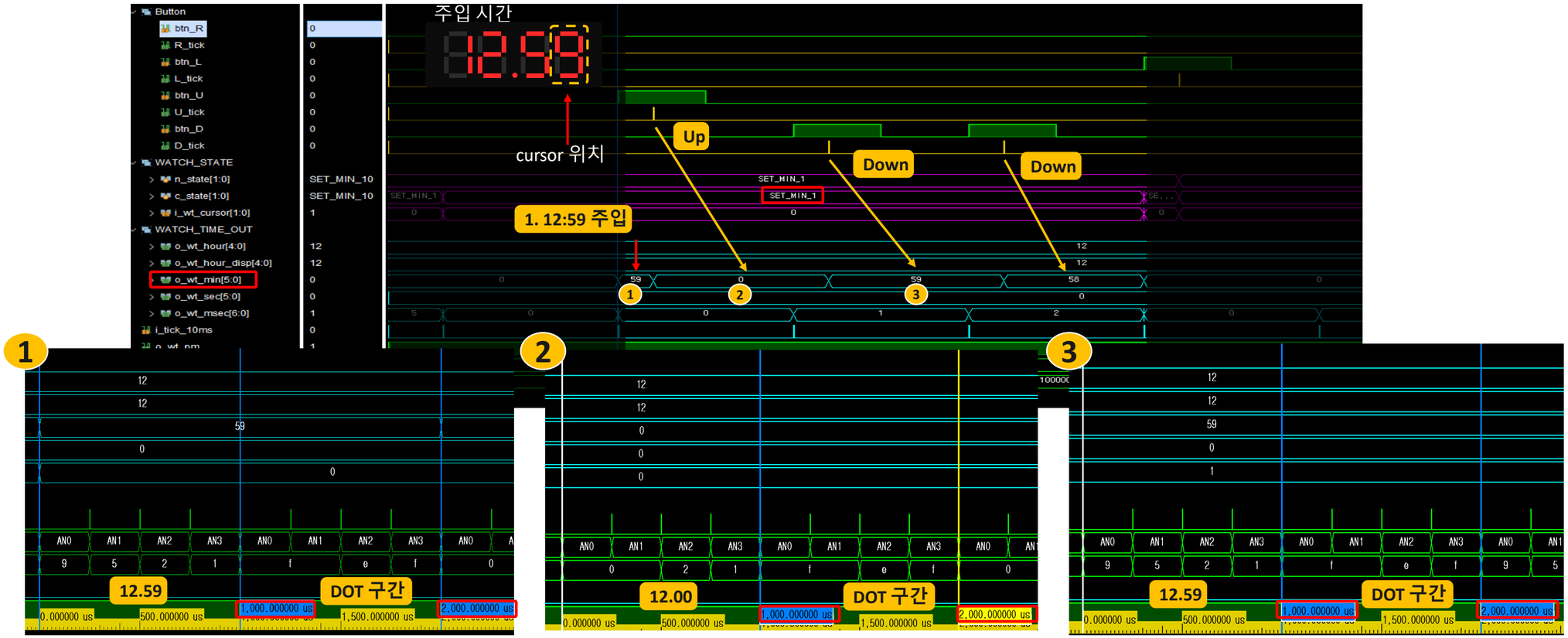
P4 분 1 자리 경계 — 12:59 에서 U → D → D

결론

- 분 **59 → 00 → 59 → 58**, 시 12 그대로 → 그 자리만 순환
- AN0 ~ AN3 = 9 · 5 · 2 · 1 (**12.59**), 자리당 0.25 ms → 숫자 1 ms + 점 1 ms 반복

판별

- 분 59 에서 +1, 00 에서 −1 할 때 시로 올림 · 내림이 없는지
- FND 자리별 BCD 값 · 스캔 주기



03-1 시계 · 시뮬레이션 버튼 동작

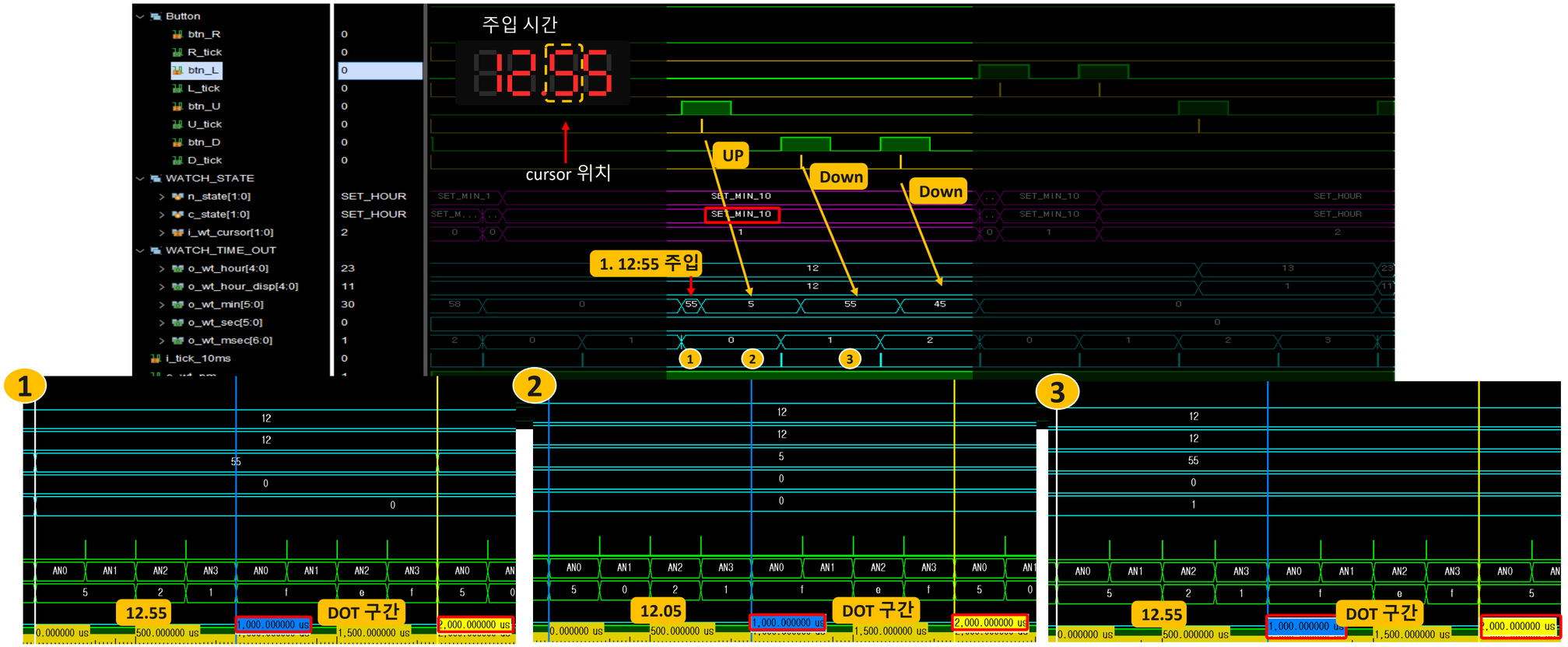
P5 분 10 자리 경계 — 12:55 에서 U → D → D

결론

- 분 **55 → 05 → 55 → 45** : 60 으로 순환하고 1 자리 5 유지
- 시 12 그대로 → 10 자리 설정도 올림 · 내림 없음

판별

- ±10 이 59 를 넘거나 0 아래로 갈 때 처리
- 넘김 뒤에도 분 1 자리(5) 가 보존되는지



03-1 시계 · 시뮬레이션 버튼 동작

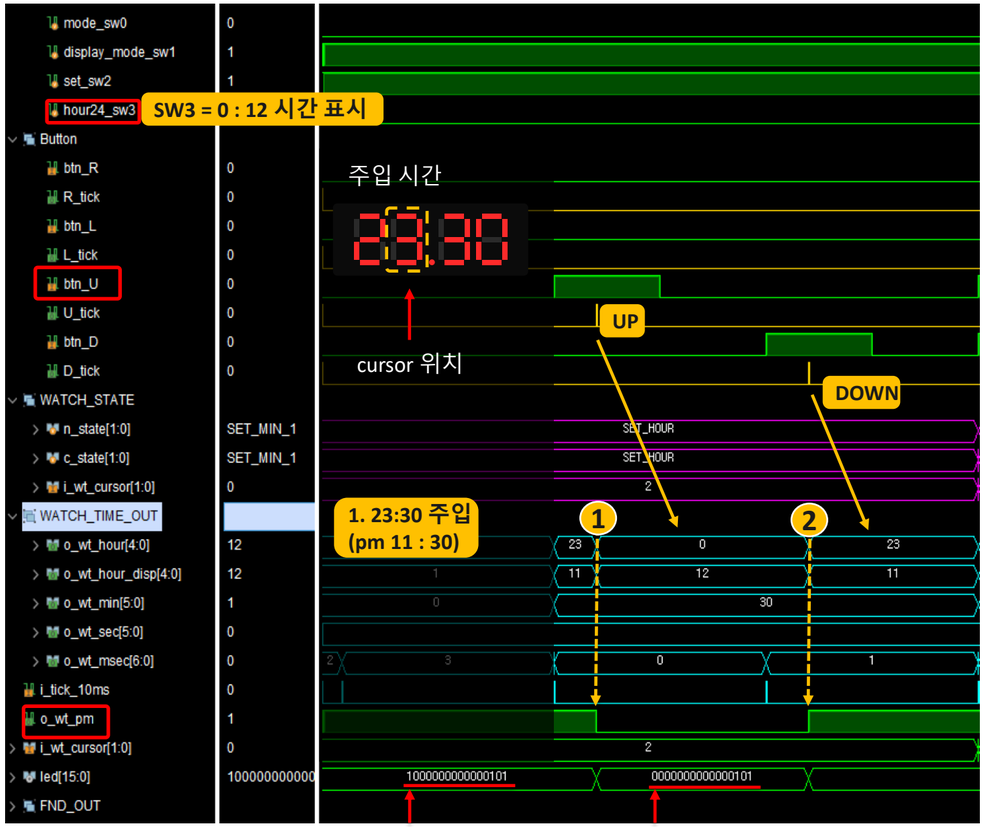
P6 시 경계 + 12 시간 표시 — 23:30 에서 U → D

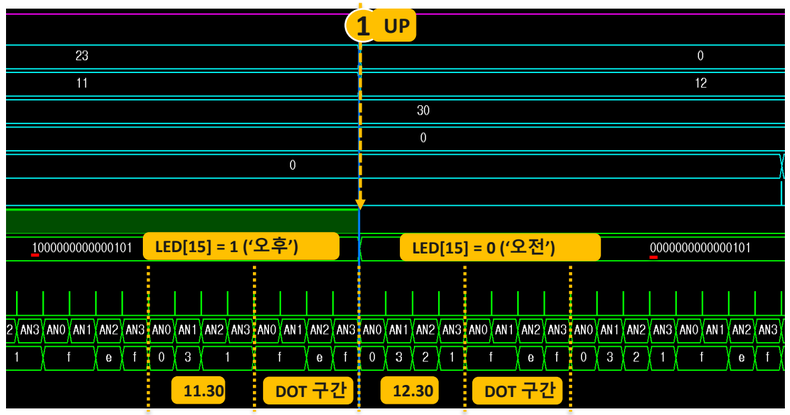
결론

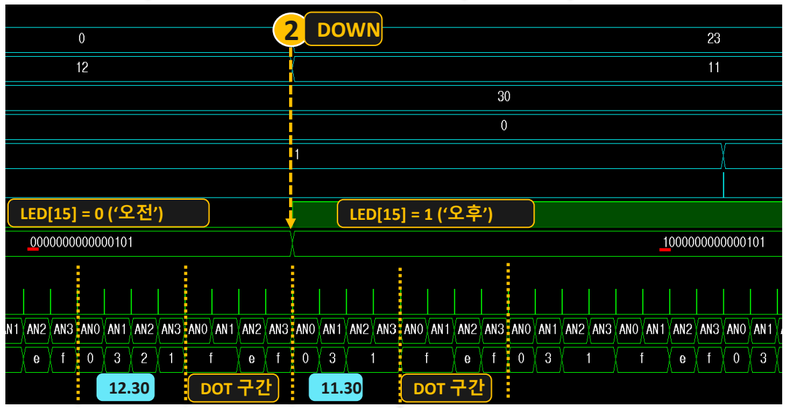
- 시 **23 → 0 → 23**, 분 30 유지
- 표시 **11 → 12 → 11**, LED15 **1 → 0 → 1** → 자정이 오전 12 시로 정상 변환

판별

- 시 23 ↔ 0 순환 때 분이 바뀌지 않는지
- SW3 = 0 (12 시간 표시) : 표시 시 · 오후 LED15 변환







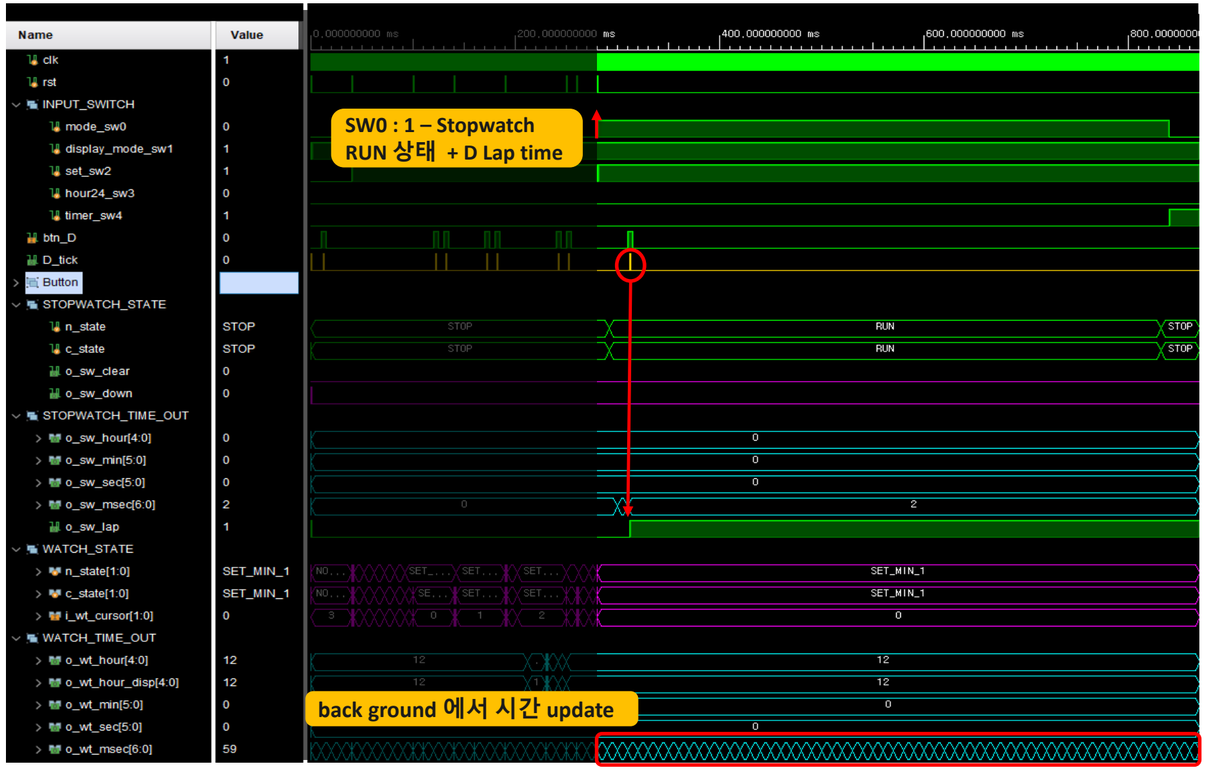
P7 스톱워치 화면(SW0 = 1) — 버튼이 시계로 가지 않음

결론

- D 입력 → 스톱워치 **LAP 저장** (RUN 유지)
- 시계는 **SET_MIN_1 · 12:00 유지** → 버튼 차단
- 시계 1/100 초는 계속 증가 → **백그라운드 동작**

판별

- 설정 중(SW2 = 1) 이어도 SW0 = 1 이면 버튼이 어디로 가는지
- 화면에 없는 시계가 계속 흐르는지



● 03-1 시계 · 설계

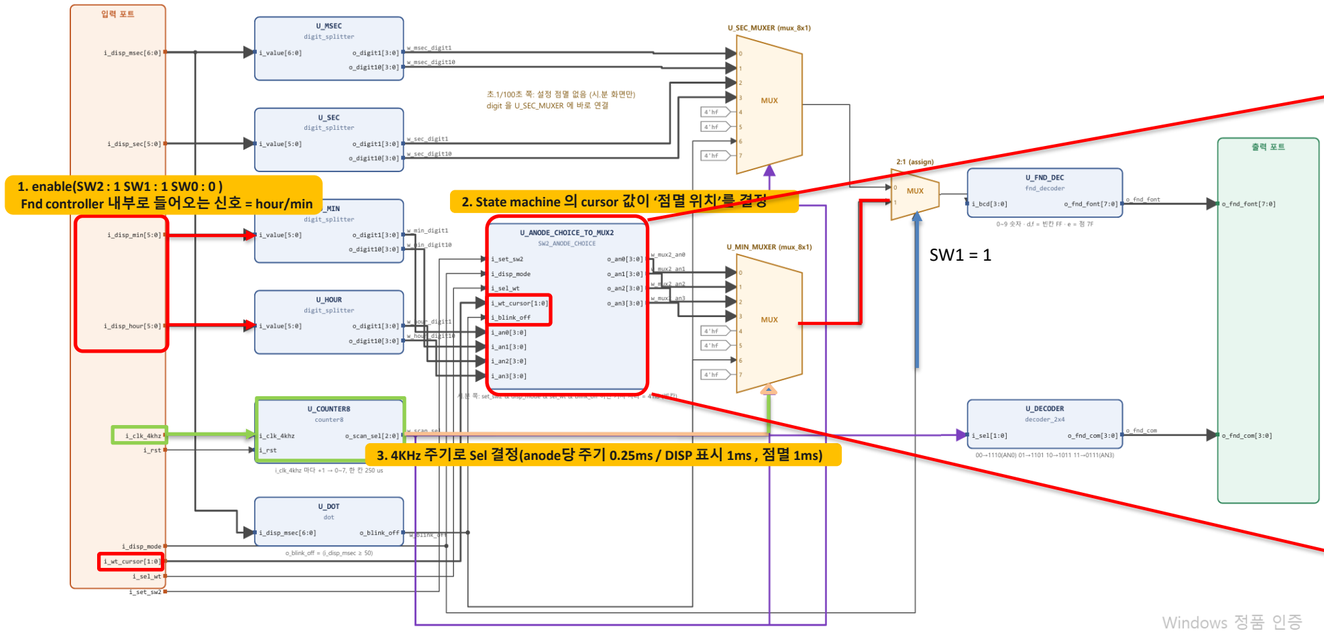
커서 자리 점멸 — 가운데 점 점멸 박자 재사용

결론

- 1/100 초 ≥ 50 이면 blink_off = 1 → 커서 자리만 빈칸 → **0.5 s 마다 점멸**
- 4 kHz 스캔 : 자리당 0.25 ms → **숫자 1 ms + 점 1 ms = 한 바퀴 2 ms**

ex) cursor = 0 (4KHz tick 사용)





U_ANODE_CHOICE 내부 logic

```
reg [15:0] r_an;

always @(*) begin
    if (i_set_sw2 & i_disp_mode & i_sel_wt) begin
        if (!i_blink_off) begin
            r_an = {i_an0, i_an1, i_an2, i_an3};
        end else begin
            case (i_wt_cursor)
                2'b00: r_an = {4'hd, i_an1, i_an2, i_an3};  // 분 일 (AN0)
                2'b01: r_an = {i_an0, 4'hd, i_an2, i_an3};  // 분 십 (AN1)
                2'b10: r_an = {i_an0, i_an1, 4'hd, i_an3};  // 시 일 (AN2)
                2'b11: r_an = {i_an0, i_an1, i_an2, i_an3};
            endcase
        end
    end else begin
        r_an = {i_an0, i_an1, i_an2, i_an3};
    end
end

assign o_an0 = r_an[15:12];
assign o_an1 = r_an[11:8];
assign o_an2 = r_an[7:4];
assign o_an3 = r_an[3:0];
```

Windows 정품 인증

P8 커서 자리 점멸 — 1/100 초 50 부터 커서 자리만 꺼짐

결론

- 1/100 초 50 에서 blink_off **0 → 1**, 커서 자리 fnd_num 만 **f** → 0.5 s 마다 그 자리만 꺼짐
- 커서 0 · 1 · 2 → **AN0 · AN1 · AN2** 만 꺼지고 나머지 자리 (12:34) 값 유지

판별

- SET 상태별 (커서 0 · 1 · 2) 1/100 초 49 → 50 에서 blink_off 가 1 이 되는지
- 그때 커서가 가리키는 자리의 fnd_num 만 f (빈칸) 로 바뀌는지



1 초 정확도 · 올림 경계 — tb_wt_2 (P1 ~ P5)

결론 **1 초 = 정확히 100,000,000 클럭 · 초 → 분 → 시 올림, 자정 23:59:59.99 → 00:00:00.00 까지 검사 13 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P1	리셋 후 아무것도 안 하고 2 초	초가 바뀌는 간격 = 1,000,000,000 ns	**1,000,000,000.000 ns** · 12:00:02.00
P2	00:00:59.95 주입 → tick 5 번	5 번째 tick 에 00:01:00.00	00:01:00.00 · PASS
P3	00:59:59.95 주입	01:00:00.00	01:00:00.00 · PASS
P4	23:59:59.95 주입 (12 시간 표시)	00:00:00.00 · 오후 LED15 1 → 0 · FND "12.00"	**자정 순환 · LED15 0 · "12.00"**
P5	설정 모드 (SW2 = 1) 에서 23:59:59.95	설정 중에도 시각은 흐름 → 00:00	00:00:00.00 · PASS

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
1 초 간격	100,000,000 클럭	1,000,000,000.000 ns	**일치**
자정 직전 tick	10 ms 마다 1 클럭	2,180,004,125 ns	**일치**
msec 99 → 0	tick + 1 클럭	2,180,004,135 ns	**일치**
sec 59 → 0	+ 2 클럭	2,180,004,145 ns	**일치**
min 59 → 0	+ 3 클럭	2,180,004,155 ns	**일치**
hour 23 → 0 · 표시 12 · LED15 0	+ 4 클럭	2,180,004,165 ns	**일치**

콘솔 결과 (xsim 실행 로그 그대로)

```
[2000.001 ms] [OK] P1.WB7 1 s = 100,000,000 clk
   exp=1000000000.000 ns
   got=1000000000.000 ns
[2180.004 ms] carry ripple after tick:
   clk+1=23:59:59.99 clk+2=23:59:59.00
   clk+3=23:59:00.00 clk+4=23:00:00.00
   clk+5=00:00:00.00
[2185.004 ms] [OK] midnight shown as 12.00
[SUMMARY] WT2 checks=13 fail=0 => PASS
```

TB: tb_wt_2_rollover.sv (팀 SV TB) · 최종 코드 · Vivado 2022.2 xsim · 23:59:59.95 같은 경계는 TB 가 카운터에 값을 직접 넣음 (실제 시간 그대로, tick 은 안 올림)

1 초 정확도 · 올림 경계 — 파형

결론 **tick 1 클럭 뒤부터 msec → sec → min → hour 가 한 클럭(10 ns)씩 차례로 올림 — 자정 2,180,004,165 ns**

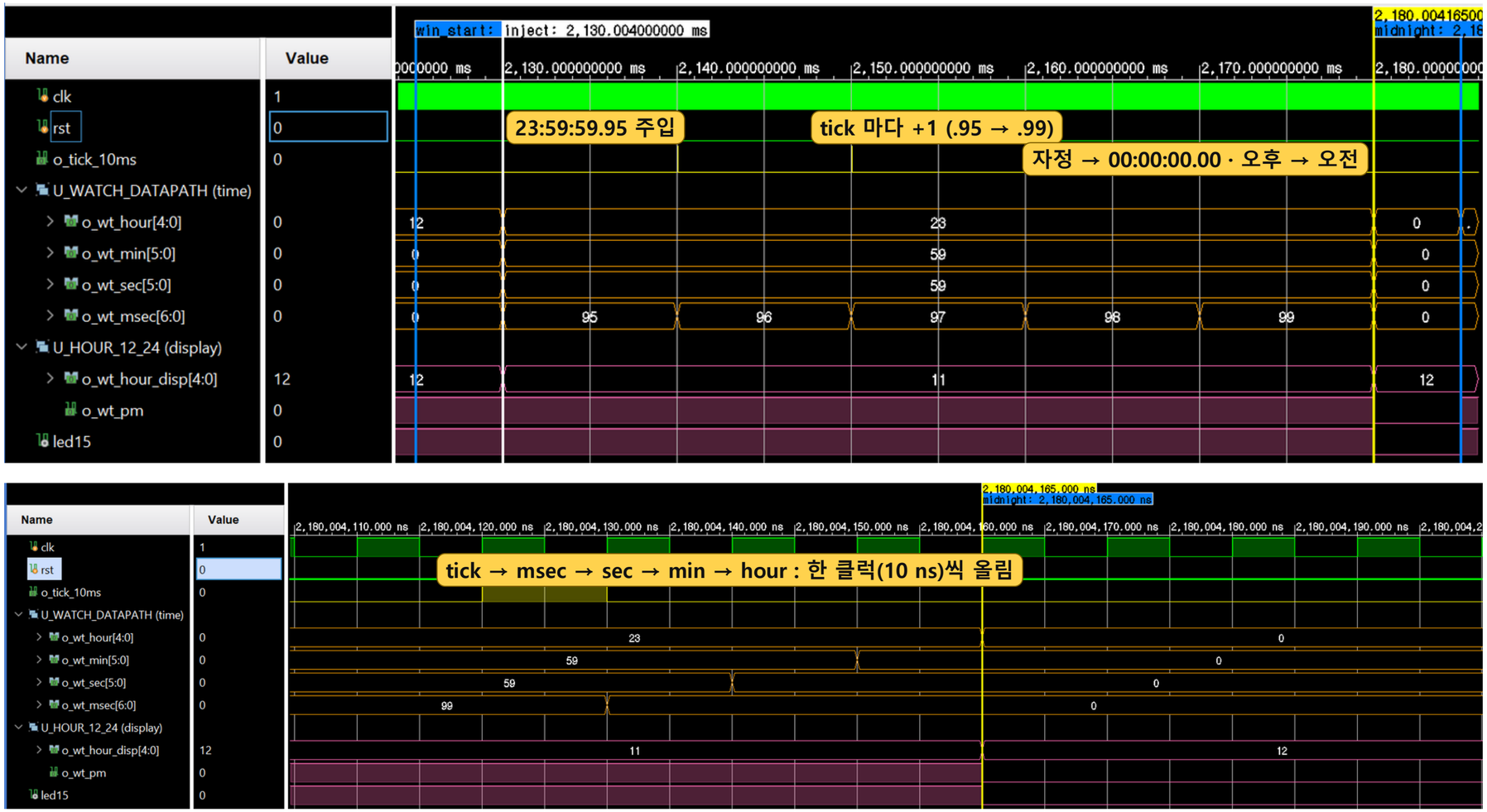


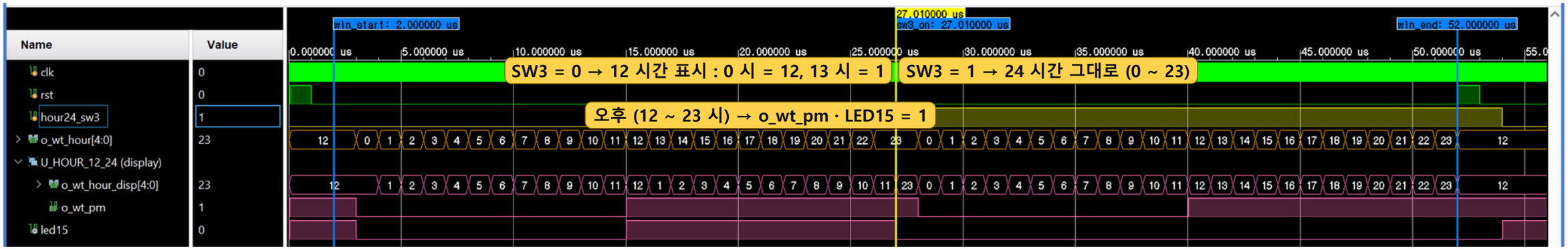
Figure. 시계 자정 올림 (위 : 2,124 ~ 2,186 ms, 아래 : 자정 순간 확대) — Vivado xsim 파형

● 03-1 시계 · 시뮬레이션

12 / 24 시간 표시 · FND 글자 · 설정 점멸 — tb_wt_3 (P1 ~ P4)

결론 **0 ~ 23 시 × SW3 = 48 경우 모두 표시 · 오후 LED15 일치, 설정 자리만 1/100 초 50 부터 꺼짐 — 검사 118 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P1	시 0 ~ 23 × SW3 (0 = 12 시간 · 1 = 24 시간)	12 시간: 0 → 12, 13 ~ 23 → 1 ~ 11 / 24 시간: 그대로 / LED15 = 12 시간 & 오후	**48 경우 × 2 = 96 검사 일치**
P2	12:34 · 13:05 주입	FND "12.34" · 24 시간 "13.05" · 12 시간 "01.05" + LED15	"12.34" · "13.05" · "01.05" · PASS
P3	12:34 설정, 커서 0 · 1 · 2 각 1 초	1/100 초 ≥ 50 동안 커서 자리만 빈칸: "12.3_" · "12._4" · "1_.34"	세 자리 모두 PASS
P4	SW1 × SW3 × 오전 9 시 / 오후 21 시 (8 경우)	led = led0 + (SW1 ? led2 : led3) + (12 시간 & 오후 ? led15)	8 경우 PASS



콘솔 결과 (xsim 실행 로그 그대로)

```
[0.052 ms] [PASS] P1 END checks=96 fail=0
[30.055 ms] [PASS] P2 END checks=4 fail=0
[565.056 ms] [OK] P3.WC8 cursor 0 blank
   while msec>=50  exp="12.3_" got="12.3_"
[1065.056 ms] [OK] P3.WC8 cursor 0 shown
   while msec<50   exp="12.34" got="12.34"
[SUMMARY] WT3 checks=118 fail=0 => PASS
```

TB: tb_wt_3_display_12h.sv (팀 SV TB) · 최종 코드 · 시는 내부에서 0 ~ 23 으로 세고 화면에서만 12 시간으로 바꿈 (hour_12_24 조합 회로)

설정 중 버튼 4 개 동시 입력 16 조합 — tb_wt_4 (P1 ~ P16)

결론 **{D, U, L, R} 16 조합 전부: 하나만 누르면 그 동작, 두 개 이상 같은 클럭이면 모두 무시 — 검사 95 개 PASS**

0000 없음 커서 1 · 12:30	**0001 R** 커서 1 → 0	**0010 L** 커서 1 → 2	**0011 R+L** 무시 · 12:30 · 커서 1
0100 U 12:30 → 12:40	**0101 R+U** 무시 · 12:30 · 커서 1	**0110 L+U** 무시 · 12:30 · 커서 1	**0111 R+L+U** 무시 · 12:30 · 커서 1
1000 D 12:30 → 12:20	**1001 R+D** 무시 · 12:30 · 커서 1	**1010 L+D** 무시 · 12:30 · 커서 1	**1011 R+L+D** 무시 · 12:30 · 커서 1
1100 U+D 무시 · 12:30 · 커서 1	**1101 R+U+D** 무시 · 12:30 · 커서 1	**1110 L+U+D** 무시 · 12:30 · 커서 1	**1111 전부** 무시 · 12:30 · 커서 1

시작: 설정 모드 · 커서 1 (분 십 자리) · 12:30 · 파란 칸 = 버튼 하나

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
U · D 디바운서 o_btn	같은 클럭에 둘 다 1	382,013,605 ns (1 클럭)	**일치**
w_btn_multi	같은 클럭 (조합 회로)	382,013,605 ns	**일치**
o_wt_up · o_wt_down	다음 클럭에도 0	0 유지	**일치**
커서 · 시각	그대로	커서 1 · 12:30	**일치**

콘솔 결과 (xsim 실행 로그 그대로)

```
[382.014 ms] P13 U+D same time
[390.014 ms] [OK] P13.BP.PV o_btn same clock
   exp={D,U,L,R}=1100 got=1100
[390.014 ms] [OK] P13.BP.UP  exp=0 got=0
[390.014 ms] [OK] P13.BP.DN  exp=0 got=0
[390.014 ms] [OK] P13.BP.CUR exp=cursor=1 got=1
[390.014 ms] [OK] P13.BP.VAL exp=12:30 got=12:30
[SUMMARY] WT4 checks=95 fail=0 => PASS
```

왜 모두 무시?

· 버튼 4 개를 기능 3 개가 공유 → 두 개가 같은 클럭에 오면 어느 쪽 의도인지 알 수 없음

· Top 의 w_btn_multi (펄스 2 개 이상) 로 게이팅에서 한 번에 막음 → **세 기능 공통**

TB: tb_wt_4_btn_priority.sv (팀 SV TB) · 최종 코드 · 조합마다 리셋 → 설정 진입 → 커서 1 → 12:30 주입 → 같은 순간 누름 → 6 검사 (P1 은 5)

설정 중 버튼 4 개 동시 입력 16 조합 — 파형

결론 **U · D 가 같은 클럭에 들어오면 w_btn_multi 가 그 클럭에 막음 → up · down 안 나옴, 커서 · 12:30 그대로**

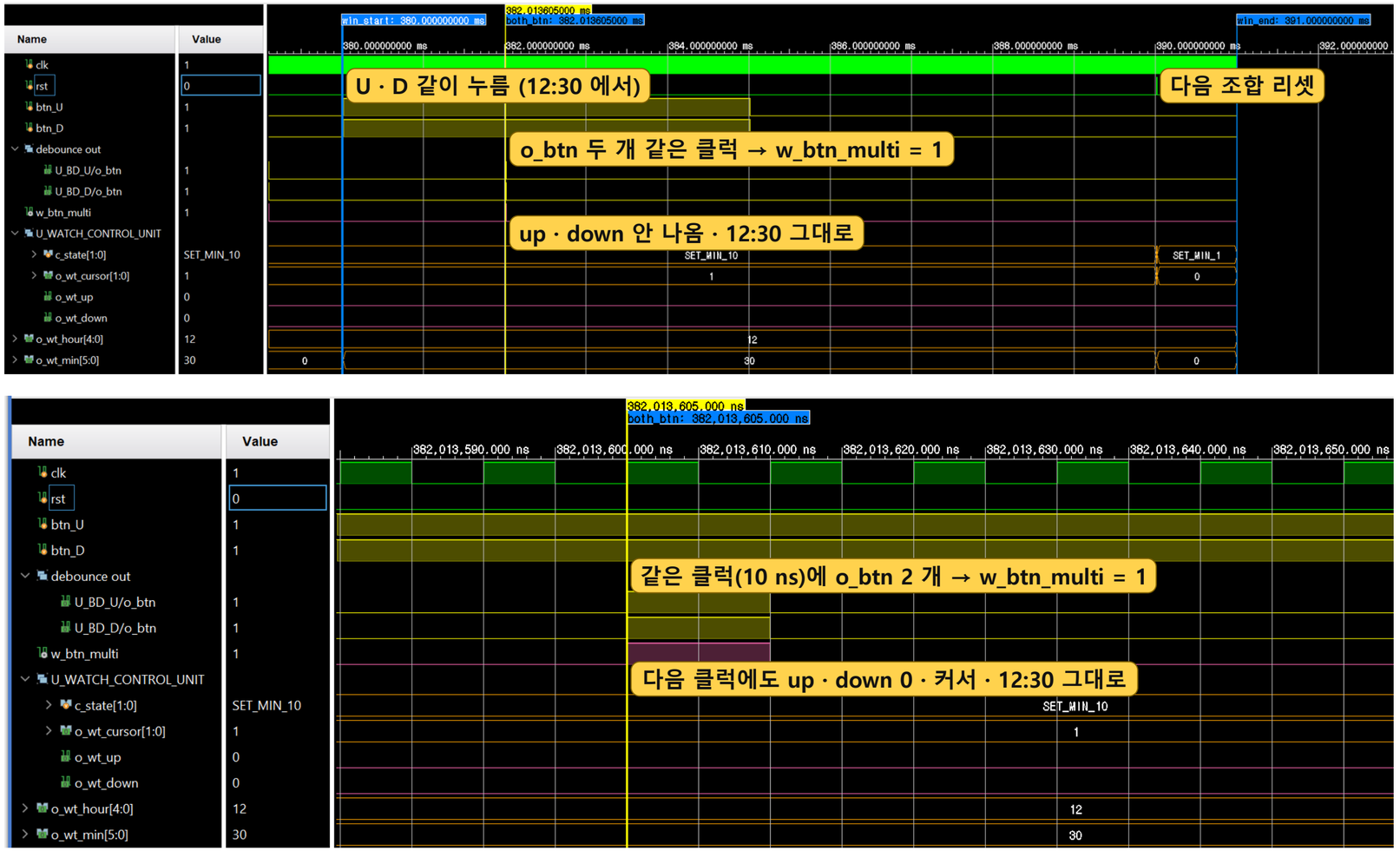


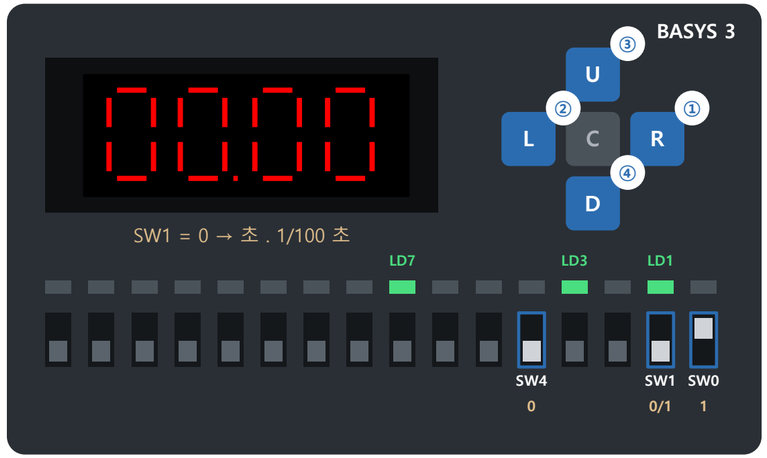
Figure. 설정 중 U + D 동시 입력 (위 : 380 ~ 391 ms, 아래 : 382.0136 ms 확대) — Vivado xsim 파형

● 03-2 스톱워치 · 기능 설명

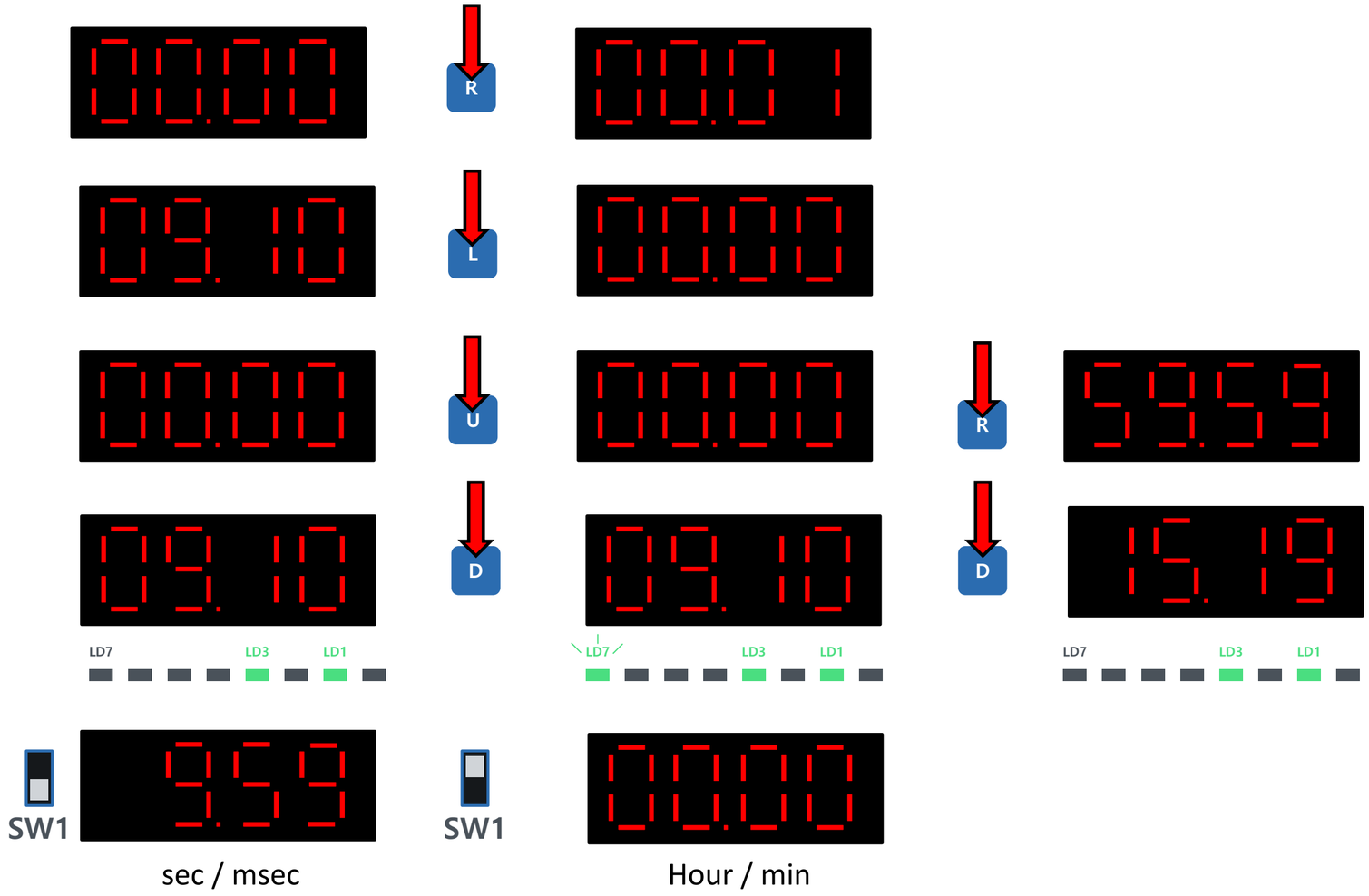
스톱워치 조작법 — 버튼 · 스위치별 동작

결론 **버튼 4 개로 시작/정지 · 초기화 · 업/다운 · LAP — LAP 은 화면만 고정하고 측정은 계속**

STOPWATCH SW4 = 0 · SW0 = 1 → LD1



#	입력	동작
①	BTNR	시작 / 정지 토글
②	BTNL	초기화 → 00:00:00.00 (정지 중)
③	BTNU	Up ↔ Down 방향 전환 (정지 중)
④	BTND	LAP : 시간 저장 · 표시 고정 / 다시 → 실시간
	SW1	화면 0 : 초.1/100초 · 1 : 시.분
	LED	LD1 스톱워치 · LD7 LAP · LD3 / LD2 화면



03-2 스톱워치 · 시뮬레이션

스톱워치 시뮬레이션 시나리오

결론 **R · L · U 기본 동작 3 개 + D LAP 2 개 — 5 시나리오 모두 예상 결과와 일치 (tb_sw_1 ~ 3 검사 53 개 PASS)**

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과
1	**R 정지 · 재실행**	RUN 중 R → 정지 → R 재입력	`w_sw_run_stop · o_sw_msec`	**정지 중 값 유지, 재실행 후 증가**
2	**L 정지 중 초기화**	R 로 정지 → L	`w_sw_clear · o_sw_msec`	**Clear 펄스 → 16 → 0**
3	**U 감소 방향 전환**	정지 중 U → R 로 실행	`w_sw_down · o_sw_msec`	**Down 1 → 5 · 4 · … · 0 감소**
4	**D LAP 저장 · 표시 유지**	측정 중 D	`o_sw_lap · save_msec`	**내부 카운터 증가, 표시 21 유지**
5	**D LAP 해제 · 현재 값**	LAP 중 D 재입력	`save_msec · w_disp_msec`	**저장값 11 유지, 표시 17 (현재)**

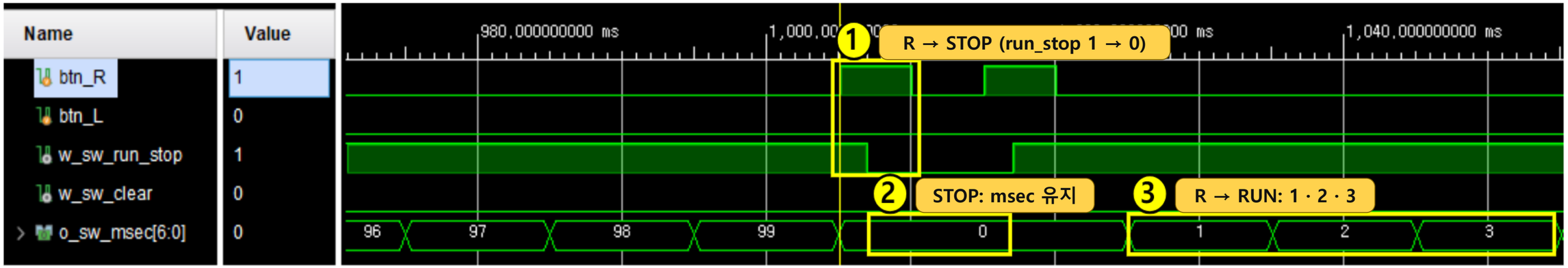
규칙 실행 중 L · U 입력 무시 (STOP 에서만 동작) · D 는 화면에 보낼 값만 바꾸고 내부 측정은 계속
담당 이종혁 — LAP 구현 및 버튼 동작 검증

Note. 시나리오 3 의 99 → 5 구간은 감소를 짧게 보이기 위한 검증용 프리로드

03-2 스톱워치 · 시뮬레이션

시나리오 1 : R 버튼 — 정지 · 재실행

결론 **R → STOP 동안 값 유지, 다시 R → 그 값에서 1 · 2 · 3 이어서 증가**

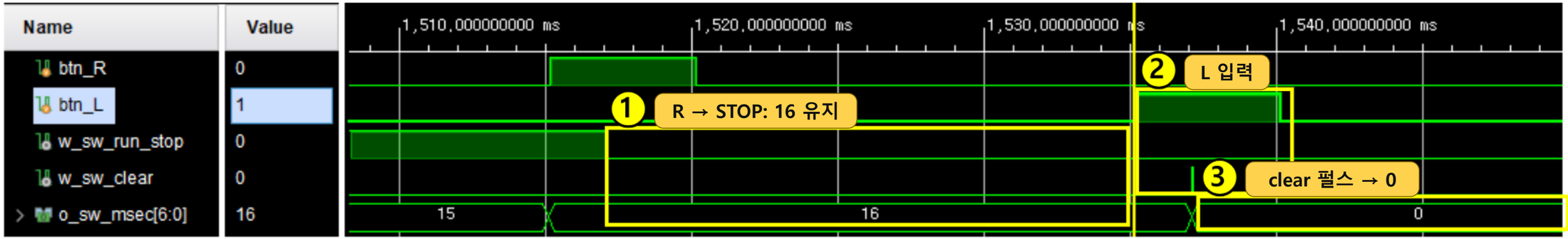


기대값 (Expectation)	시뮬레이션 결과	판정
정지 중 값 유지, 재실행 후 증가	STOP 구간 값 그대로 → R 뒤 1 · 2 · 3 증가	일치

Figure 1. 스톱워치 R 버튼 정지 · 재실행 — Vivado xsim 파형

시나리오 2 : L 버튼 — 정지 중 초기화

결론 **정지 중 L → clear 1 클럭 펄스 → msec 16 → 0 초기화**

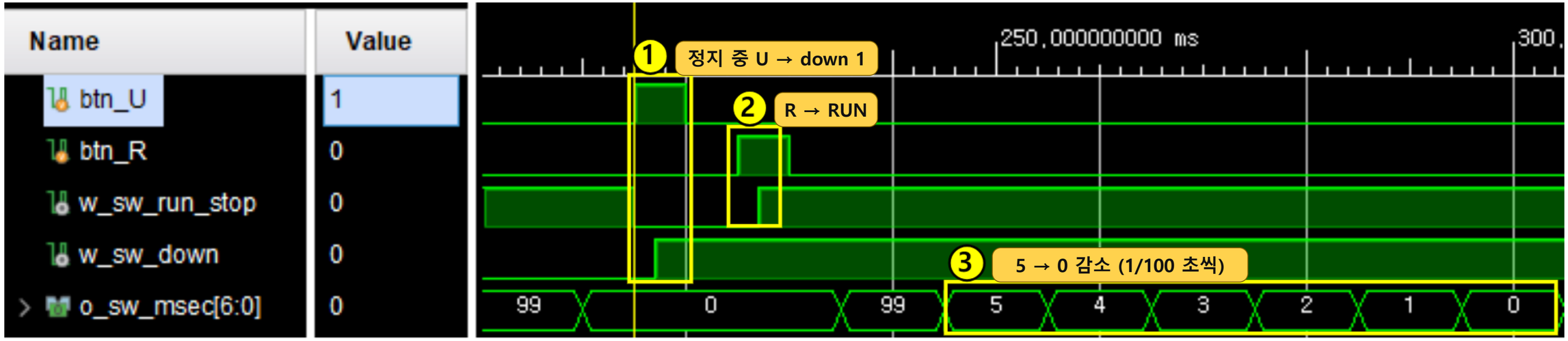


기대값 (Expectation)	시뮬레이션 결과	판정
Clear 펄스 → 16 → 0	L 뒤 clear 1 클럭 → msec 0	일치

Figure 2. 스톱워치 L 버튼 정지 중 초기화 — Vivado xsim 파형

시나리오 3 : U 버튼 — 감소 방향 전환

결론 **정지 중 U → down, R 로 실행하면 5 → 0 으로 1/100 초씩 감소**



기대값 (Expectation)	시뮬레이션 결과	판정
Down 1 → 5 · 4 · … · 0 감소	down 1 → R 뒤 5 · 4 · 3 · 2 · 1 · 0	일치

Figure 3. 스톱워치 U 버튼 감소 방향 전환 (99 → 5 는 검증용 프리로드) — Vivado xsim 파형

LAP 기능 코드 설계 — 저장 · 출력 선택

요약 **D 입력마다 LAP 토글, 켤 때만 저장 · 출력은 LAP 상태로 선택**

① 상태 전환과 시간 저장

```
if (i_sw_btn_D) begin
    lap_enable <= ~lap_enable;

    if (!lap_enable) begin
        save_msec <= i_cnt_msec;
        save_sec  <= i_cnt_sec;
        save_min  <= i_cnt_min;
        save_hour <= i_cnt_hour;
    end
end
```

② 출력값 선택

```
assign o_sw_msec =
    lap_enable ?
    save_msec : i_cnt_msec;
```

LAP 1: 저장값 출력
LAP 0: 현재 카운터 값 출력

초·분·시도 같은 방식으로 선택

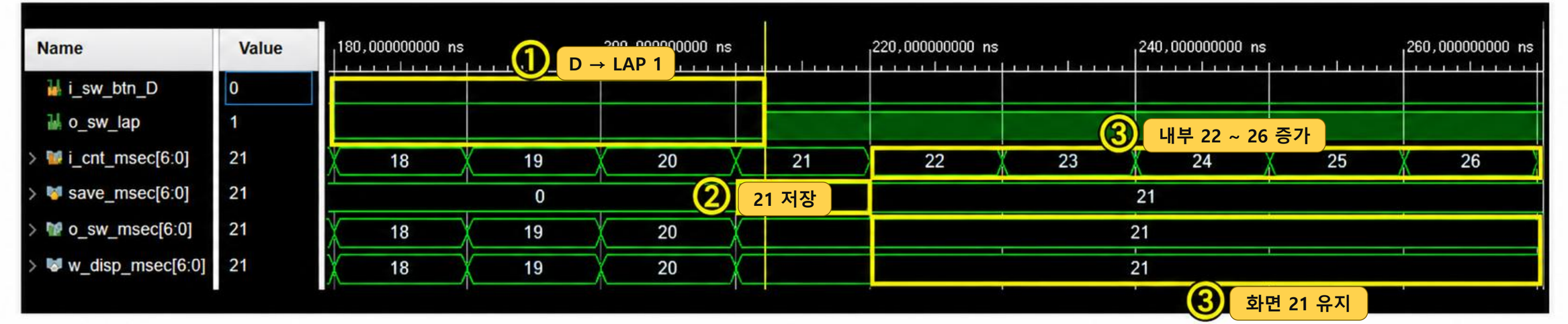
현재 카운터는 이 선택과 별개로 계속 동작

LAP을 켤 때만 저장하고, 해제할 때는 저장값 유지

Reset 또는 Clear 입력 시 LAP 상태와 저장값 초기화

시나리오 4 : LAP — 시간 저장 · 표시 유지

결론 **D → LAP: 21 을 저장해 화면 고정, 내부 카운터는 22 ~ 26 계속 증가**

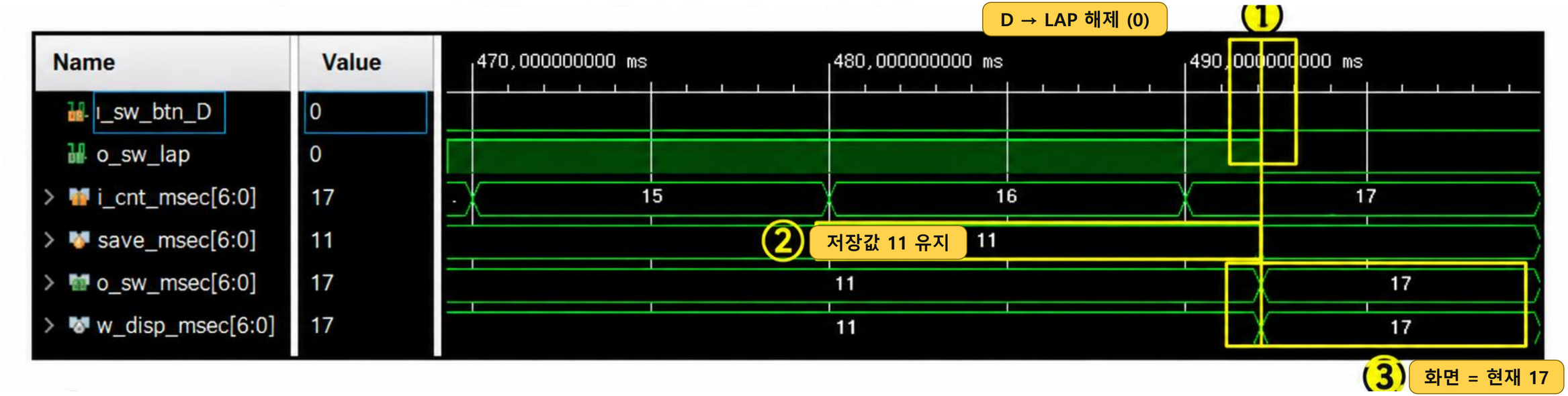


기대값 (Expectation)	시뮬레이션 결과	판정
내부 카운터 증가, 표시 21 유지	save 21 · 내부 22 ~ 26 · 화면 21	일치

Figure 4. 스톱워치 LAP 시간 저장 · 표시 유지 — Vivado xsim 파형

시나리오 5 : LAP — 해제 · 현재 값 표시

결론 **LAP 중 D → 해제: 저장값 11 은 그대로, 화면은 현재 값 17 로 복귀**



기대값 (Expectation)	시뮬레이션 결과	판정
저장값 11 유지, 표시 17 (현재)	LAP 0 · save 11 · 화면 17	일치

Figure 5. 스톱워치 LAP 해제 · 현재 값 복귀 — Vivado xsim 파형

올림 · 빌림 경계 — tb_sw_2 (P1 ~ P5)

결론 **업: 23:59:59.99 → 00:00:00.00 순환 · 다운: 00:00:00.00 → 23:59:59.99 빌림, 동작 중 U 무시 — 검사 15 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P1	업 · 동작 중 00:00:59.95 주입	5 번째 tick 에 00:01:00.00	00:01:00.00 · PASS
P2	업 · 00:59:59.95 주입	01:00:00.00	01:00:00.00 · PASS
P3	업 · 23:59:59.95 주입 (시.분 화면)	00:00:00.00 · FND "00.00"	**자정 순환** · PASS
P4	정지 중 U → R → 00:00:00.05 주입	U 로 다운 · 0 다음 tick 에 23:59:59.99 → 59.98	**00:00:00.00 → 23:59:59.99 → 59.98**
P5	다운 동작 중 U → R(정지) → U → R	동작 중 U 무시, 정지 중 U 로 다시 업	PASS

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
빌림 직전 tick	10 ms 마다 1 클럭	297,004,155 ns	**일치**
msec 0 → 99	tick + 1 클럭	297,004,165 ns	**일치**
sec 0 → 59	+ 2 클럭	297,004,175 ns	**일치**
min 0 → 59	+ 3 클럭	297,004,185 ns	**일치**
hour 0 → 23	+ 4 클럭	297,004,195 ns	**일치**

콘솔 결과 (xsim 실행 로그 그대로)

```
[225.006 ms] [OK] P4.SB6 U in STOP -> down
[295.004 ms] [OK] P4.SB2 before boundary
   exp=00:00:00.00 got=00:00:00.00
[305.004 ms] [OK] P4.SB2 on boundary tick
   exp=23:59:59.99 got=23:59:59.99
[315.004 ms] [OK] P4.SB2 keeps counting down
   exp=23:59:59.98 got=23:59:59.98
[SUMMARY] SW2 checks=15 fail=0 => PASS
```

TB: tb_sw_2_rollover_updown.sv (팀 SV TB) · 최종 코드 · 경계값은 TB 가 카운터에 직접 주입

● 03-2 스톱워치 · 시뮬레이션

올림 · 빌림 경계 — 파형

결론 **다운 카운트 00:00:00.00 → 23:59:59.99 빌림도 tick 뒤 한 클럭씩 4 클럭에 걸쳐 전달**



Figure. 스톱워치 다운 카운트 빌림 (위 : 240 ~ 316 ms, 아래 : 297.004 ms 확대) — Vivado xsim 파형

동작 · 예외 전체 — tb_sw_1 (P1 ~ P7) · tb_sw_3 LAP (P1 ~ P6)

결론 **RUN 1.000 s → 1.00 초, 동작 중 L · U · 동시 R+L 무시, LAP 중 화면 고정 · 내부 계속 — 두 TB 검사 38 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
SW1 P2	R → 정확히 1.000 s 뒤 R	RUN 1,000.000 ms ± 20 µs, tick 100 번 → 1.00 초	1.00 초 · PASS
SW1 P3 · P4	정지 200 ms / 정지 중 D(랩) → L	정지 중 값 고정 / L 로 0 · 랩 해제	PASS
SW1 P5	동작 중 L, 그다음 U	clear 펄스 안 나옴 · 방향 그대로 · 계속 증가	**무시 · 계속 증가**
SW1 P6	정지 중 R 과 L 같은 순간	w_btn_multi = 1 → 둘 다 무시	정지 · 값 유지
SW1 P7	동작 중 시계 화면에서 R · L · U · D	스톱워치 버튼 0 (게이팅) · 백그라운드 계속	PASS · led 8009
SW3 P1 · P2	동작 200 ms 후 D → 다시 D	화면 고정 · 내부 9 번 이상 증가 · led7 / 해제 시 현재 값	PASS
SW3 P3 · P4	랩 중 L(무시) → R → L / 정지 중 D → R	동작 중 L 무시 · 정지 후 L 로 0 / 랩은 데이터패스라 상태와 무관	PASS
SW3 P5 · P6	59.98 주입 직후 D / 랩 중 SW2 = 1	저장 59.98 · 내부 1:00 / 스톱워치 화면은 점멸 없음	PASS

콘솔 결과 (xsim 실행 로그 그대로)

```
[1015.129 ms] [PASS] SW1 P2 END checks=4 fail=0   RUN window exactly 1.000 s -> count 99..100
[1930.138 ms] [PASS] SW1 P5 END checks=3 fail=0   RUN: L ignored, U ignored
[2210.141 ms] [PASS] SW1 P6 END checks=3 fail=0   R and L same instant -> both ignored
[SUMMARY] SW1 checks=23 fail=0 => PASS          [SUMMARY] SW3 checks=15 fail=0 => PASS
```

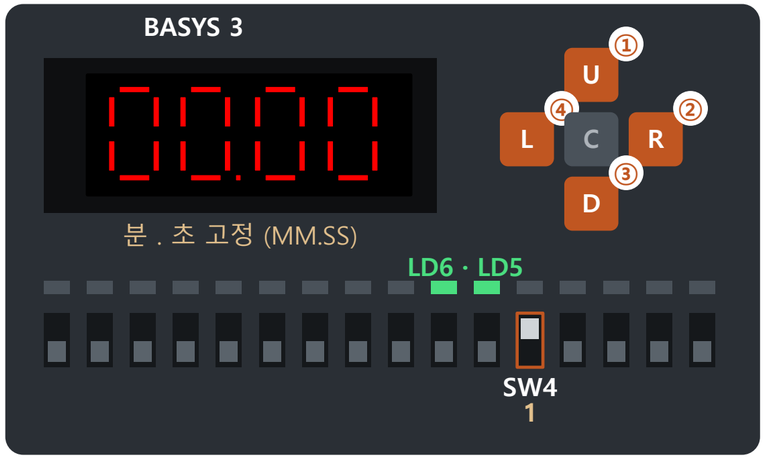
TB: tb_sw_1_run_stop_clear.sv · tb_sw_3_lap.sv (팀 SV TB) · 최종 코드 · 파형은 앞 장 시나리오 1 ~ 5 (R · L · U · LAP) 캡처

● 03-3 타이머 · 기능 설명

타이머 조작법 — 버튼 · 스위치별 동작

결론 **버튼 4 개로 +10 초 · +1 분 · +10 분 · 시작/취소 — 0 이면 LED 16 개 0.5 s 점멸 알람**

TIMER SW4 = 1 (우선) → LD5 · 진행 중 LD6



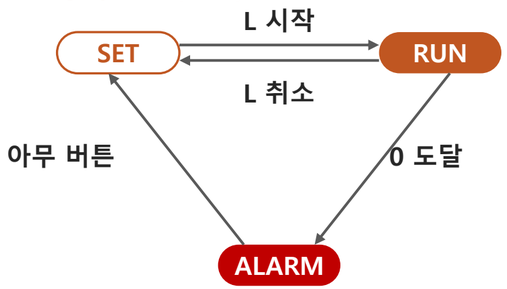
#	입력	동작
①	BTNU	+ 10 초
②	BTNR	+ 1 분
③	BTND	+ 10 분 (상한 59 : 59)
④	BTNL	시작 (0 이면 무시) · 진행 중 취소 → 0
	ALARM	0 도달 → LED 16 개 0.5 s 점멸
	끄기	알람 중 아무 버튼 → SET (값 유지)

화면 : 분 . 초 고정 (MM.SS)

SET 설정 **RUN** 감소 중 **ALARM** 알람



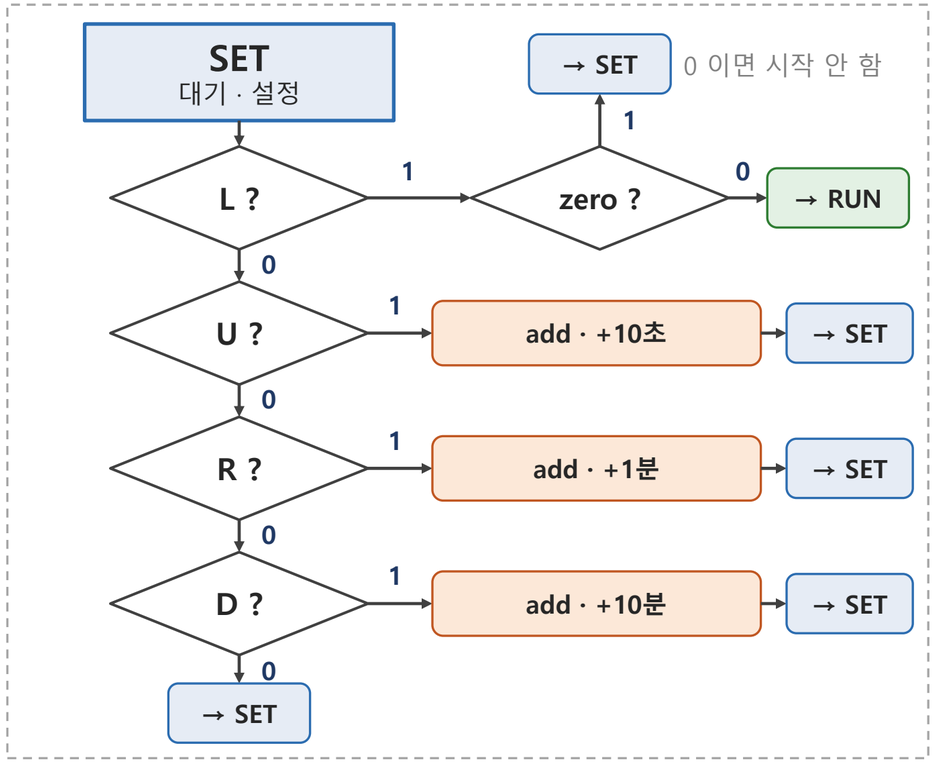
상태 흐름 (FSM)

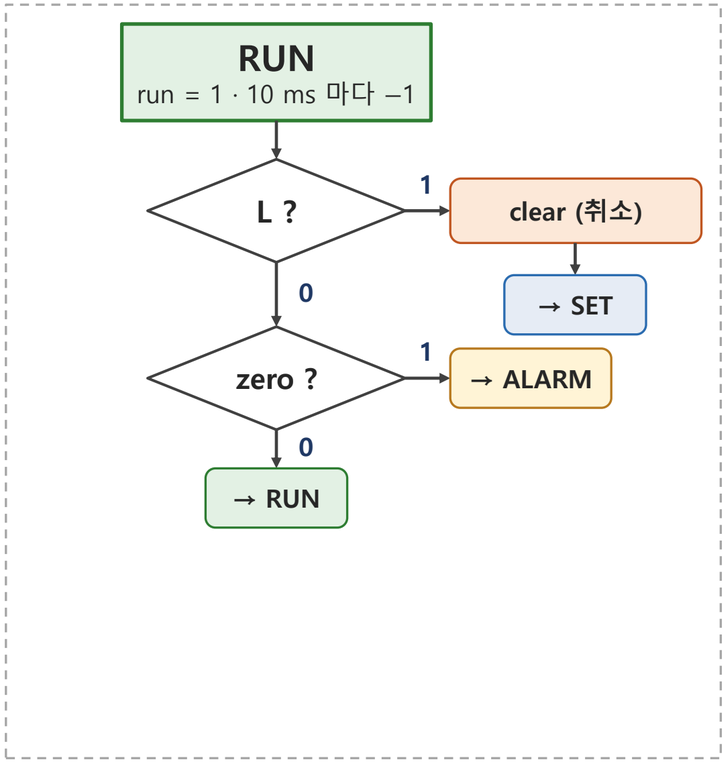


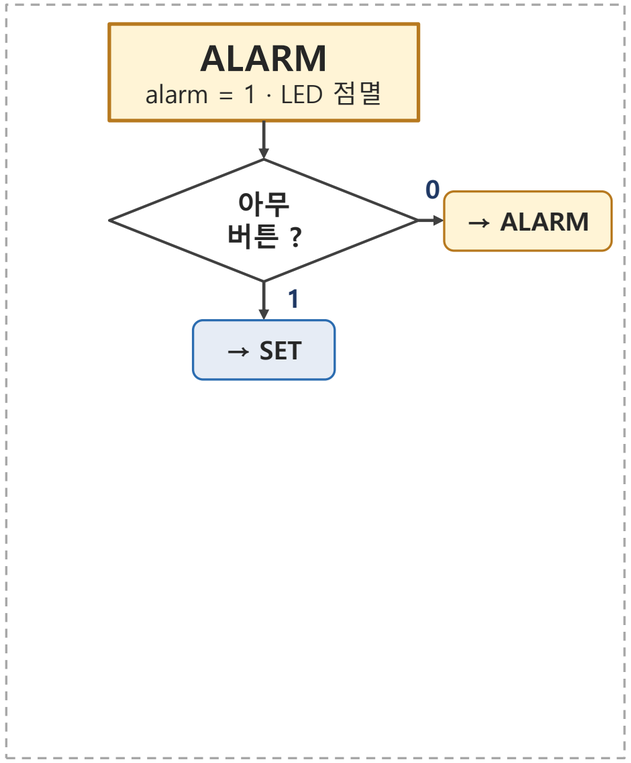
03-3 타이머 · 기능 설명

타이머 FSM — ASM 차트 (SET · RUN · ALARM)

결론 **더하기는 SET 에서만, RUN 은 L · zero 만 봄 → 동작 중 다른 버튼은 무시 (오조작 차단)**







□ **상태** 안의 출력은 그 상태 내내 1 ◇ **조건** 버튼 · zero ▭ **주황** 조건부 출력 (그 클럭에만 1)

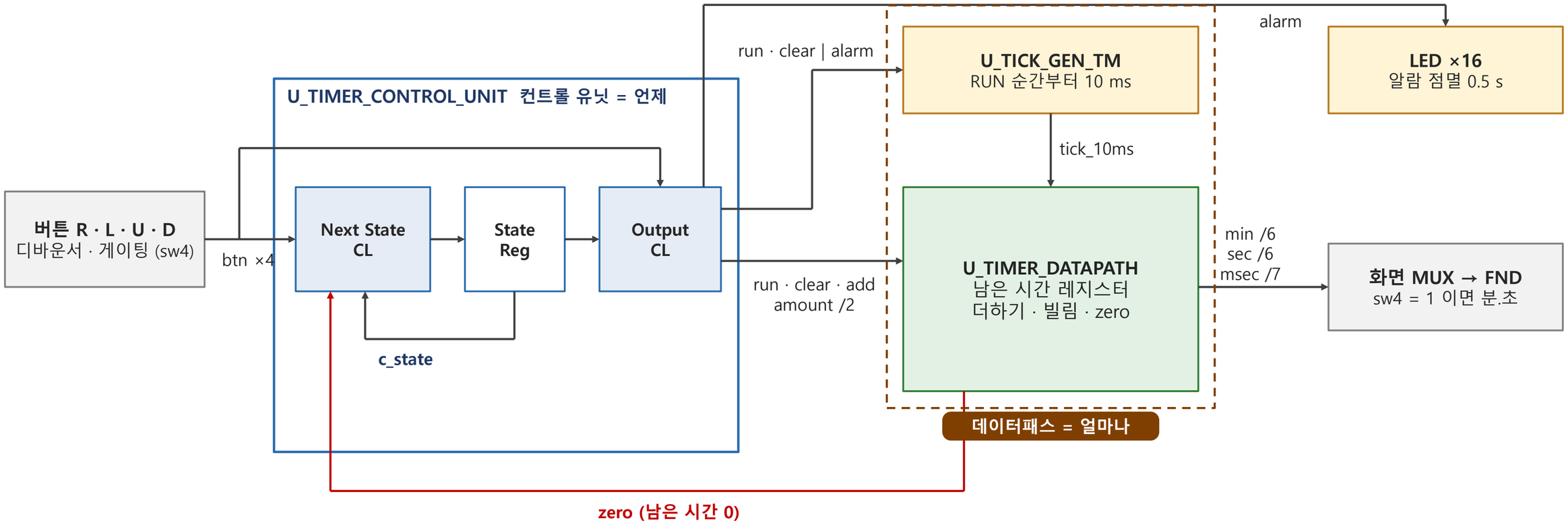
zero = 데이터패스의 남은 시간이 0 **add** = 데이터패스에 더하기 (상한 59:59)

Figure. 타이머 ASM 차트 — timer_control_unit.sv 의 조건 · 출력과 같음

03-3 타이머 · 기능 설명

타이머 구조 — 컨트롤 유닛 · tick 생성기 · 데이터패스

결론 **컨트롤 유닛은 언제(상태 · 버튼)만, 데이터패스는 얼마나(남은 시간)만 → zero 한 줄로 이어져 따로 시험 가능**



출력 run · alarm 은 상태로 정해짐 (무어) · add · clear · amount 는 버튼 누른 클럭에만 (밀리)

되먹임 State Reg 의 c_state 를 Next State CL 로 되돌려 다음 상태를 정함

Figure. 타이머 블록도 — 모듈 · 신호 이름은 소스(verification_v2/source_file) 그대로

03-3 타이머 · 시뮬레이션

타이머 시뮬레이션 시나리오

결론 **TB 3 개 · 검사 73 개를 자동 판정 → 전부 PASS**

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과	TB
1	**더하기 · 자리 올림**	SET 에서 U 6 번 (+10 초씩)	`o_tm_add · o_tm_min/sec`	**00:50 + 10 초 = 01:00**	**TM1**
2	**59:59 상한**	59:40 에서 U 2 번	`o_tm_min · o_tm_sec`	**59:50 → 59:59 유지 (60:00 X)**	**TM1**
3	**10 초 카운트다운 · 알람**	00:10 설정 → L 시작	`c_state · o_tm_alarm · led`	**알람까지 10 s + 20 ns · LED 0.5 s 점멸**	**TM2**
4	**다운카운트 빌림**	01:00.00 · 10:00.00 에서 tick	`o_tm_min/sec/msec`	**00:59.99 · 09:59.99**	**TM2**
5	**백그라운드 · 화면 우선순위**	RUN 중 sw4 1 → 0 / 알람 중 R	`c_state · o_tm_alarm · led`	**시계 화면에서도 알람 · R 은 알람만 끔**	**TM3**

```
# Vivado xsim 콘솔 — 팀 TB (SystemVerilog) 마지막 줄
[  455.019 ms] [SUMMARY] TM1 checks=21 fail=0 => PASS
[11597.002 ms] [SUMMARY] TM2 checks=41 fail=0 => PASS
[  547.009 ms] [SUMMARY] TM3 checks=11 fail=0 => PASS
# 같은 시나리오를 수업 Verilog 문법 (err 카운트) 으로 옮긴 TB
TM1 checks = 21, TOTAL MISMATCH = 0   (TM2 41 · TM3 11 도 0)
```

73 / 73

검사 PASS (TM1 21 · TM2 41 · TM3 11)

0

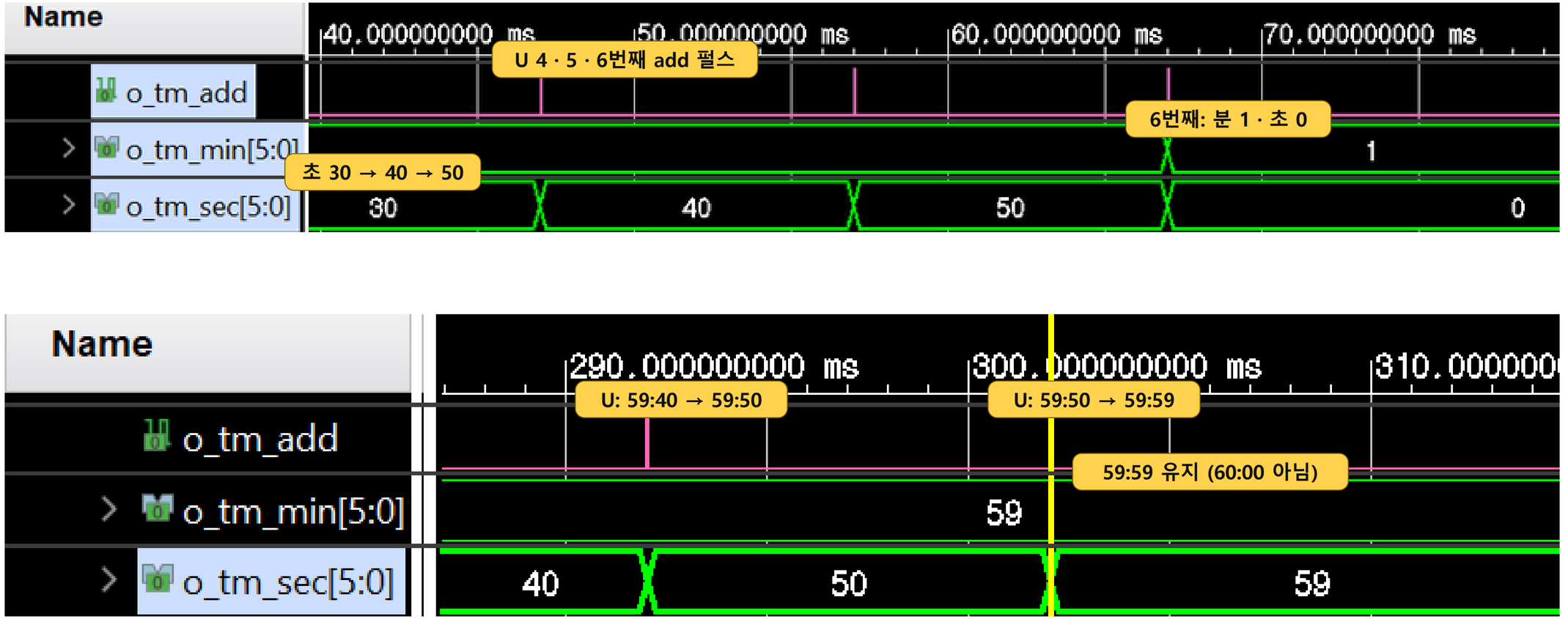
TOTAL MISMATCH (Verilog TB)

Note. 판정 예 — if (값 !== 기대값) err = err + 1; $display("FAIL ...") · 캡처는 10/17 소스, 결과 숫자는 10/18 최종 소스 기준 (둘 다 전부 PASS)

03-3 타이머 · 시뮬레이션

시나리오 1 · 2 : 더하기 · 자리 올림 · 59:59 상한

결론 **00:50 + 10 초 = 01:00 (초 60 → 분 올림) · 59:50 + 10 초 = 59:59 에서 멈춤 → 상한 동작**

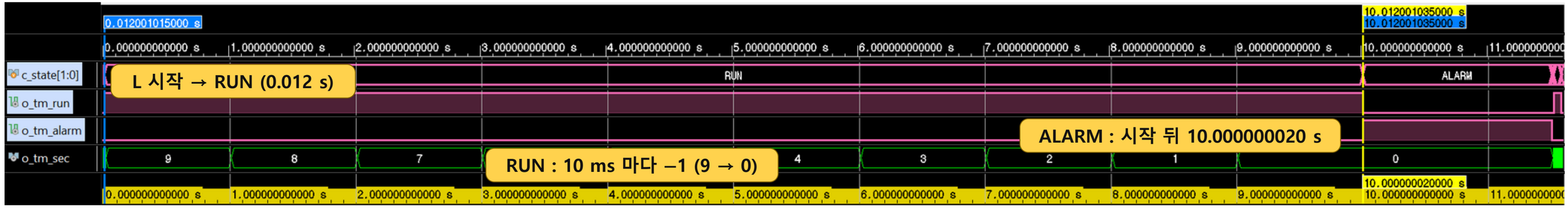


기대값 (Expectation)	시뮬레이션 결과	판정
00:50 + U → 01:00 · 59:50 + U → 59:59 (60:00 아님)	01:00 · 59:59 유지 (tb_tm_1 검사 21 개)	일치

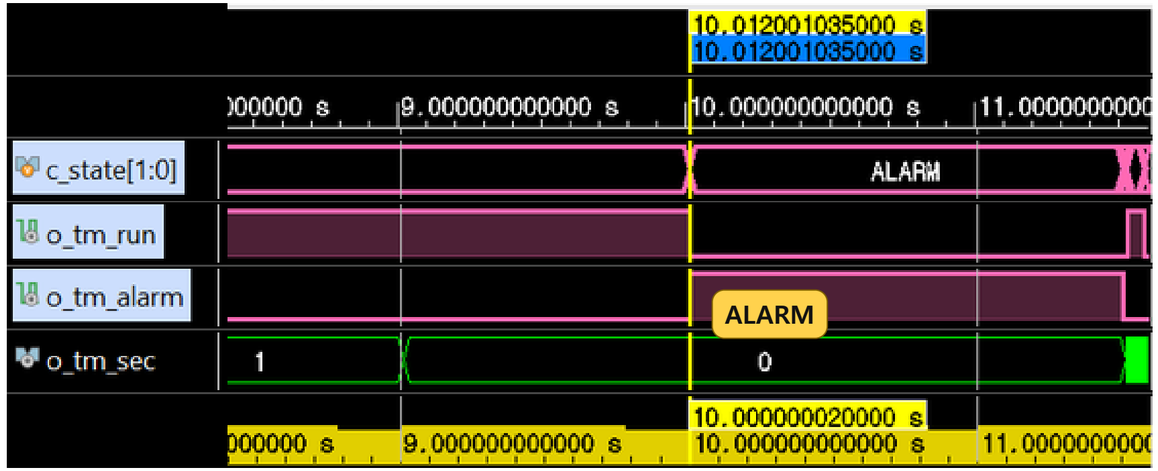
Figure 6. 타이머 더하기 · 59:59 상한 (위 : 더하기, 아래 : 상한) — Vivado xsim 파형

시나리오 3-1 : 10 초 카운트다운 → 알람

결론 **10 초 설정 → 알람까지 10.000000020 s : 오차 +20 ns = 2 클럭 고정 (매번 같음)**



끝부분 확대 (같은 캡처 · 시간축 연속)



20 ns = 2 클럭 (매번 같음)
마지막 tick (정확히 10 s) → **클럭 ①** 남은 시간 0 · zero = 1
→ **클럭 ②** 상태 레지스터 c_state = ALARM → alarm 1

기대값 (Expectation)	시뮬레이션 결과	판정
00:10 → L → 10.000 s 뒤 ALARM	10.000000020 s (tb_tm_2 판정 10 s −1 ~ +3 클럭)	일치

Figure 7. 타이머 10 초 카운트다운 → 알람 (0 ~ 11 s 전체, 시간축 생략 없음) — Vivado xsim 파형

시나리오 3-2 : 알람 LED 점멸

결론 **알람 중 LED 16 개가 0.5 s 켜짐 · 0.5 s 꺼짐 반복**

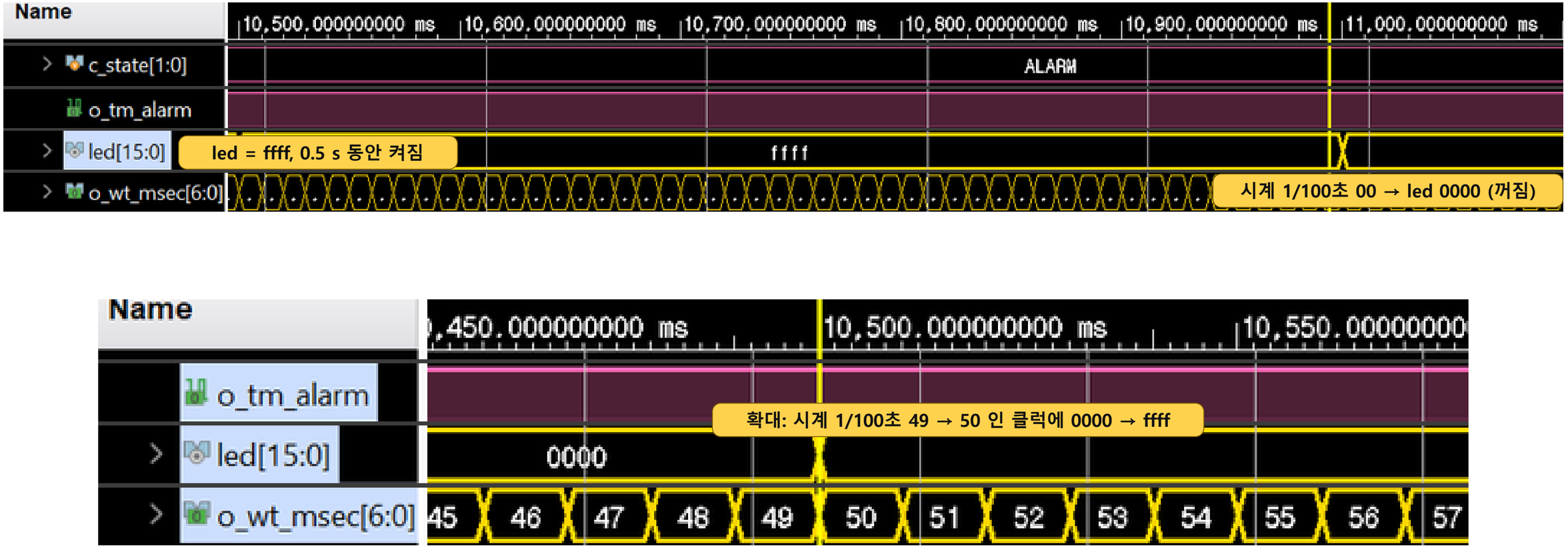


Figure 8. 타이머 알람 LED 점멸 (아래 : 49 → 50 확대) — Vivado xsim 파형

03-3 타이머 · 시뮬레이션

시나리오 4 : 다운카운트 빌림

결론 **빌림 정상 — 01:00.00 → 00:59.99, 10:00.00 → 09:59.99**

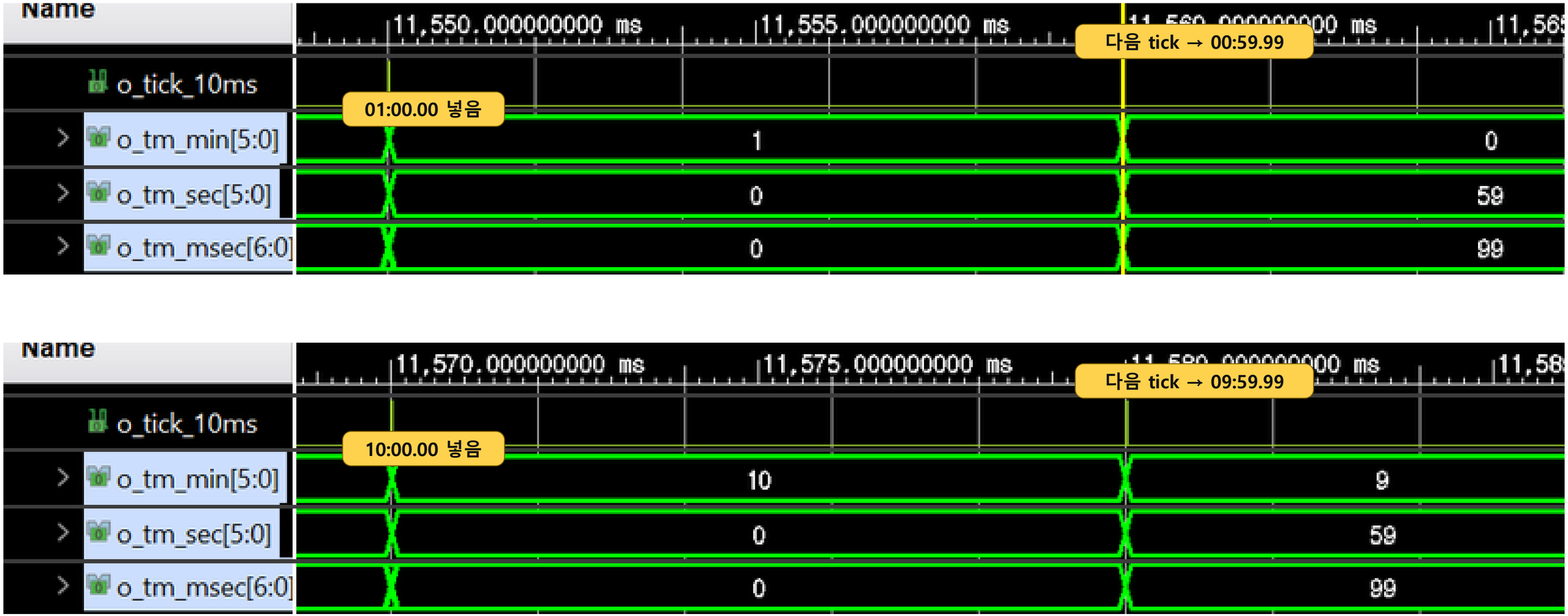
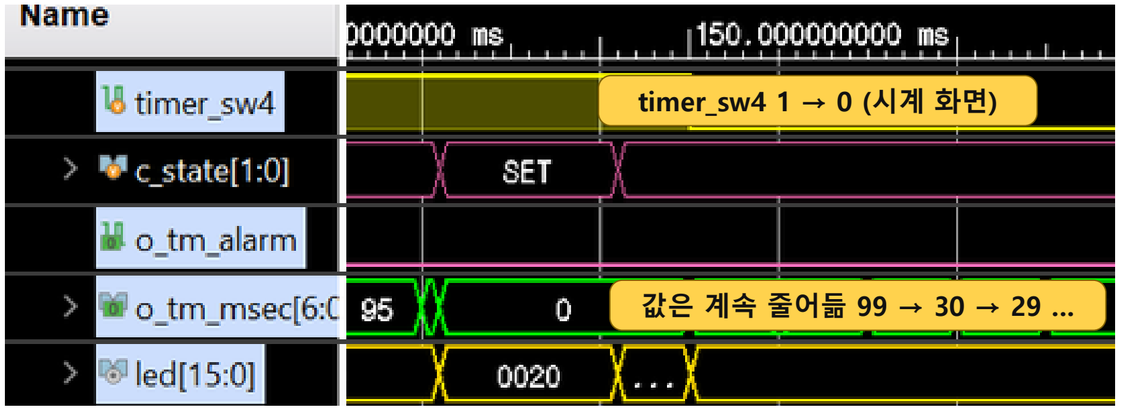
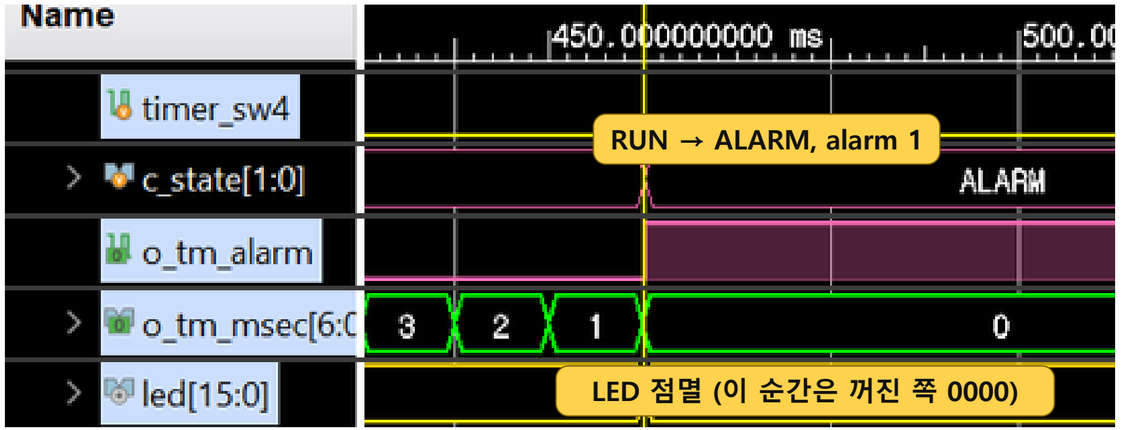


Figure 9. 타이머 다운카운트 빌림 — Vivado xsim 파형

시나리오 5-1 : 백그라운드 동작

결론 **시계 화면으로 바꿔도 타이머는 계속 줄고, 0 이 되면 알람**





~150 ms sw4 → 0 (시계 화면)

약 310 ms 동안 화면은 시계, 타이머는 계속 감소

~460 ms 0 → ALARM

Figure 10. 타이머 백그라운드 동작 (왼쪽 : 화면 전환, 오른쪽 : 알람) — Vivado xsim 파형

03-3 타이머 · 시뮬레이션

시나리오 5-2 : 화면 우선순위 · 알람 끄기

결론 **sw4 · sw0 모두 1 → 타이머 화면, 알람 중 R 은 알람만 끄고 값 유지**

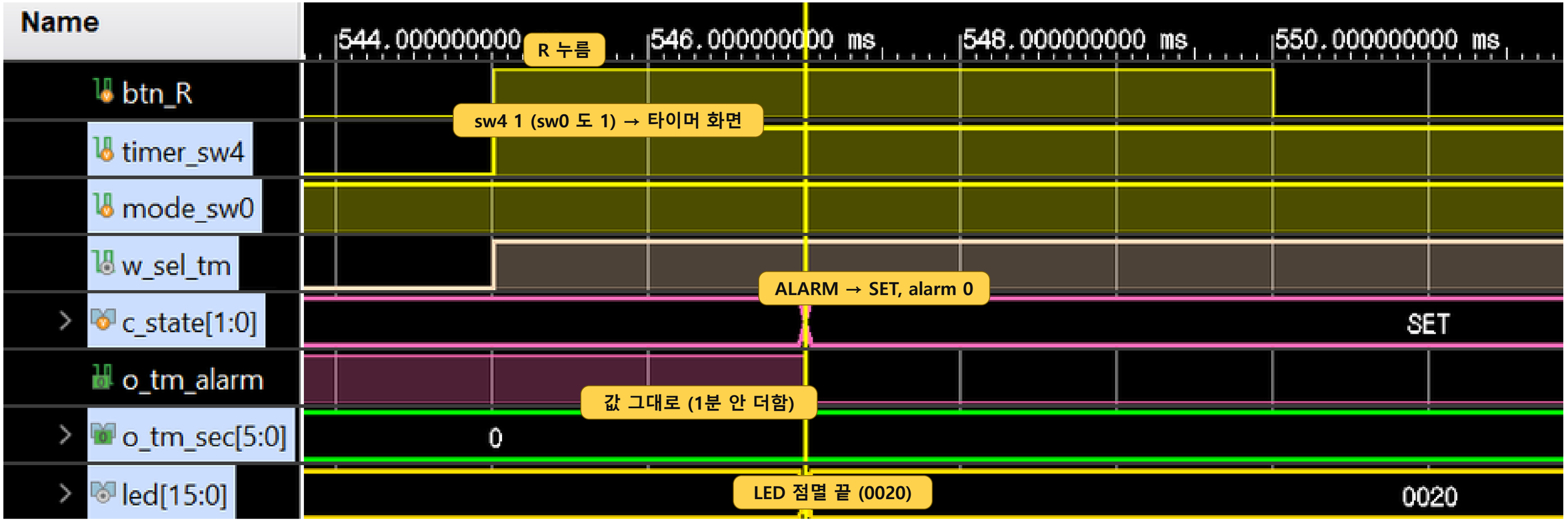


Figure 11. 타이머 화면 우선순위 · 알람 끄기 — Vivado xsim 파형

타이머 검증 신뢰도 — 변이 테스트

결론 **bug 2 개를 일부러 넣으면 TB 가 FAIL 7 · 1 개로 잡음 → MISMATCH 0 은 설계가 맞다는 근거**

일부러 넣은 bug	TB	결과
timer_datapath: 59:59 상한 조건을 끔	TM1	**FAIL 7개**
timer_control_unit: RUN 에서 U 를 받아 +10초	TM3	**FAIL 1개**
원래 설계	TM1 · TM3	**MISMATCH 0**

```
# bug 넣은 설계로 돌린 콘솔
[310015010 ns] FAIL P5 TB1 59:50 + U exp 59:59.00 got 60:0.0
TM1 checks = 21, TOTAL MISMATCH = 7
[50003010 ns] FAIL P1 TC2 U/R/D during RUN exp (5989,5999) got 6996
TM3 checks = 11, TOTAL MISMATCH = 1
```

→ MISMATCH 0 은 "검사가 엉성해서 0" 이 아니라 설계가 사양대로라는 뜻

타이머 Verilog TB 4 개 — 팀 SV TB 와 같은 결과

결론 **수업 Verilog 문법(err 카운트 · TOTAL MISMATCH)으로 다시 짠 TB 4 개 검사 108 개 불일치 0 — 10 초 → 10,000,000,020 ns**

TB	확인하는 것	검사	결과
tb_tm_1_add_cap.v	U +10 초 · R +1 분 · D +10 분 · 59:59 상한 (P1 ~ P7)	21	**TOTAL MISMATCH = 0**
tb_tm_2_countdown_alarm.v	10 초 카운트다운 → 알람 · LED 점멸 · 빌림	41	**TOTAL MISMATCH = 0**
tb_tm_3_cancel_background.v	동작 중 L 취소 · 다른 화면에서 백그라운드 · 알람 끄기	11	**TOTAL MISMATCH = 0**
tb_timer_simple.v	for 로 U 30 번 → 59:59, 10 초 실행, 취소 (발표용 간단판)	35	**TOTAL MISMATCH = 0**
tb_tm_1 + bug ① 상한 제거	같은 TB 를 59:59 상한 지운 설계에	21	MISMATCH = 7 (잡음)
tb_tm_3 + bug ② RUN 중 U 받음	같은 TB 를 동작 중 U 받는 설계에	11	MISMATCH = 1 (잡음)

콘솔 결과 (xsim 실행 로그 그대로)

```
[645021010 ns] TM1 checks = 21, TOTAL MISMATCH = 0          (tb_tm_1_add_cap.v)
[1] add x30 done: 59:59 (expect 59:59)                          (tb_timer_simple.v)
[3] run -> alarm = 10000000020 ns                               TOTAL 35, MISMATCH = 0
[310015010 ns] FAIL P5 TB1 59:50 + U exp 59:59.00 got 60:0.0   (bug ① 상한 제거 → 잡음)
[50003010 ns] FAIL P1 TC2 U/R/D during RUN exp (5989,5999) got 6996   (bug ② → 잡음)
```

TB: Mini_Project_watch/timer_tb_verilog (3_Code/tb/verilog) · bug 주입 설계 사본 3_Code/tb/mutant · 10 초 = 10 s + 20 ns (알람까지 2 클럭 고정)

시뮬레이션 전체 — TB 24 개 결과 한눈에

결론 **최종 코드로 TB 24 개 다시 실행 — 기능 TB 21 개 불일치 0, bug 주입 TB 전부 검출, F3 은 옛 설계 기준 판정 (다음 장들)**

기능	TB	검사	결과
시계	tb_wt_1 설정 UI	38	PASS
시계	tb_wt_2 1 초 · 올림	13	PASS
시계	tb_wt_3 12/24 · 점멸	118	PASS
시계	tb_wt_4 버튼 16 조합	95	PASS
스톱워치	tb_sw_1 동작 · 예외	23	PASS
스톱워치	tb_sw_2 올림 · 빌림	15	PASS
스톱워치	tb_sw_3 LAP	15	PASS
타이머	tb_tm_1 더하기 · 상한	21	PASS
타이머	tb_tm_2 10 초 · 알람	41	PASS
타이머	tb_tm_3 취소 · 백그라운드	11	PASS
공통	tb_top_1 디바운서	24	PASS
공통	tb_top_2 FND 스캔	22	PASS

기능	TB	검사	결과
공통	tb_top_3 동시 입력	11	PASS
타이머	Verilog TB 4 개	108	불일치 0
tick	tb_tick_restart 첫 tick	3	불일치 0
tick	tb_sw_stop_edge 정지 경계	3	불일치 0
디바운서	tb_bounce 1 MHz vs 4 kHz	20 번	4 kHz 오인식 0
bug	tb_tm_1 / tm_3 + bug ① ②	21 / 11	검출 7 / 1
bug	tb_f1 FND 자리 지연	13	검출 85
bug	tb_f2 버튼 끊김	44	검출 12
bug	tb_f4 리셋 글리치	26	검출 5
시점	tb_f3 누르는 시점	57	판정 FAIL 11 *
합계	팀 SV TB 13 개	447	전부 PASS

* tb_f3 은 공용 tick 시절 "누른 시점 따라 오차가 보여야 한다" 로 짠 TB → 최종 코드는 오차가 없어 판정만 FAIL (값은 항상 100 칸 · 1,000.000020 ms, 뒤 장)

로그: 2_Simulation/logs/<TB 이름>/xsim.log · 최종 코드 (팀 저장소 verification_v2/source_file + 동시 입력 무시) · Vivado 2022.2 xsim · 실행 순서 run_all.sh

디바운서 — 누름 길이 · 튐 · 길게 누름 — tb_top_1 (P0 ~ P5)

결론 **누름 1.7 ms 이하 무시 · 2.0 ms 이상 정확히 1 번, 튐 · 2 초 길게 눌러도 1 번, 펄스 폭 = 1 클럭 (10 ns) — 검사 24 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P0	리셋 직후 1 µs	출력 X 없음 · LED 8009 · 12:00	PASS
P1	U 를 1.0 / 1.5 / 1.7 / 2.0 / 2.5 / 5 / 100 ms 누름	1.7 ms 까지 0 개, 2.0 ms 부터 1 개	**0 · 0 · 0 · 1 · 1 · 1 · 1 개**
P2	튐 (40 ~ 300 µs 조각) + 20 ms 누름	정확히 1 개	1 개 · 분 +1
P3	2 초 길게 누름	1 개 (자동 반복 없음)	1 개
P4	누른 채 중간에 300 µs 끊김	2 개 = 설계 한계 (뗄 때 1 샘플)	2 개 [LIMIT] 기록
P5	지금까지 모든 버튼 펄스	폭 정확히 10 ns	10 ns

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
1.7 ms 누름	8 샘플 못 채움 → 무시	o_btn 없음 · 분 0	일치
2 ms 누름 시작	—	19,205,010 ns	—
첫 4 kHz 샘플	누른 뒤 0 ~ 250 µs	19,253,005 ns (+48 µs)	일치
8 번째 샘플 → o_btn	첫 샘플 + 7 × 250 µs	21,003,005 ns	일치
누름 → 인정	1.75 ~ 2.0 ms	1.798 ms	일치

콘솔 결과 (xsim 실행 로그 그대로)

```
[19.205 ms] press 1.7 ms -> pulses=0
[26.205 ms] press 2.0 ms -> pulses=1 min 0->1
[148.705 ms] press 100 ms -> pulses=1
[2179.961 ms] press 2 s -> pulses=1
[2205.264 ms] [LIMIT] P4.GLITCH 300 us
   dropout while held = 2nd press
[SUMMARY] TOP1 checks=24 fail=0
   limit=1 => PASS
```

왜 1.75 ~ 2.0 ms: 4 kHz (250 µs) 로 8 번 연속 1 이어야 인정 → 누른 시점과 샘플 위상에 따라 1.75 ~ 2.0 ms · 한계 P4 는 결론 · 고찰 ② (뗄 때도 8 번 확인)

디바운서 — 파형

결론 **4 kHz 샘플 8 번 연속 1 이어야 인정 — 1.7 ms 누름은 무시, 이번 누름은 1.798 ms 만에 o_btn 1 번**

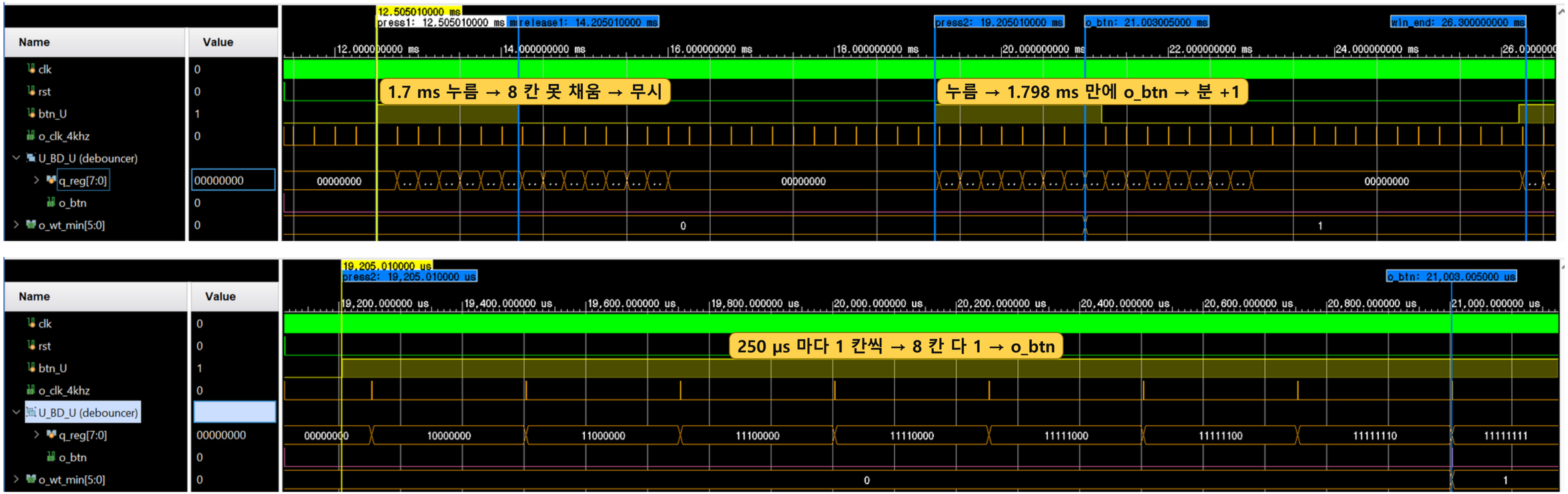


Figure. 디바운서 누름 길이 (위 : 12.5 ~ 26.3 ms, 아래 : 두 번째 누름 확대) — Vivado xsim 파형

● 03-4 공통 · 시뮬레이션

FND 스캔 — 자리 순서 · 한 칸 250 µs · 글자 전 구간 비교 — tb_top_2 (P1 ~ P4)

결론 **자리 AN0 → AN3 · 한 칸 250,000 ns (한 바퀴 2 ms), 화면 글자 전 구간 참조 모델과 불일치 0 — 검사 22 개 PASS**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P1	20 ms 동안 fnd_com 80 번 바뀜	1110 → 1101 → 1011 → 0111 반복, 한 칸 정확히 250,000 ns	**250,000 ns** · 순서 일치
P2	시계 시.분: 리셋 → 12:34 → 설정 1 초	"12.00" → "12.34" → 커서 0 점멸 "12.3_" ↔ "12.34"	PASS
P3	시계 초.1/100 초 1 초	가운데 점 = 1/100 초 < 50 인 프레임만 (0.5 초 주기)	PASS
P4	스톱워치 · 타이머 화면	"34.56" · "00.00" · 타이머 "12.34" · 타이머는 점멸 없음	PASS

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
자리 바뀜 간격	250 µs (4 kHz)	251,005 → 501,005 ns = 250,000 ns	**일치**
자리 순서	AN0 → AN1 → AN2 → AN3	1110 → 1101 → 1011 → 0111	**일치**
한 바퀴	8 칸 × 250 µs = 2 ms	o_scan_sel 0 → 7 → 0	**일치**

콘솔 결과 (xsim 실행 로그 그대로)

```
[20.251 ms] [PASS] P1 END checks=2
   com order AN0->AN1->AN2->AN3
   slot width 250,000 ns
[1055.254 ms] [PASS] P2 END checks=6
[2065.255 ms] [PASS] P3 END checks=3
[2180.257 ms] [PASS] P4 END checks=11
[SUMMARY] TOP2 checks=22 fail=0 => PASS
```

TB: tb_top_2_fnd_scan.sv · fnd_monitor 가 fnd_com · fnd_font 를 참조 모델과 시뮬 전 구간 계속 비교 (다르면 [FAIL] + 시각) · 동시 입력 tb_top_3 은 부록 B

FND 스캔 — 파형

결론 **자리 하나 250,000 ns · AN0 → AN3 순서 · 8 칸 한 바퀴 2 ms (4 ~ 7 칸은 빈칸 · 점)**

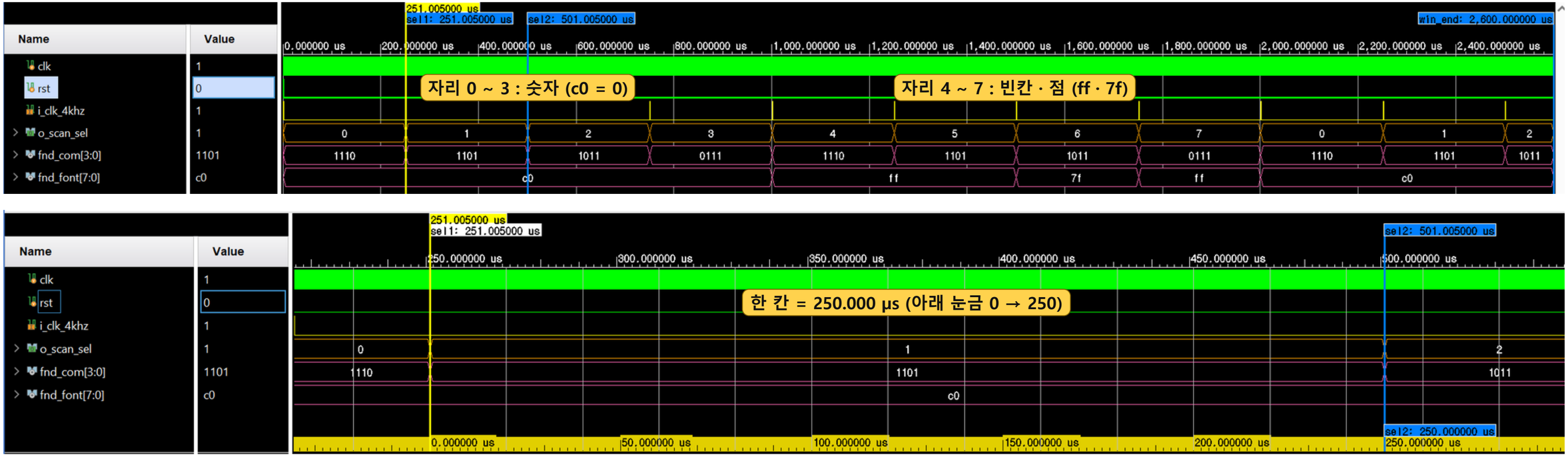


Figure. FND 스캔 (위 : 0 ~ 2.6 ms, 아래 : 한 칸 확대 · Floating Ruler) — Vivado xsim 파형

● 03-5 tick · 타이밍 체크

tick 카운트 체크 — 첫 tick · 정지 경계 · 1 초 = 1 억 클럭

결론 **시작 → 첫 1/100 초 매번 10,000,000 ns, tick 나오는 클럭에 정지해도 1 칸 안 잃음, 시계 1 초 = 100,000,000 클럭**

TB	확인하는 것	기대값	결과
tb_tick_restart.v	타이머 ① 처음 시작 ② 알람 뒤 ③ 취소 뒤 → 첫 tick 까지	셋 다 10,000,000 ns	**10,000,000 ns × 3** (고치기 전 ② 9,999,980)
tb_sw_stop_edge.v	스톱워치 tick 나오는 바로 그 클럭에 STOP	마지막 칸까지 셈 → msec 5	**msec = 5** (고치기 전 4)
tb_wt_2 P1	시계 초가 바뀌는 간격	1,000,000,000 ns	1,000,000,000.000 ns
tb_timer_simple.v	10 초 실행 → 알람	10 s + 2 클럭	10,000,000,020 ns
tb_sw_1 P2	스톱워치 RUN 정확히 1.000 s	tick 100 번 → 1.00 초	1.00 초

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
RUN 시작	L 누름 (디바운스 뒤)	12,001,005 ns	—
첫 tick	RUN + 1,000,000 클럭	22,001,005 ns	**일치**
간격	10,000,000 ns	10,000,000 ns	**일치**
카운터	999,999 → 0 에 tick	999,999 → 0 · tick 1 클럭	**일치**

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
보통 정지	멈췄다 이어서	5 에서 멈춤 → 6 부터	**일치**
tick 클럭에 정지	마지막 칸도 셈	565 ns 정지 · tick	—
msec	다음 클럭 +1	575 ns : 4 → 5	**일치**

tick_gen_100hz_run: RUN 일 때만 세고, 정지는 멈춤 · 취소 / 알람은 0 으로 → 트러블슈팅 1 해결의 근거 · tb_sw_stop_edge 는 tick 간격을 10 클럭으로 줄인 모듈 단위 TB

● 03-5 tick · 타이밍 체크

tick 카운트 체크 — 파형 ① 타이머 첫 tick

결론 **RUN 순간 카운터 0 에서 시작 → 정확히 10,000,000 ns (1,000,000 클럭) 뒤 첫 tick**

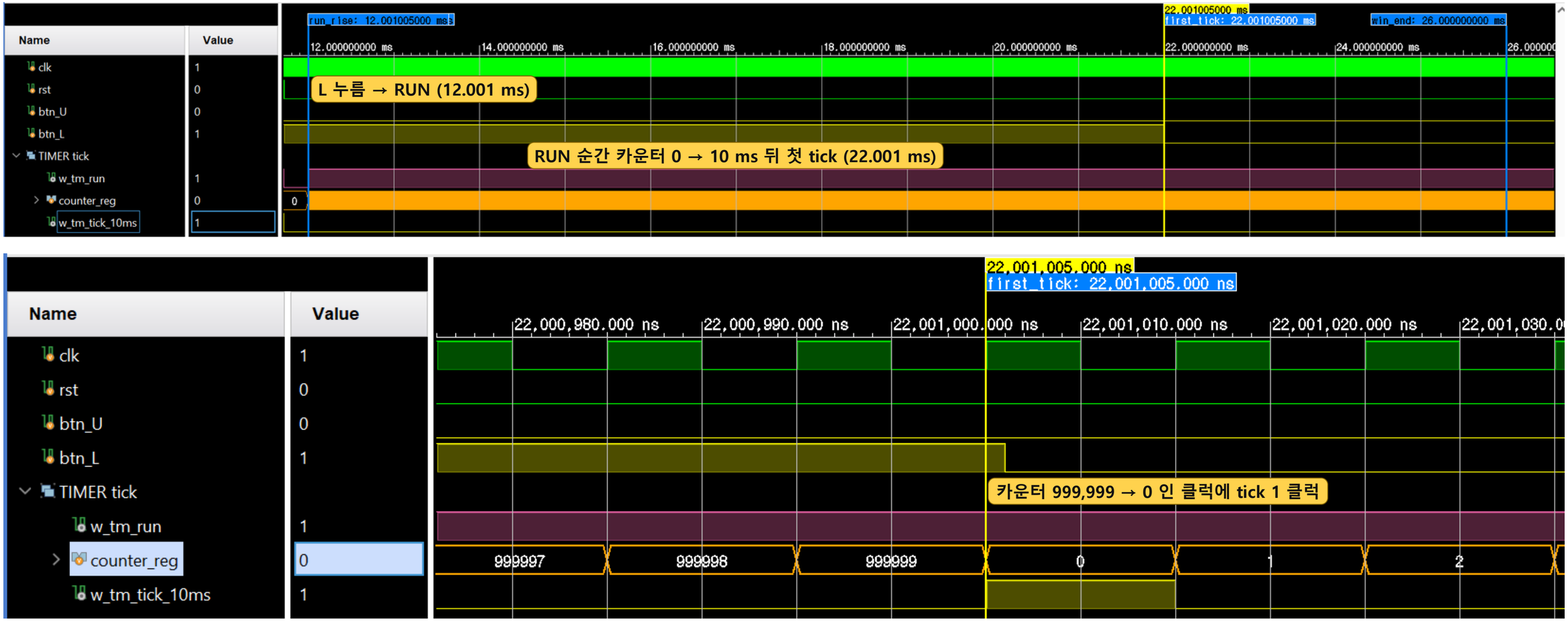


Figure. 타이머 첫 tick (위 : 12 ~ 26 ms, 아래 : 22.001 ms 확대) — Vivado xsim 파형

● 03-5 tick · 타이밍 체크

tick 카운트 체크 — 파형 ② 스톱워치 정지 경계

결론 **정지와 tick 이 같은 클럭(565 ns)에 겹쳐도 다음 클럭에 msec 4 → 5 — 마지막 1/100 초를 셈**

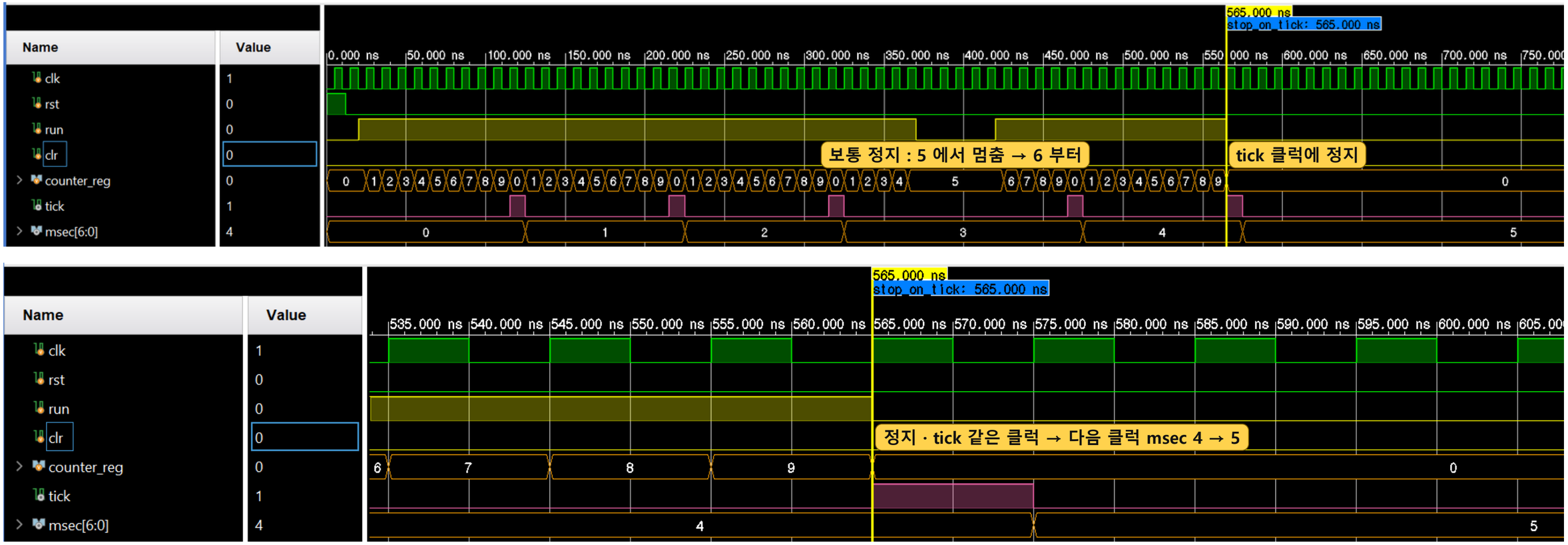


Figure. 스톱워치 정지 경계 (모듈 TB · tick 간격 10 클럭, 위 : 0 ~ 760 ns, 아래 : 565 ns 확대) — Vivado xsim 파형

● 03-5 tick · 타이밍 체크

버튼 누르는 시점 체크 — tb_f3 (tick 기준 0 ~ 9 ms 옮겨 누름)

결론 **누른 시점과 무관하게 스톱워치 항상 100 칸 · 타이머 항상 1,000.000020 ms → 공용 tick 오차 사라짐 (트러블슈팅 1 해결 확인)**

누른 위상 (ms)	0	1	2	3	4	5	6	7	8	9
스톱워치 1.005 s 동작	100	100	100	100	100	100	100	100	100	100
타이머 1 초 (ms)	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002	1000.00002

P3 짧은 탭 × 4 kHz 샘플 위상 → 인식 (O) / 무시 (X)

누름 길이	+30 µs	+90 µs	+160 µs	+220 µs
1.5 ms	X	X	X	X
1.8 ms	X	X	X	O
1.9 ms	X	X	O	O
2.0 ms	O	O	O	O

· 1.5 ms 이하 항상 무시 · 2.0 ms 항상 인식, 그 사이는 샘플 위상에 따라 — 4 kHz × 8 샘플 = 1.75 ~ 2.0 ms 와 일치

판정이 FAIL 11 인 이유 (설계 bug 아님)

· F3 은 **공용 tick 시절** 에 "위상 따라 100 / 101 칸이 둘 다 나와야" · "타이머 ≤ 1 초 + 1 클럭" 으로 짠 TB
· 최종 코드 (기능별 tick, 트러블슈팅 1) 는 **항상 100 칸** → P1 "둘 다 나옴" 판정 FAIL 1
· 타이머는 알람까지 **2 클럭 고정 (20 ns)** → "+1 클럭 이내" 판정 FAIL 10
· 측정값 = 해결 근거, 판정 기준은 팀 TB 라 그대로 둠

콘솔 결과 (xsim 실행 로그 그대로)

```
[10532.126 ms] [LIMIT] P1.F3.SW same 1.005 s measured 10 times: 10 x 1.00 s, 0 x 1.01 s
[11572.127 ms] [LIMIT] P2.F3.TM phase=0ms run->alarm = 1000.000020 ms (err 0.000 ms)   ... phase=9ms 같음
[21154.098 ms] [SUMMARY] F3 checks=57 fail=11 defect=0 limit=25 => FAIL
```

TB: defect_3_press_timing/tb_f3_press_timing.sv (설계 변경 없음 · 누르는 시점만 옮김) · 시뮬 21.2 s (실행 약 3.6 시간) 라 파형 대신 콘솔 표

타이밍 체크 — 정적 타이밍 분석 (구현 후, 100 MHz)

결론 **setup 여유 +3.440 ns · hold 여유 +0.206 ns · 위반 0 — 가장 긴 경로는 타이머 데이터패스 6.301 ns (배선 83 %)**

항목	값	뜻
WNS (setup)	+3.440 ns	10 ns 주기 안에서 가장 늦게 도착하는 신호도 3.44 ns 일찍 도착
WHS (hold)	+0.206 ns	가장 빨리 바뀌는 경로 (스톱워치 분 카운터 1 → 2 비트) 도 hold 안 깨짐
WPWS	+4.500 ns	클럭 High / Low 폭 여유
최장 경로	6.301 ns · LUT 5 단	U_TIMER_DATAPATH sec_reg[4] → msec_reg[0] CE · 로직 1.076 ns (17 %) · 배선 5.225 ns (83 %)
검사 밖 FF	35 개 (no_clock)	4 kHz 펄스를 클럭으로 쓴 디바운서 · FND 스캔 FF → 타이밍 검사 안 받음 (TIMING-17) → 고찰 ①
입출력 지연	미지정 (8 / 28)	버튼 · 스위치 · FND · LED 핀 = 사람 · 눈 속도 → 입출력 지연 제약 안 둠

최장 경로 해석

· 남은 초가 0 인지 판정 (초 · 분 비교) → 1/100 초 카운터 enable — 조합 5 단
· 배선이 83 % → 로직보다 배치 거리 문제
· 여유 3.44 ns = 최대 약 **150 MHz** 까지 같은 구조로 가능 (1 / 6.56 ns, 추정)
· 개선이 필요하면 0 판정을 1 클럭 앞서 FF 로 끊기 (현재 100 MHz 에선 불필요)

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	3.440 ns	Worst Hold Slack (WHS):	0.206 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	325	Total Number of Endpoints:	325	Total Number of Endpoints:	194

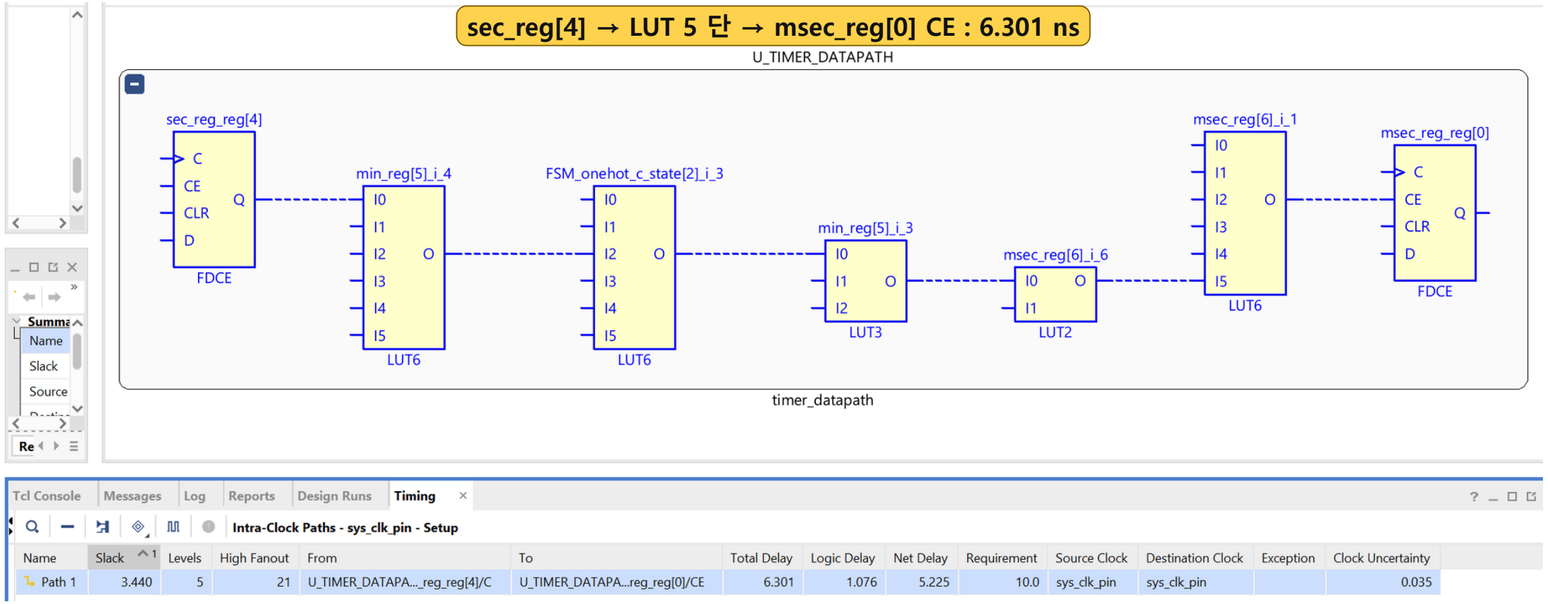
All user specified timing constraints are met.

Vivado 2022.2 · xc7a35tcpg236-1 · 기본 전략 · report_timing_summary (Slow · Fast 코너) · 최종 코드 (동시 입력 무시 포함) LUT 384 · FF 228

● 03-5 tick · 타이밍 체크

타이밍 체크 — 가장 느린 경로

결론 **가장 느린 경로 6.301 ns (주기 10 ns) = 타이머 데이터패스의 초 → 1/100 초 enable, 여유 +3.440 ns**



Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	3.440	5	21	U_TIMER_DATAPA..._reg_reg[4]/C	U_TIMER_DATAPA...reg_reg[0]/CE	6.301	1.076	5.225	10.0	sys_clk_pin	sys_clk_pin		0.035

항목	값
요구 주기	10.000 ns (100 MHz)
데이터 지연	6.301 ns = 로직 1.076 + 배선 5.225
setup 여유 (WNS)	+3.440 ns · 위반 0
hold 여유 (WHS)	+0.206 ns · 위반 0
로직 단수	5 (LUT6 · LUT6 · LUT3 · LUT2 · LUT6)

Figure. Report Timing Summary → Intra-Clock Paths → Setup 첫 경로 회로도 — Vivado 2022.2 구현 후

bug 주입 ① FND 자리 신호 지연 (고스트) — tb_f1

결론 **자리 신호가 늦으면 옆 자리 글자가 비침 — 지연 5 / 30 / 125 µs 모두 검출 (결함 85 건), 지연 0 은 검출 0 · 끄면 바로 회복**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P0	SW15 = 0 (주입기 통과)	원본과 같음	PASS (검사 9)
P1 N = 0	TB 안 지연 모델 0 클럭, 화면 "12.34"	고스트 0	자리마다 0 µs
P1 N = 500	5 µs 지연	옆 자리 글자 비침	AN0 5 · AN1 10 · AN2 5 · AN3 5 µs
P1 N = 3,000	30 µs 지연	비침 시간 비례	AN0 30 · AN1 60 · AN2 30 · AN3 30 µs
P1 N = 12,500	125 µs (한 칸의 절반)	고스트 50 %	AN0 125 · AN1 250 · AN2 125 · AN3 125 µs
P2	RTL 주입기 SW15 0 → 1 → 0 (각 20 ms)	1 구간만 FND ≠ 참조 → 0 이면 회복	**DEFECT 82 → 회복**

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
SW15 켬	—	127,007,010 ns	—
컨트롤러 com 바뀜	font 와 같은 순간	127,506.9 µs	—
핀 com 바뀜	정상 : 같은 순간	127,631.9 µs (+125.0 µs)	**검출**
고스트 시간	0	125 µs (한 칸 250 µs 의 절반)	**검출**

콘솔 결과 (xsim 실행 로그 그대로)

```
[96.006 ms] N=500 (5.000 us skew):
   ghost AN0 5.000 us | AN1 10.000 us
[96.006 ms] N=12500 (125.000 us skew):
   ghost AN0 125 us | AN1 250 us
[127.007 ms] fault_sw15 -> 1
[SUMMARY] F1 checks=13 fail=0
   defect=85 => DETECTED
```

해결 방법: 자리를 바꿀 때 슬롯 앞부분을 블랭킹 (com 전부 끔) · 지연 N 에 비례해 고스트 비율 (500 → 2 %, 3,000 → 12 %, 12,500 → 50 %)

● 03-5 bug 주입 · 시뮬레이션

bug 주입 ① FND 자리 신호 지연 (고스트) — 파형

결론 **SW15 를 켜면 핀 자리 신호가 125 µs 늦게 바뀜 → 그동안 다음 자리 글자가 앞 자리에 찍힘 (고스트)**

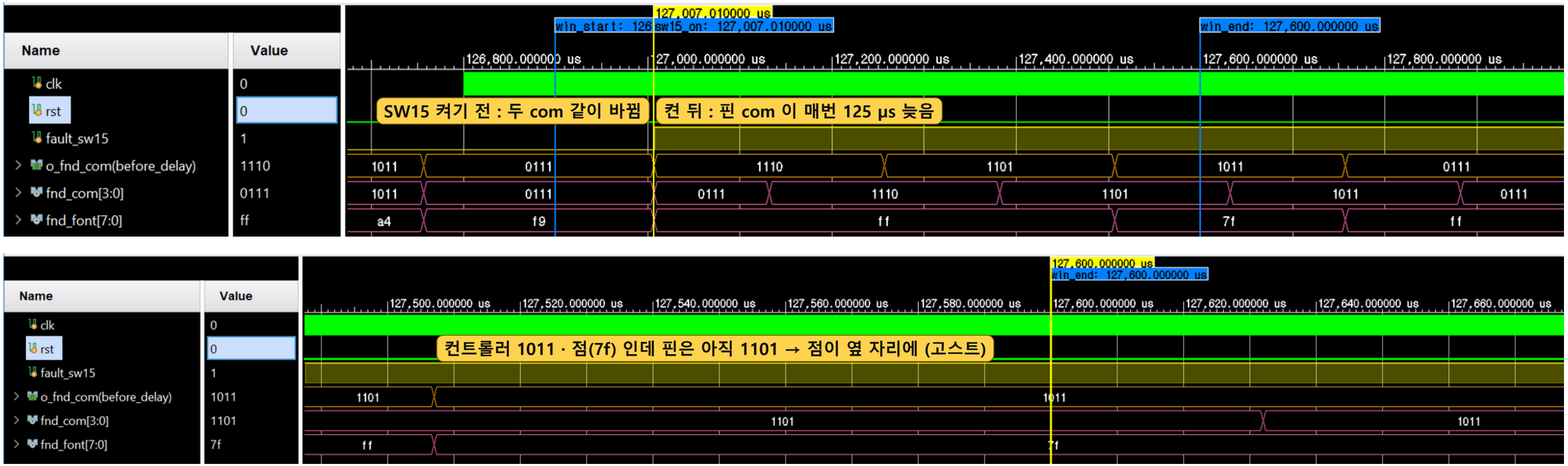


Figure. bug ① FND 자리 신호 지연 (위 : 126.9 ~ 127.6 ms, 아래 : 고스트 구간 확대) — Vivado xsim 파형

● 03-5 bug 주입 · 시뮬레이션

bug 주입 ② 버튼 끊김 · 잡음 — tb_f2

결론 **누른 채 250 µs 이상 끊기면 위상과 무관하게 두 번 인식 — 끊김 4 번 → 펄스 5 개 (시계 분 +5) 검출, 안 누른 핀 잡음은 0 개**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P1	안 누른 btn_U 핀에 스파이크 200 ms	펄스 0 개	0 개 · 견고
P2	튐 패턴	1 개	1 개
P3	100 ms 누름 + 300 µs 끊김 × 4	끊김마다 새 누름 → 5 개	**5 개 · 시계 분 0 → 5 · 타이머 +50 초**
P5	RTL 주입기 SW15 = 1 (끊김 · 잡음)	누름 연타로 검출 → 끄면 1 개	**5 개 (자동 반복처럼)**

P4 끊김 폭 × 샘플 위상 → 펄스 수

끊김	+30 µs	+90 µs	+160 µs	+220 µs
50 µs	1	1	1	2
100 µs	1	1	2	2
200 µs	1	2	2	2
250 µs	2	2	2	2
300 µs	2	2	2	2

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
끊김 시작	—	297,264,010 ns	—
4 kHz 샘플이 0 읽음	끊김 300 µs > 샘플 간격 250 µs	≈ 297.51 ms (처음부터)	**검출**
두 번째 o_btn	8 샘플 다시 채움 (1.75 ~ 2 ms)	299,512,005 ns	**검출**
결과	정상 : 1 번	끊김 4 번 → 5 번 (분 0 → 5)	**검출**

콘솔 결과 (xsim 실행 로그 그대로)

```
[382.264 ms] P3 one long press -> 5 pulses, watch min 0 -> 5
[477.265 ms] P3 timer U -> 5 pulses, timer = 00:50.00
[SUMMARY] F2 checks=44 fail=0 defect=12 => DETECTED
```

원인: 디바운서가 누를 때만 8 샘플 확인, 뗄 때는 1 샘플 → 해결: 뗄 때도 8 번 연속 0 (히스테리시스) = 결론 · 고찰 ② · 트러블슈팅 2 와 같은 디바운서

bug 주입 ② 버튼 끊김 · 잡음 — 파형

결론 **300 µs 끊김을 샘플이 읽으면 처음부터 다시 셈 → 2 ms 뒤 두 번째 입력 (한 번 누름이 5 번으로)**

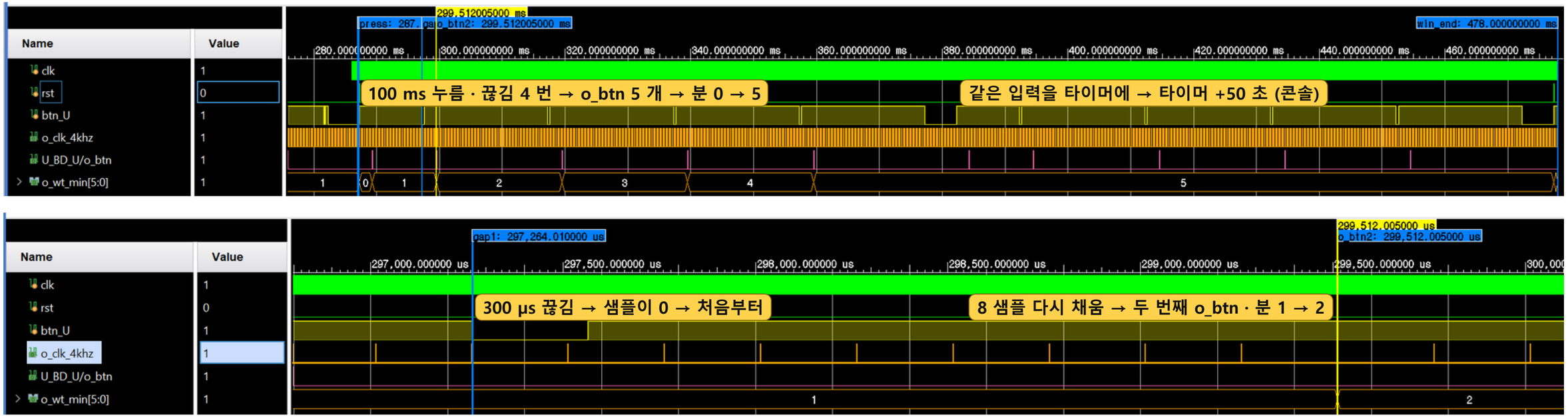


Figure. bug ② 버튼 끊김 (위 : 287 ~ 478 ms, 아래 : 첫 끊김 확대) — Vivado xsim 파형

bug 주입 ③ 리셋 핀 글리치 — tb_f4

결론 **리셋 핀에 0.5 ns 글리치만 들어와도 전부 초기화 (시계 09:41 → 12:00, 스톱워치 · 타이머 정지) — 5 번 모두 검출, LED 표시 없음**

Phase	자극 (TB 가 하는 것)	기대값 (Expectation)	시뮬레이션 결과
P0	SW15 = 0 (주입기 통과)	원본과 같음	PASS (검사 9)
P1	시계 09:41 · 스톱워치 RUN · 타이머 RUN 00:49.99 중 rst 핀 글리치 0.5 ns (클럭과 무관한 시점)	상태 유지돼야 함	**전부 초기화 → DEFECT**
P1	같은 글리치 1 / 5 / 10 ns	상태 유지	**매번 DEFECT**
P2	RTL 주입기 SW15 0 → 1 (내부 리셋 1 클럭)	상태 유지 · 고장 표시	**DEFECT** · LED 8005 (표시 없음)

타이밍 분석 — 예상 vs 실측 (파형 : 다음 장)

사건	예상	실측	판정
글리치	—	115,007,062.3 ns · 폭 0.5 ns	—
초기화 시점	비동기 리셋 : 펄스 순간 바로	062.3 ns (클럭 엣지 전)	**검출**
결과	정상 : 영향 없어야	시계 9:41 → 12:00 · 스톱워치 · 타이머 정지 · 0	**검출**

콘솔 결과 (xsim 실행 로그 그대로)

```
[115.008 ms] [DEFECT] P1.F4.W0
   glitch width=0.5ns at 115.007 ms:
   watch 09:41:00.04->12:00:00.00
   sw 00:00:00.33->00:00:00.00
   timer 00:49.99->00:00.00
[SUMMARY] F4 checks=26 fail=0
   defect=5 => DETECTED
```

원인: 비동기 리셋은 폭과 상관없이 FF 를 지움 → 해결: 리셋 동기화기 (비동기 assert · 동기 release) + 최소 폭 필터 · 처음 -debug off 로 돌렸을 때 판정식이 잘못 계산돼 PASS → 기본 설정 재실행

● 03-5 bug 주입 · 시뮬레이션

bug 주입 ③ 리셋 핀 글리치 — 파형

결론 **0.5 ns 펄스에도 클럭 엣지를 기다리지 않고 바로 전부 초기화 (비동기 리셋)**

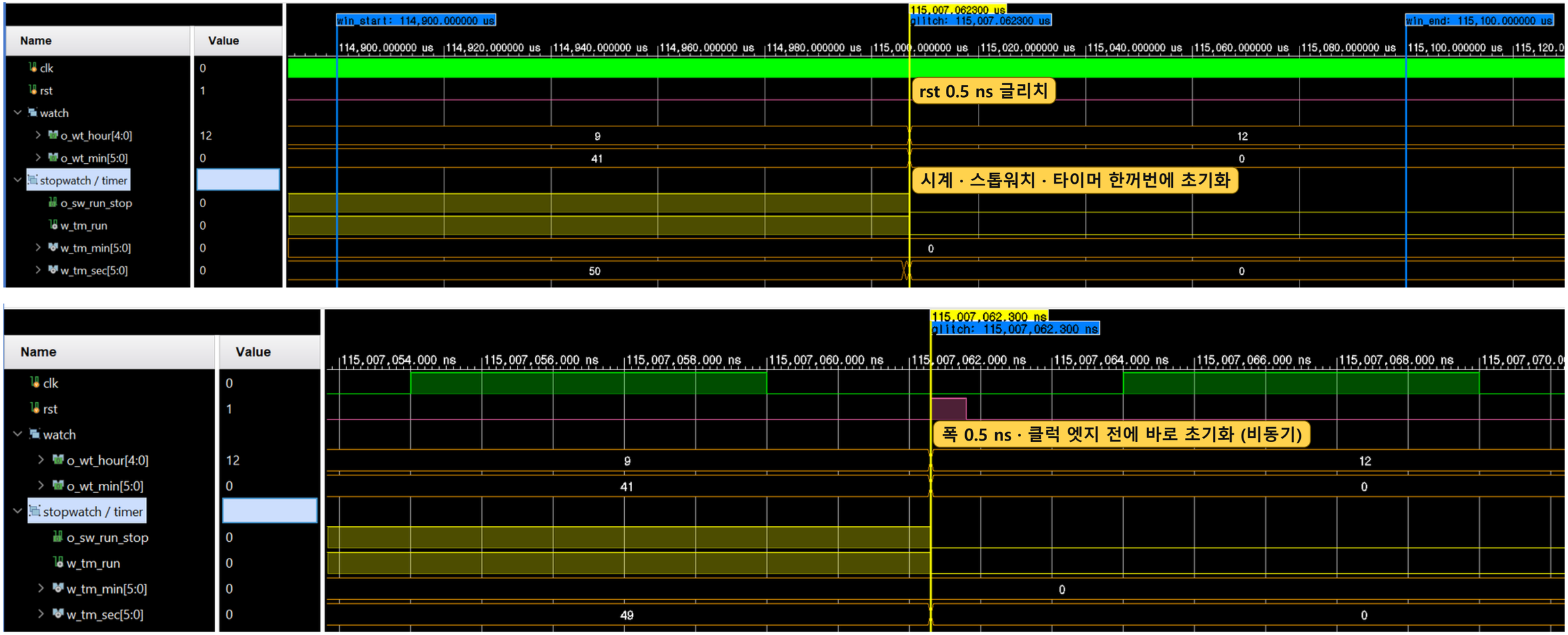


Figure. bug ③ 리셋 핀 글리치 (위 : 114.9 ~ 115.1 ms, 아래 : 0.5 ns 펄스 확대) — Vivado xsim 파형

04

트러블 슈팅

분석 순서
① 현상 → ② 원인 → ③ 해결 → ④ 적용 결과 (Before / After)

1. 10 초 타이머가 9.998 초에 울림

100 Hz tick 공유 → 기능별 tick 분리

2. 한 번 눌렀는데 두 번 인식

디바운서 샘플 1 MHz → 4 kHz

트러블 슈팅 1 — 10 초 타이머가 9.998 초에 울림

① 현상 ② 원인 ③ 해결 ④ 적용 결과

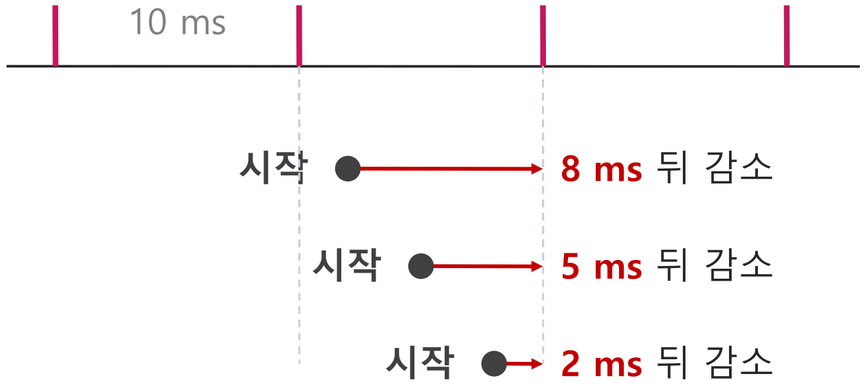
① 현상 · ② 원인 · ③ 해결 **공유 100 Hz tick 이 시작과 무관 → 10 초가 9.998 초 → 100 Hz 만 기능마다 따로 두어 매번 10 ms**

① 현상 기대 10.000 s → 실제 9.998000010 s

· 누르는 시점에 따라 **9.990 ~ 10.000 s** (매번 다름, 오차는 첫 감소 1 번뿐)
· 시계: "시작" 이 없어 박자와 무관
· 스톱워치 · 타이머: **누른 순간부터 10 ms** 를 세야 함

② 원인

공유 tick (리셋부터 항상 10 ms 박자)



③ 해결 ④ 채택 : 매번 정확히 10 ms + 수업 방식 ① 보다 LUT −31 · FF −64 (② 보다 FF +42)

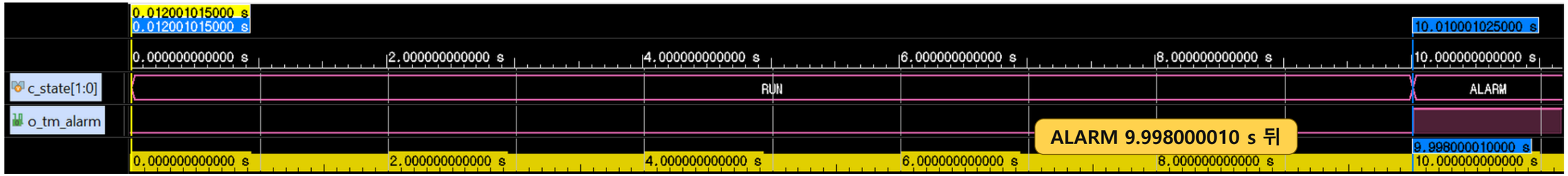
판	구조	LUT	FF	WNS	시작 → 첫 tick
①	수업 방식: 디바운서마다 분주 + 데이터패스마다 tick	415	292	+3.28	매번 10 ms
②	기존 팀 판: 4 kHz 공유 + 100 Hz 1개 셋이 공유	321	186	+2.42	**최대 10 ms 일찍**
④	**4 kHz 공유 + 100 Hz 기능마다 따로 (RUN 순간부터)**	**384**	**228**	**+3.44**	**매번 정확히 10 ms**

근거 tb_tm_2 P1 9,998 ms · 시작 시각을 3 ms 씩 옮긴 TB (tb_phase_check) 에서 첫 감소 8 / 5 / 2 ms

트러블 슈팅 1 — 적용 결과

④ 적용 결과 **9.998000010 s → 10.000000020 s : 매번 10 s + 2 클럭 · 재시작 · 정지 경계 TB 2 건도 FAIL → PASS**

Before



After




```
# 잔여 2건 — 같은 TB 를 고치기 전 / 고친 뒤 소스로 (콘솔 요약)
[before] 알람 뒤 재시작 → 첫 tick = 9999980 ns   FAIL
[before] tick 클럭에 STOP → msec = 4 (10 ms 잃음)  FAIL
[after ] 알람 뒤 재시작 → 첫 tick = 10000000 ns
[after ] tick 클럭에 STOP → msec = 5   TOTAL MISMATCH = 0
```

고친 것 알람 때도 tick 카운터를 0 으로 · 스톱워치 데이터패스의 중복 조건 삭제

트러블 슈팅 2 — 한 번 눌렀는데 두 번 인식

① 현상 ② 원인 ③ 해결 ④ 적용 결과

④ 적용 결과 같은 튀는 입력 20 번 → 펄스 158 개 → 20 개 (샘플 1 MHz → 4 kHz, 2 ms 연속 눌림만 인정)

① 현상 한 번 눌렀는데 여러 번 인식



1 MHz 판 o_btn (창 8 µs)



4 kHz 판 o_btn (창 8 × 250 µs = 2 ms)



그림은 개념도 (가로 1 칸 ≈ 1 ms)

② 원인

1 MHz 샘플 8 번 = 8 µs 창 → 접점 튐 조각마다 "눌림" 으로 인정

③ 해결

샘플 4 kHz → 8 번 = **2 ms 연속** 눌려야 인정

같은 4 kHz 로 FND 스캔 (8 칸 × 250 µs = 2 ms 한 바퀴) → 분주기 1 개 공유

④ 결과	Before 1 MHz	After 4 kHz
시뮬: 같은 튀는 입력 20번 → o_btn 개수	**158개 (20번 모두 틀림)**	**20개 (모두 1번)**
대가	반응 8 µs	반응 1.75 ~ 2 ms (못 느낌)

튐 모델: 누를 때 · 뗄 때 20 ~ 400 µs 조각 4 ~ 11 개 (실제보다 심하게) · Icarus Verilog 시뮬 (tb_bounce)

05 결론 · 고찰

구현 결과 — 자원 사용률 · 모듈별 자원

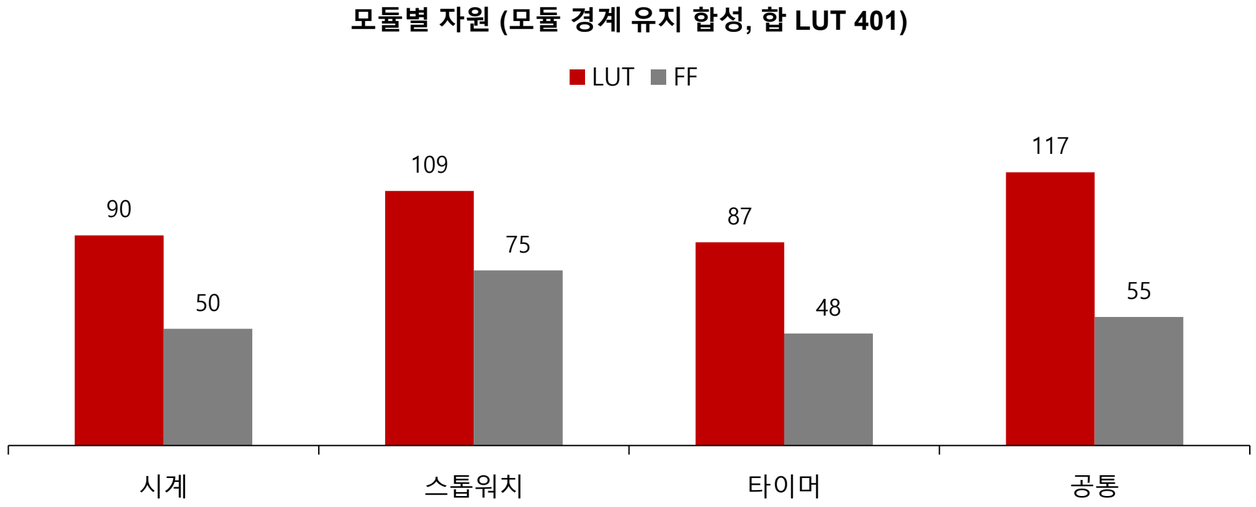
결론 **칩의 LUT 1.8 % · FF 0.5 % 사용, 타이밍 여유 +3.44 ns — FF 의 28 % 는 100 Hz 분주 카운터 3 개**

LUT 384 / 20,800 (1.8 %)

FF 228 / 41,600 (0.5 %)

WNS +3.440 ns (100 MHz 한 주기 10 ns 중 여유)

래치 0 · 기본 합성 (실제 비트스트림 기준)



항목	자원	근거 · 해석
타이머 (내 담당)	LUT 87 · FF 48	시계(90)와 비슷, 스톱워치(109)보다 작음 · 단 **쓰지 않는 hour_reg 5 비트** 남음 → 지우면 LUT −6 · FF −5 (측정)
전용 tick 2 개 (TS 1)	LUT 62 · FF 42	"누른 순간부터 10 ms" 의 비용 — 21 비트 카운터 (0 ~ 999,999) × 2
100 Hz 분주 3 개	FF 63 (28 %)	FF 를 가장 많이 먹는 것은 기능이 아니라 10 ms 를 세는 카운터
랩 (스톱워치)	LUT 26 · FF 25	멈춘 시각을 통째로 저장하는 레지스터 + 화면 선택 MUX

Vivado 2022.2 · xc7a35t · 구현(route) 후. 게이지 = 기본 합성 · 모듈별 = -flatten_hierarchy none (경계 유지라 LUT 17 개 많음)

결론 · 고찰

결론 **세 기능 목표 전부 동작 — Top TB 13 개 검사 447 개 PASS · 보드 시연 · 10 초 타이머 오차 20 ns**

기능	목표	Top TB	보드
스톱워치	시작 · 정지 · 지우기 · 업다운 · 랩 · 자정	**53 PASS**	**확인**
시계	12:00 시작 · 설정 · 점멸 · 12/24 표시	**264 PASS**	**확인**
타이머	더하기 · 59:59 상한 · 10 s · 알람	**73 PASS**	**확인**
공통	디바운서 · FND 스캔 · 동시 입력 무시	**57 PASS**	**확인**

결론 (객관 · 수치)

· 컨트롤 유닛(언제) + 데이터패스(얼마나) 구조로 시계 · 스톱워치 · 타이머를 같은 틀로 구현

· 버튼 → FND 전체 경로를 100 MHz 그대로 시뮬레이션 — **447 개 검사 모두 기대값과 일치**

· 1/100 초 = **10.000 ms**, 10 초 타이머 = 10 s + 20 ns (2 클럭 고정)

· LUT 384 (1.8 %) · FF 228 (0.5 %) · WNS +3.440 ns · 래치 0

고찰 — 아쉬운 점 · 원인

① 4 kHz 펄스를 클럭으로 써서 플립플롭 35 개 타이밍 검사 못 받음 (TIMING-17)

② 디바운서는 누를 때만 8 번 확인 → 누른 채 잠깐 끊기면 두 번 인식 가능

③ 타이머에 쓰지 않는 hour_reg (5 비트) 남음

④ 23:59:59 같은 긴 경계는 초기값을 넣어 확인

개선 방향 · 향후 계획

① 4 kHz 를 enable 로 → 비교판에서 경고 0 개 확인

② 뗄 때도 8 번 연속 0 확인

③ hour_reg 삭제 → LUT −6 · FF −5 (측정)

④ 미니프로젝트 2 (센서 · UART) 에 같은 CU/DP 구조 · 공통 tick 재사용

06 시연 영상

시연 ① 스톱워치 — 보드 동작 (GIF 자동 반복)

결론 **보드에서 경계 · 예외 동작 5 가지 모두 시뮬레이션과 같음**













59.99 → 1 분 올림	LAP 중에도 내부 계속	동작 중 L 무시	다운 0 → 23:59:59.99	업 23:59:59.99 → 0
같음	같음	같음	같음	같음

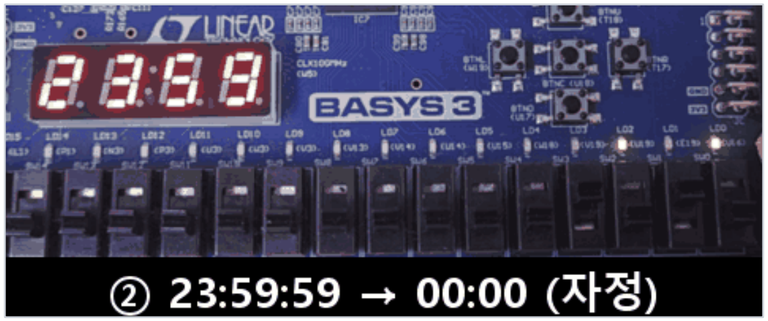
GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

06 시연 영상

시연 ② 시계 — 보드 동작 (GIF 자동 반복)

결론 **보드에서 자정 · 12 시간 표시 · 설정 점멸 · 자리별 ± 모두 시뮬레이션과 같음**











23:59:50 → 10 초 뒤 자정	12 시간 표시 0 시 = 12	설정 자리 0.5 s 점멸	그 자리만 ±
같음	같음	같음	같음

GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

06 시연 영상

시연 ③ 타이머 — 보드 동작 (GIF 자동 반복)

결론 **보드에서 0 시작 거부 · 상한 · 취소 · 백그라운드 알람 모두 시뮬레이션과 같음**















보드에서 확인한 것	시뮬
0 에서 시작 안 함	같음
60 초 = 1 분	같음
최대 59:59	같음
동작 중 L = 취소	같음
다른 화면에서도 알람	같음
버튼 = 알람만 끔	같음

GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

Q & A

감사합니다

부록 A — 검증 전체 : TB 13 개 · 검사 447 개 PASS

결론 **Top TB 13 개 · 검사 447 개 모두 PASS — 최종 코드로 다시 실행**

폴더	TB	확인하는 것	검사	결과	시뮬 시간
top	tb_top_1	디바운서: 누름 시간 · 바운스	24	PASS	2,205 ms
top	tb_top_2	FND 스캔 · 자리별 출력 · 점	22	PASS	2,180 ms
top	tb_top_3	버튼 동시 입력: 세 기능 모두 무시 (10/18)	11	PASS	302 ms
스톱워치	tb_sw_1	시작 · 정지 · 지우기 · 동작 중 무시 · 동시 R+L	23	PASS	2,290 ms
스톱워치	tb_sw_2	올림 · 자정 순환 · 다운 빌림	15	PASS	485 ms
스톱워치	tb_sw_3	랩 고정 · 해제 · 지우기	15	PASS	1,230 ms
타이머	tb_tm_1	더하기 · 올림 · 59:59 상한	21	PASS	455 ms
타이머	tb_tm_2	실제 10 초 · 알람 · 빌림	41	PASS	11,597 ms
타이머	tb_tm_3	취소 · 백그라운드 · 화면 우선순위	11	PASS	547 ms
시계	tb_wt_1	설정 UI · 커서 · 경계	38	PASS	874 ms
시계	tb_wt_2	1초 정확도 · 자정 올림	13	PASS	2,245 ms
시계	tb_wt_3	12/24 표시 · 점멸 · LED	118	PASS	6,050 ms
시계	tb_wt_4	설정 중 R · L · U · D 동시 누름 16가지	95	PASS	480 ms

최종 코드(동시 입력 무시 포함)로 13 개 전부 다시 실행 · xsim 배치 (10/18)

부록 질문 대비

부록 B — 버튼 두 개를 같은 순간에 누르면? → 모두 무시

결론 **같은 클럭에 버튼 펄스 2개 이상 → 그 클럭 입력 모두 버림 (세 기능 같은 규칙, 10/18)**

화면 · 상태	같은 클럭에 동시 입력	전 (기능마다 우선순위)	지금 (10/18)
스톱워치 STOP	R + L	RUN 시작 + 0 으로 지움	**무시 · STOP · 값 유지**
시계 설정	L + R	L 우선 (커서 왼쪽)	**무시 · 커서 그대로**
시계 설정	U + D	무시 (10/18 오전 수정)	**무시 · 값 그대로**
타이머 SET	U + R	U 만 (+10초)	**무시 · 00:00 그대로**
타이머 SET	L + D	L 만 (시작)	**무시 · 시작 안 함**
버튼 하나만	L · U 등	보통 동작	**보통 동작 (그대로)**

어디서 Top 의 버튼 게이팅: w_btn_multi = 펄스 두 개 이상 → 12개 게이팅 AND 에 ~w_btn_multi (컨트롤 유닛 · 랩까지 한 번에)
확인 tb_top_3 (세 기능 동시 입력 · 검사 11개) · tb_sw_1 P6 · tb_wt_4 (시계 16가지) — 고치기 전 소스로 돌리면 tb_top_3 FAIL 7개

같은 클럭이 되는 조건: 디바운서 4 kHz(250 µs) 샘플 → 두 버튼이 250 µs 안에 같이 안정될 때만

부록 C — 합성 · 구현 전략 비교 (PPA) → 기본값 유지

결론 **자원 1.8% · WNS +3.44 ns · 0.106 W 로 셋 다 여유 → 제약이 없어 기본값 유지 (Area 는 LUT −13.5% ↔ WNS −0.84 ns)**

전략 (Vivado 2022.2)	LUT	FF	WNS @ 100 MHz	전력 (추정)
기본값 (Vivado Defaults) — 채택	384	228	+3.440 ns	0.106 W
합성 Area (Flow_AreaOptimized_high)	332 (−13.5%)	227	+2.600 ns	0.115 W
합성 Area + 구현 Area_Explore	336	227	+2.011 ns	0.114 W
성능 (PerfOptimized_high + Performance_Explore)	384	228	+3.440 ns	0.106 W
전력 (Power_DefaultOpt, power_opt_design)	390	228	+3.733 ns	0.104 W

왜 기본값 전략은 빡빡한 제약 쪽을 고르는 것 — 이 설계는 칩의 1.8%, 타이밍 실패 경로 0, USB 전원이라 셋 다 제약이 없음. Area 를 고르면 얻는 것 없이 타이밍 여유만 줄고, Power 효과는 추정 오차 안 (전체 0.106 W 중 0.072 W 는 칩 누설)
네 판 모두 같은 경향 트러블 슈팅 1 표의 ① ~ ④ 에 같은 전략을 적용해도 Performance 는 전부 기본값과 동일, 합성 Area 는 LUT −7 / −28 / −26 / −52 에 WNS 전부 감소, Power 는 0 ~ −0.002 W

최종 코드 · 같은 xdc · xc7a35t · 구현 후 · 전력은 Vivado 추정(신뢰도 Low) · 배치 실행